GE VERNOVA

ADMS

16.25.1

Architecture Overview

Copyright

Change History

Date	Change Description
May 29, 2023	ADMS 3.16: • Updated Plug-In Components. • Updated references to "Storm Assist" to new product name "Outage Assist". • Editorial and formatting updates.
Oct 20, 2023	ADMS 16.1.0: • The product version numbering has changed. The prefix '3' is no longer included in the version numbering for ADMS.
Apr 4, 2024	ADMS 16.7.0: • Converted document to new format.

Contents

1. Architecture Background

1.1. Solution Background

GE is a leading vendor in the Energy Management System (EMS) and Market Management System (MMS) markets. When it decided to expand to the Distribution Management System (DMS) market, it started by offering a solution based on GE's Control application. It had some success in Europe, but mainly on small-scale projects. When it started to look into the U.S. market and larger-scale projects, it realized that a different architectural approach was needed. GE formed a team with broad project experience in distribution, SCADA, advanced analytical applications, user interface development, three-phase power flow, and IT infrastructure, to look into a new DMS solution.

1.2. Significant Driving Requirements

DMS and Outage Management System (OMS) are key software systems that address the operational needs of distribution utilities. There is a very wide spectrum of requirements due to the diverse regulatory structure and the distribution network design.

The following list highlights some of the requirements that have architectural significance for GE's product design:

- The solution must have the ability to solve a large electric power distribution network (possibly in excess of three million buses).
- Because changes in the distribution network happen continually, the model update process should allow incremental updates with no need to bring the system down or even fail over to a standby server.
- The solution needs to scale from large utilities to small cooperatives. It must be able to meet integration requirements in utilities with large IT shops and in organizations with minimal support staffs.
- The solution has to maintain high performance under stress conditions or major events such as hurricanes.
- The solution has to handle a large number of concurrent users (a large number of user interface clients).
- The solution must integrate DMS, OMS, AMI, and Scada functions.
- The solution must have a unified user interface that provides effective visualization and user interactions.
- With the push of smart grid technologies, the solution needs an optimization engine that can be extended to build advanced analytical applications.

1.3. Architectural Approaches

1.3.1. Solve a Large Distribution Network

When solving a large electric power distribution network (possibly in excess of three million buses), an important concept is determining how to accurately solve smaller portions of the distribution network. For example, the entire distribution network does not need to be solved if a switch open/close status on one radial distribution feeder has changed.

To take full advantage of multiple processors, it is also necessary to break the solution of the full model into smaller pieces. The concept of a study area, or network selection, is introduced to partition the distribution network into smaller pieces and be solved in each network application.

1.3.2. Model Management

Many distribution utilities have already invested heavily in an asset management system (GIS) or customer database (CIS). Many organizations already have a data maintenance process in place for those systems. GE's solution cannot force duplicate data entry if information has already been stored elsewhere. The Model Management System is designed to allow exchanging model information with external systems in a standard format (for example, IEC/CIM), supplemented with editors for entering missing pieces.

1.3.3. Online Model Update

A key requirement is the ability to apply model changes without disrupting operations. Applications need to have a smooth transition when applying changes. The design of the model partition affects the granularity of changes that can be applied. A model built on a per-station basis provides a logical partition that can be easily understood and implemented.

1.3.4. Integration

The solution should be an integrated solution. The operation of and information in DMS, OMS, and Scada functions should work in harmony and flow seamlessly. GE's design effort has been focused on the integration and interaction of the functional modules, to ensure that they work as one system.

1.3.5. User Interface

A user interacts with the system via the UI client. An integrated solution should have a unified user interface that allows a user to navigate efficiently, to view information about and perform operations on different functions, including SCADA, incident management, switching, and analytical

studies.

The UI client integrates with Browser. It has a rich client version that can offload the server resource loading when large numbers of concurrent users are connected to the system. It also allows operators the ability to view the system and perform meaningful studies even if the client is temporarily disconnected from the system. The UI client supports remote (non–control room) operators access using virtual desktop technology such as Citrix. Finally, the UI client shares the same authentication and authorization mechanism across all functions.

1.3.6. Platform Choice

GE's DMS solution supports the Microsoft Windows operating system.

1.3.7. Security

NERC's jurisdiction as it relates to the CIP Standards is currently confined to transmission by law. From a pure compliance perspective, a utility does not have a legal obligation to put distribution assets on the Critical Asset regulatory list. However, this is going to change.

To anticipate the change, GE aims to deliver a DMS/OMS solution that helps distribution utilities be NERC CIP compliant. Changes are being made in some areas to remove known issues to NERC CIP compliance.

1.4. Product Line Reuse Considerations

The software modules have been developed to support both product vision and project deliveries. The following were some of GE's considerations and technology choices:

- Product variants: Not all DMS projects require OMS functions. The Distribution Operator Training Simulator (DOTS) is an optional product. Interfacing with GIS, CIS, and AMI from different vendors is supported.
- Integration with Browser: Browser is used in a different product line and it has a different release cycle. This might affect when the ADMS client can adapt to new technology.
- Sharing applications or developing new solutions: Some ADMS functions have similar requirements as EMS or MMS but, at the same time, there are enough differences that warrant a different approach (for example, trending and archiving).

2. High-Level Design

2.1. Functional Overview

ADMS is a collection of major functions that work together to fulfill the operational needs of a distribution utility.

A conceptual view of the major functional components that make up ADMS is shown in the following Figure.

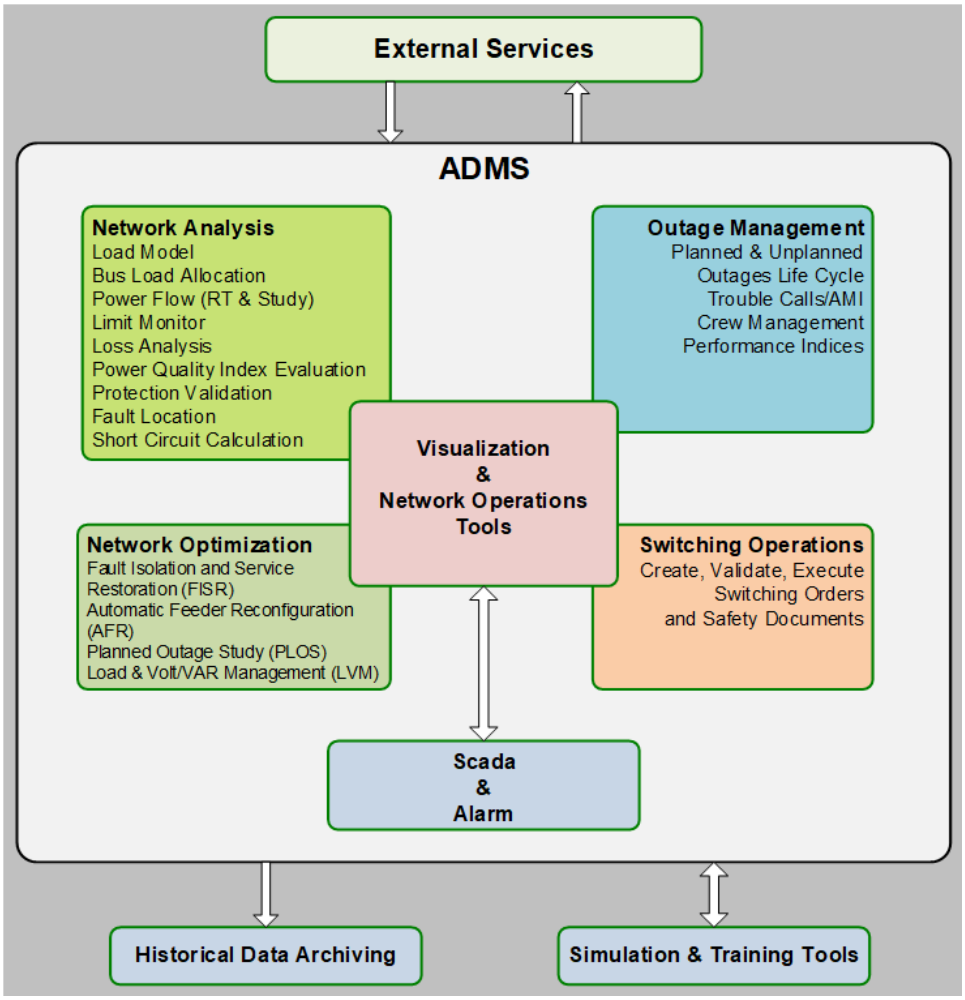


Figure 1. Conceptual View of ADMS

The following sections give brief descriptions of the major functions.

2.1.1. Network Analysis

The Distribution Network Analysis Functions (DNAF) make up an integrated package of applications

that can be executed in real-time and study modes. The memory resident Distribution Network Object Model (DNOM) provides the connectivity and attributes data required by the DNAF.

In real-time mode, the DNAF use the current power system conditions (SCADA measurements, capacitor time of operation schedules, load schedules) to evaluate the system. The DNAF can be set up to run periodically, by prespecified events, or on operator demand. Event triggers include the following:

- Topology changes (both inside and outside the station fence), including automated and manual switching device status changes, and the insertion and removal of temporary modification (cuts and jumpers).
- Sudden feeder load or feeder head voltage changes obtained from SCADA. If the change in the feeder head current flow or voltage is greater than a user-specified value (in amps or volts), the DNAF are triggered.

In study mode, the DNAF solve operational studies using data retrieved from a saved case or a real-time snapshot of the system. The future operating point for studies is estimated using typical load shapes. The load shape information is based on the study time and day type. In study mode, the DNAF are manually executed.

The DNAF suite applications include:

- Load Model and Estimation: Estimates the loads based on the current time, day type, and season.
- Bus Load Allocation (BLA): Provides estimates for the loads (kW and kVAR) at the distribution feeder nodes. The BLA is used by all of the DNAF.
- Distribution Power Flow (DPF): Calculates the complex voltages at all nodes and the power flows through all feeder segments in the distribution system.
- Limit Monitor (LIM): Checks for branch flows and voltages that are outside of acceptable operating limits.
- Loss Analysis (LA): Calculates instantaneous technical losses for conductors, transformers, and feeders.
- Power Quality Index Evaluation (PQI): Determines the extent of feeder voltage violations, including voltage imbalance.
- Protection Validation (PRV): Calculates the maximum and minimum short circuit current for a phase-to-phase or phase-to-ground fault at any selected points in the distribution network. The calculated fault currents are then used to verify that each segment of the feeder is properly protected.
- Fault Location (FL): Uses feeder breaker fault amp measurements and faulted phase status (target) measurements to determine potential fault locations along the distribution feeders.
- Short Circuit Calculation (SCC): Calculates single phase-to-ground, double phase-to-ground, phase-to-phase, and three-phase currents for each selected location (power transformer, line, and switch device). Available only in study mode.

2.1.2. Network Optimization

The optimization-based network analysis applications are extensions of the DNAF suite. The goal is to provide additional tools to generate plans that improve the efficiency and economics of the distribution system operations.

The core of these applications is a solution engine referred to as the "Distribution System Plan Generation Engine" (DSPGE). The Plan Generation Engine's three-phase algorithm is a combination of Linear Programming (LP) and heuristic switching techniques using ranking indices. The algorithm can be used for a variety of purposes by activating selected constraints and controls and by applying different optimization objective functions.

The optimization functions include:

- Fault Isolation and Service Restoration (FISR): Generates switching sequences to isolate faulted sections and to restore service to the non-faulted sections.
- Fault Isolation and Service Restoration Contingency Analysis (FISRCA): Executes a FISR analysis by sequentially placing a fault on each switchable section in a given substation to determine the availability of a valid restoration plan. Available only in study mode.
- Automated Feeder Reconfiguration (AFR): Provides plans to unload overloaded devices and to optimize the location of normally open points.
- Planned Outage Switching (PLOS): Generates switching plans to support planned outages. This study-only tool together with switching management can help to fully manage planned outages.
- Load and Voltage/VAR Management (LVM): Optimizes and coordinates capacitors, voltage regulating devices, generators, and distributed energy resources to provide reactive power support and to maintain system operation within voltage limits. The LVM application can be used as a least-intrusive demand management tool.

The optimization applications can run in closed loop mode, semi-automatic, or advisory mode. When an application runs in advisory mode, the generated plan is stored in the DNOM for reference; the operator can check plan steps and then implement the plan by issuing controls manually. In semi-automatic mode, output commands are sent after operator approval. In closed loop mode, the generated plan is executed automatically without an operator's intervention.

2.1.3. Outage Management

Network outage management is a trouble call and outage management system that forms an integral part of the ADMS suite. It allows operators to manage scheduled and unscheduled outages from within a unified operating environment that integrates real-time DMS, SCADA, crew monitoring, and switching orders.

Network outage management provides dispatchers with the means to centrally collect and coordinate all possible information about network outages. In case of unplanned outages, customer calls, Advanced Metering Infrastructure (AMI) observations, SCADA data, and emergency

services calls are all analyzed to assist in determining the most likely point at which a customer supply has been interrupted.

From initial notification of a fault, through prediction, crew assignment, and fault isolation, to return-to-normal switching, the dispatcher is able to work from a single set of network views. All of the necessary information for each phase of the job is clearly presented in a way that allows the dispatcher to manage each outage efficiently while also staying aware of other network activity.

One design criteria is to handle major storm situations by providing high-performance and effective tools combined with the ability to easily divide or combine areas of responsibility and therefore balance the dispatcher's workload. A complete record of all actions taken in the restoration process is recorded for each incident. Network reliability and performance indices are automatically calculated.

2.1.4. Switching Operations

Switching operations consist of two main components: switching orders and safety documents.

Switching orders can also be generated automatically by the network optimization functions. A switching order encompasses both planned and unplanned switching. The system is automated and paperless wherever possible. The switching operations functionality covers the complete scope of the switching process, allowing users to:

- Generate requests and submit switching orders directly from the network geographic view.
- Execute individual steps manually (field staff performs the switching) or automatically (the operator directly performs a SCADA operation).
- Manage the execution of the switching order by verifying that each step has been correctly completed.
- Simulate the execution of the switching order against a study model of the network, based on projected conditions at the planned time of switching.
- Save and retrieve switching orders for record keeping and later use.
- Automatically create "Return to Normal" switching steps to reduce the work required to return the network to its original state.
- Modify partially executed switching orders in the case of equipment failure or an unexpected problem.
- Support a daily log of unplanned switching steps.

A number of different types of safety documents can be defined. A safety document creates a list of the tags that are to be placed as part of the switching process that isolates the required section of the network. The issuing of a safety document is confirmed by selecting each tag on the network view to verify that the tags have been correctly placed. In this way, the issuance and release of safety documents are interlocked with the execution of the related switching orders.

2.1.5. User Interaction and Visualization

Providing a unified user interface for distribution operations is the foundation of integrating the underlying functionalities. The user interface is designed to provide a consolidated view of network conditions. It should support distribution operators who perform day-to-day routine tasks as well as handle emergency situations resulting from major outages.

Visualization capabilities include the geographic and schematic views, tabular displays, area world view, pop-up dialog boxes, and navigating, panning, zooming, and decluttering features. From the ADMS user client, the operator should be able to:

- View network topology and device energization statuses
- View customers' information and customers' calls
- Navigate between ADMS and Scada displays
- Perform SCADA operations such as remote control and tagging
- Perform device operations and temporary network modifications (jumpers, cuts, grounds)
- Perform network tracing and analytical studies
- Manage incidents
- Issue switching orders and safety documents, and perform switching and restoring steps
- View and dispatch crews
- Perform incident history correction

In addition to the operation-oriented displays, ADMS also provides tools to create read-only dashboard displays and reports for management.

2.1.6. Scada Integration

A key part of the ADMS concept is real-time control and monitoring (Scada) integration. The integration allows the Distribution Network Object Model (DNOM) to be automatically updated with real-time data (status and measurements) from the field, and allows the distribution operator to directly control network devices from the ADMS user interface. The integration also includes using the common PERMISSION ALARM, TAGGING, and SAFETY components. (More discussion about Scada integration is included in section Scada Integration.)

2.1.7. Historical Data

The requirements of the ADMS historical data archiving function are driven by reporting requirements. The primary data source applications are:

- DMS
- OMS
-

Switching Order

- Safety Document
- Tagging
- Scada

Historical data archiving requires recording the business activities conducted by these applications.

The primary data objects that need to be stored in the archive are:

- Distribution Power Flow (DPF) solution summaries
- Fault Location (FL) results
- Fault Isolation and Service Restoration (FISR) plans
- Load and Volt/VAr Management (LVM) plans
- Automatic Feeder Reconfiguration (AFR) plans
- Station and feeder exceptions
- Network model file information
- Incidents
- Switching orders
- Daily logs
- Safety documents
- Tags
- SCADA measurements

All of the data objects are from ADMS applications, with the exception of safety documents, tags, and SCADA measurements, which are from Habitat–based applications. Historian is responsible for capturing the tags and SCADA measurements, while ADMS Archive stores the ADMS application data and safety documents. Tags stored in the Historian database schema are visible to the ADMS Archive database schema through a database link.

2.1.8. Simulation and Training Tools

ADMS includes a collection of simulation engines and training modules. Together with the other software components such as the real-time network management system, it forms an integrated Distribution Operator Training Simulator (DOTS). With DOTS, dispatchers can be trained in both routine and emergency operations that accurately represent the behavior and response of the real system.

The simulation engine includes a power flow solution and event scripts. The power flow engine is the same software module as in the real-time server. The power flow solution is based on the current DOTS topology. It can configure to add noise on top of the calculated values before the solution is fed to SCADA. This gives the simulation the ability to feed power flow–related data points

to the simulated SCADA and support training and testing of the network analysis and optimization functions.

The simulation engine provides a full OMS simulation. This functionality includes:

- Trouble order message simulation based on location of faults placed by event scripts
- Fault location
- Switching operations for restoration
- AMI interface simulation

Note that replicas of external systems such as Interactive Voice Response System (IVR)/ Voice Response Unit (VRU), Customer Information System (CIS), etc., are not present in the training simulator. However, interfaces to these external systems are available, so that interaction of these systems with ADMS can be represented in the simulation environment.

The event scripter provides the means of creating and playing scripts for the Distribution Operator Training Simulator. The scripts are a list of network events, each with a time tag giving the number of seconds after the start of the script at which the event is to be executed.

Script events can consist of:

- Faults on lines
- Customer calls
- Modified OMS parameters
- Switch changes

Fault scripts can also be automatically generated to simulate a defined scenario. The scenario is built up by defining the probabilities of different types of events causing faults on lines, transformers, and customers. The stations to be included and a time scale for the scenario are selected. The system then calculates a list of faults. The fault list can be saved as a script or merged into an existing script. The scenarios can also be saved separately so that they can be used with different scripts. The overall execution timing of a script can be compressed or expanded as required. The execution time for each event is rescaled proportionately. Scripts can be saved and recalled, or shared with other users via the common library.

2.1.9. Study Mode

Study mode allows the operators to perform detailed "what if" analysis of proposed network configurations and switching procedures.

Study mode uses the same static models as real-time mode, but with a separate instance of the dynamics. This removes the requirement to separately set up and maintain the study mode environment.

Changes made to the study mode network model are applied only to the user’s workstation and are not visible to others. Both real-time and study modes can be run concurrently on the same

workstation, allowing an operator to perform a study immediately prior to executing the operation in the real-time environment.

The study mode environment can be initialized from the nominal model, from the real-time mode, or from a savecase. The study case setup tool allows the user to quickly create the study environment with the required stations.

2.1.10. External Interfaces

ADMS needs to integrate with numerous external systems in order to accomplish the objective of providing users with a single consistent operating environment to manage the distribution network efficiently. A summary of the required external interfaces is provided in section Integration with External Services.

The peer-to-peer interfaces have been implemented using **.**NET-based technology. They are using XML-based messages for the communication, which is an industry standard. They perform XML serialization and de-serialization using **.**NET-provided classes.

The REST based web service has been implemented to retrieve the OMS and DMS data from the system.

2.2. Application Architecture

The design of ADMS is based on client-server architecture. The client provides the user interface, while the server performs computing-intensive tasks and interfaces with external services.

The following sections first use the main network server application to illustrate the architectural pattern of the server application, and then explain the major usage of the other server applications and user interface components.

2.2.1. DMS and OMS Servers

The following ADMS network server applications provide distribution and outage management functions.

- **DMS Main Server (NetSrv):** NetSrv is the core component of the ADMS solution. It provides core system functionality and maintains interactions among ADMS system components. It is also the master of the in-memory DNOM database.
- **OMS Server (NetServer):** Provides functionality for the Outage Management System. It is the master of the in-memory OMS database.
- **DNAF Server (DnafSrv):** Provides functionality for the Distribution Network Analysis Functions (DNAF) and is the master of the in-memory DNAF results.
- **DNA Server (DnaSrv):** If enabled, the Distribution Network Applications (DNA) server runs DNAF solutions for the DNAF server. The DNA server has a smaller performance footprint: it

does not load a complete system model and dynamics.

- **Simulator Server (SimSrv and NetServerSIM):** Provides functionality for the Distribution Operator Training Simulator environment.
- **Read-Only DMS Server (RoSrv):** Provides functionality for the DMS Read-Only mode.
- **Read-Only DMS and OMS Server (NetServerRO):** Provides functionality for the DMS and OMS Read-Only mode.
- **Customer-Only Mode Server (NetServerCM):** Provides functionality for the Customer-Only mode, which operates with the Advanced Metering Infrastructure (AMI) system.
- **DMS Server in Follower Mode:** A DMS server can run in follower mode to the enabled DMS server. In Independent Follower mode, follower servers receive changes from the enabled server and independently implement DMS commands from the follower server clients. In Dependent Follower mode, follower servers receive changes from the enabled server and send requests to the enabled server to implement DMS commands from the follower server clients. An OMS server can receive permissions from a DMS server in follower mode.
- **OMS Server in Follower Mode:** An OMS server can run in follower mode to the enabled OMS server. In Independent Follower mode, follower servers receive changes from the enabled server and independently implement OMS commands from the follower server clients. In Dependent Follower mode, follower servers receive changes from the enabled server and send requests to the enabled server to implement OMS commands from the follower server clients.

> **Note**
>
> Multiple DMS and OMS follower servers can run simultaneously. Follower servers do not support redundancy.

The servers are implemented as multi-threaded applications. Message objects are passed between modules by the application's main thread. The main thread is responsible for delivering intra-module messages. The architectural pattern also applies to the thread design inside some modules. Each module has a single "dispatch thread", which schedules worker threads and exchanges intra-module messages, as well as sending and receiving messages from the main thread. An administrator can configure the maximum number of worker threads based on the size of the server machine.

The following figure illustatres the flexibility of the modular applications. A client can connect to the server that is responsible for getting the type of data the client needs. This architecture allows DMS and OMS features to be deployed and patched independently.

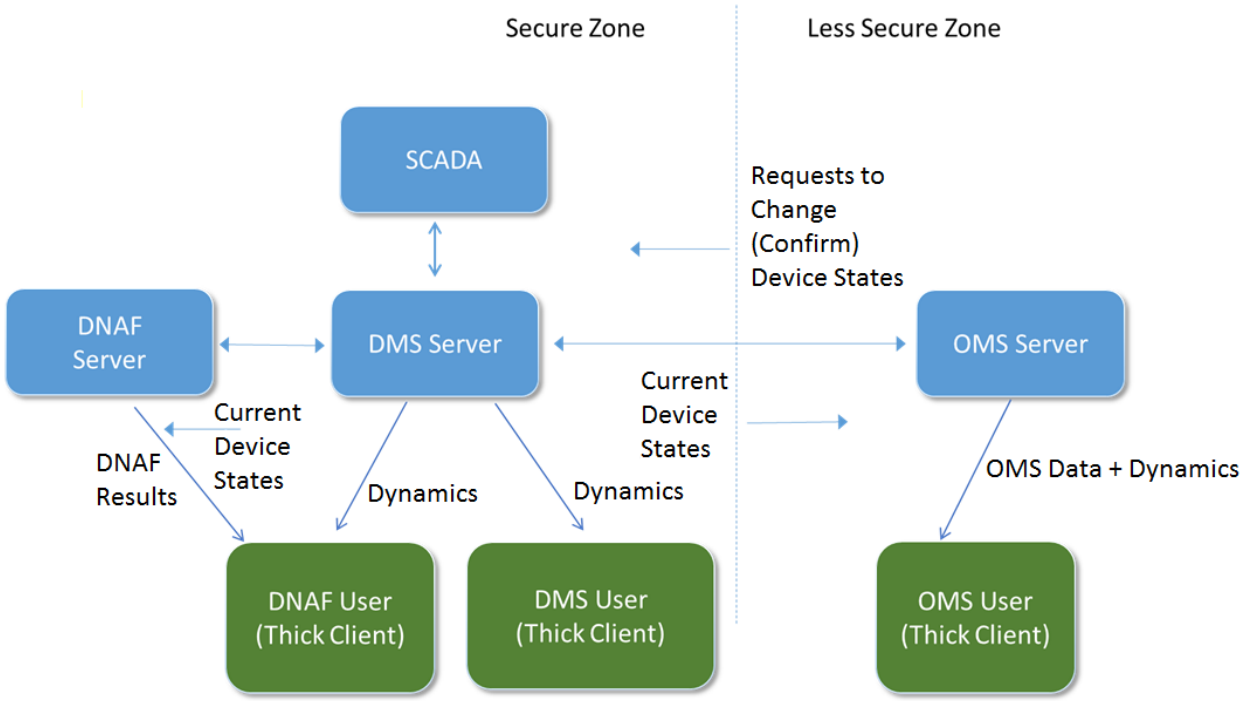


Figure 2. Modularized Applications

The following figure illustrates the server messages concept.

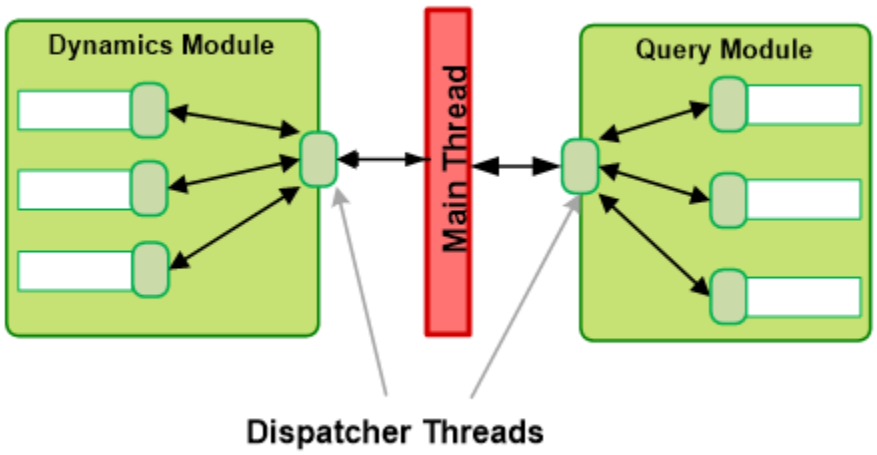


Figure 3. Server Messages Concept

The server applications are WinForm applications. NetSrv wraps the DMS modules (C++ unmanaged code). NetServer references the OMS server components and OMS utilities (both are C# managed code) as its main functional modules. The OMS server components define the OMS engine, OMS loader, and OMS query modules, while OMS utilities define miscellaneous utilities.

The following figure illustrates the software modules and the threads model for a NetSrv and NetServer pair.

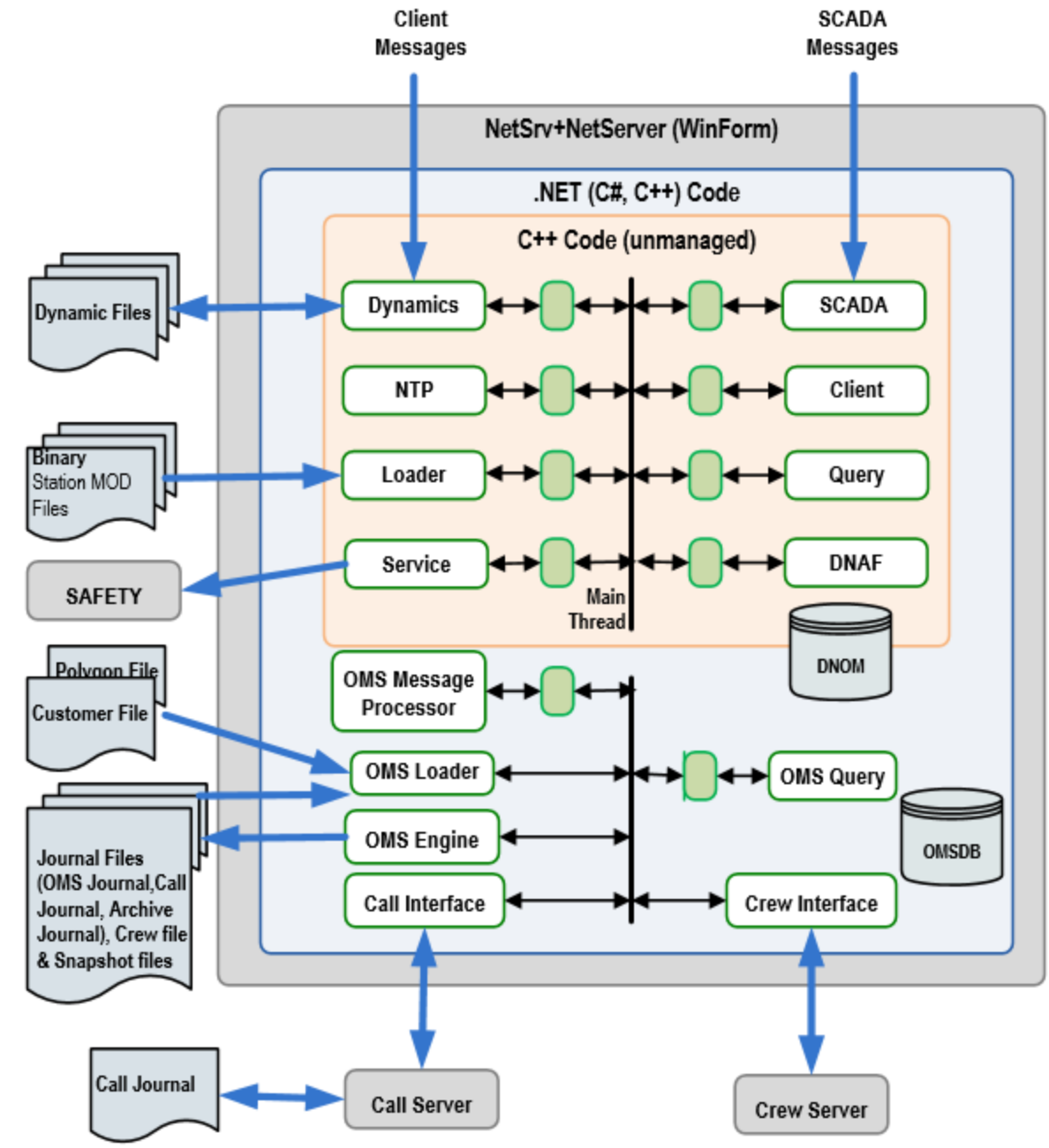


Figure 4. Software Modules and Threads Model of NetSrv and NetServer

> **Note**
>
> The Simulator server does not instantiate the OMS engine module or the OMS query module. Instead, it instantiates the OMS loader module and the OMS SimEngine module (not shown in the diagram).

2.2.1.1. Loader Module

The loader module monitors changes to station files and DNAF parameter files placed on the server. The location for these files is set in the ADMS NetSrv application's configuration file.

When updating a station model:

1. The existing station file is replaced with a new file.
2. The loader thread polls the directory and identifies the new file.
3. The loader deserializes the binary station data from the file into memory.
4. The loader unloads the station whose model file is removed.

The loader module is not responsible for reading or applying dynamic data from the data persistent files.

2.2.1.2. Dynamics Module

The dynamics module handles dynamic or topology changes from the nominal position submitted by ADMS clients or by Scada.

A dedicated thread maintains the TCP/IP connection with the clients, sending and receiving the ADMS binary messages.

There is a worker thread pool that is used for dynamic processing.

The dynamics module also maintains a dedicated thread that manages a connection with PERMIT in EMS. The permission structure from PERMIT is loaded and stored on the server. This structure defines permissions against a Global Unique Identifier (GUID) that is submitted by the ADMS client with each request. Communication uses ADMS's proprietary protocol.

The dynamics module is responsible for journaling the dynamics, and for performing the midnight compression and periodic backup of the dynamics files. When the server starts up, the dynamics module reads and applies dynamics from the saved files.

2.2.1.3. SCADA Module

The SCADA module maintains a TCP/IP connection with the remote SCADA; it uses GE's proprietary ISD protocol for communication. The SCADA module processes the SCADA messages and creates an ADMS message to notify the dynamics module. The dynamics module applies the change to the dynamic data.

The SCADA module also has the ability to communicate with Control for SCADA data using the ISD protocol. It uses the same internal ADMS message, so the rest of ADMS is insulated from the SCADA communication protocol.

> **Note**

When the SCADA module and Control are integrated, the Control does not have knowledge of tags on DMS objects, and it does not support permissions.

2.2.1.4. Network Topology Processor Module

The Network Topology Processor (NTP) module is notified of changes to the topology of the network, and it makes a network selection based on the changes. This includes the station that contains the changes and all of its connected stations. To avoid conflicts, each thread works on one station at a time (there may be multiple threads processing multiple stations).

The NTP module implements its own scheduling. If it is receiving a lot of changes, it can schedule the processing to run once following the receipt of the entire group of changes.

The processor determines the energization/de-energization of the network, including identifying any loop states (where a section is receiving power from more than one source) within the network. The results of the processing are written to the DMS server's DNOM. These results are not pushed to the clients; they execute their own topology processing on the local model.

Note

This implies that a change in the NTP module may potentially affect both servers and clients. Patch releases should have release notes clearly identify the potential inconsistency, so that customers can incorporate the necessary steps in their change management process to coordinate the patch roll-out.

This client processor is built on the same code that the server uses; however, the processing is simpler. Topology processing occurs quite often, so executing it on the client greatly reduces the amount of information that must be sent from the server and makes the system more efficient.

Results of this processing are used by the DNAFs to perform network analysis.

2.2.1.5. Query Module

The query module receives requests for network analysis information from the ADMS clients. These requests access data within the DNOM created by a particular analysis function. The client's request is received by the dynamics module, which recognizes it as a query, so it passes the message through to the query module.

Queries are submitted using a simplified SQL-like syntax. Every class within the DNOM has a serializable name, and each attribute has a name. Every object has an associated ID and name. These are used within the SQL syntax to identify the data to retrieve. Dynamic data also has record and field names, so these can also be included in a query.

For example, the following statement returns the stated data from all switches with station DOUGLAS: .Sample Silence

```
SELECT ID, Name, State, NormalState
```

```
FROM rt.sub(DOUGLAS).swt
```

Results from the query can be provided in XML, or in tab-separated or comma-separated formats.

This module also provides the ability to write helper functions, which can be included in queries. These helper functions are created within code; therefore, they can provide a more sophisticated query. The query engine holds a pointer to this function, allowing it to execute the function on request.

2.2.1.6. Client Module

The client module provides communications among the Habitat configuration manager (CFGMAN), the primary (active) server, and the standby server (or servers). The standby server(s) appears as a client to the active server; therefore, it receives the updated station files and dynamic data as any normal client does.

2.2.1.7. Distribution Network Analysis Functions Module

The Distribution Network Analysis Functions (DNAF) module manages the execution of the various analysis functions written for distribution management systems. These analysis functions execute against the current state of the network, providing information to the operator as required. Different events can trigger an analysis function, including a user request, a network change, or a timed schedule.

These analysis functions are built to work with a three-phase unbalanced network; this requires more effort than a balanced network. However, these analysis functions work with either.

The Distribution Power Flow (DPF) provides the base analysis for the network. It uses information provided by the integrated SCADA and the distribution network model to determine power details across the different elements within the network. This information is used by the other analysis functions during execution.

The DNAF module runs with the DNAF server application. It is possible to turn functions on and off in the DNAF server configuration file or the client DNAF System Configuration display.

2.2.1.8. Service Module

The service module exposes DMS services to external systems through a set of service-based API functions. The portable programmer's client side framework is provided to utilize DMS services. For .NET developers, a .NET version of API functions is provided.

Currently, the following interfaces are supported:

- IQuery
- ISafety

- IDnaf

The client's application can execute DMS service functions as if they were the client's local functions. The query interface exposes all queries from the query module.

Subscription to the following events is provided:

- Topology change
- Station model file load/unload
- Plan manager step execution events
- Communication change events

The client's application can subscribe to various DMS service events to be notified about DMS server changes.

2.2.1.9. OMS Message Processor Module

The OMS message processor module is part of the managed .NET C code. As shown in the "Software Modules and Threads Model of NetSrv and NetServer" figure, the NetSrv and NetServer applications integrate modules developed with C unmanaged code and modules developed with C# managed code. Since the dynamics module (unmanaged code) exchanges messages with the ADMS clients, for OMS-related messages, it needs to pass to the OMS modules (managed code). It needs transformation to handle exchanges of intra-module messages among these managed and unmanaged code modules. The transformation is a light-weight operation. Since the modules reside in the same process, a simplified view of the transformation is like a memory copy of the objects from one form to another form. The OMS message processor module is responsible for the transformation.

2.2.1.10. OMS Loader Module

The OMS loader module monitors and loads data files related to the OMS functions. The static files of interest are the compiled customer and crew files. Upon server application startups or model updates, the OMS loader module is also responsible for reading and applying the dynamics from the snapshot and journal files to restore the application to the current system state. The location of these files is set in the server application's configuration file.

2.2.1.11. OMS Query Module

The OMS query module receives requests for OMS information from the ADMS clients. These requests access data within the in-memory OMSDB or information stored in files. The client's request is received by the dynamics module, which recognizes it as an OMS query, so it passes the message through to the OMS message processor module, which performs the transformation and then passes to the OMS query module. The OMS query module resolves the query based on .NET reflection, and any public field and property of a class becomes available for querying as soon as it

gets added to the system. The query is supported using SQL-like syntax similar to the query module. For example, the following statement returns the incidents based on the read assignment of Area of Responsibilities (AOR) that the operator has:

SELECT
INCIDENTTYPE,GEONAME,X,Y,OPENDEVICESTATION,PROTECTIVEDEVICENAME,OPENDEVICEID,STARTTIME,ETR

FROM INCIDENT

WHERE AOR READASSIGNED %WEBFGGUID

2.2.1.12. OMS Engine Module

The OMS engine module is the core analytical module of the OMS. It performs the business logic of incident management, outage extent analysis, outage life-cycle management, calculation of outage indices, and crew monitoring and assignment. This module is also responsible for writing the incident journal files and call journal files on the Primary (enabled) ADMS Server. It also creates and maintains snapshot files and archive journal files.

2.2.1.13. Call Interface Module

The call interface module manages communication between the NetServer application and the call server application. It maintains the TCP/IP connection with the call server application, sending and receiving call messages. The call messages are then queued or relayed to the OMS engine module.

2.2.1.14. Crew Interface Module

The crew interface module manages communication between the NetServer application and the crew server application. It maintains the TCP/IP connection with the crew server application, sending and receiving crew messages. The crew messages are then queued or relayed to the OMS engine module.

2.2.1.15. OMS Customer Engine Module

The OMS customer engine module is the core module for the DMS running in customer-only mode. It enables certain functions of the OMS.

2.2.1.16. Plugin Module

The plugin module provides a mechanism for quickly extending the functionality of the main server by loading pre-registered dynamically linked application libraries (DLLs). These libraries can contain additional functionality specific to a customer or entirely new applications that can be optionally

used. By using separate libraries, updates to the system functionality can be made without having to update the main server. Functionality can be adjusted for specific clients by using different libraries or library combinations.

The plugin module is event driven. When system changes occur, such as NTP processing completion, DNAF processing completion, and configuration file changes, the main server notifies the plugin module. The plugin module forwards these notifications to all registered application libraries so that they may do specific processing.

The application libraries have access to the DNOM object model and can use any data that the main server applies to perform its calculations.

The plugin module includes worker threads, configuration parameters, and triggering messages.

2.2.1.17. Plan Manager Module

The plugin module provides a mechanism for managing reconfiguration and optimization plans generated by real-time advanced DNAF applications (AFR, FISR, and LVM). The module supports the Advanced Application Plan Status System View display. The functionality of the main module allows for:

- Implementing, removing, cancelling, and failing plans.
- Implementing plans automatically or step-by-step.
- Checking SCADA status measurements after a step implementation.

2.2.1.18. Modules in Different System Configurations

This section illustrates enabled modules in different system components.

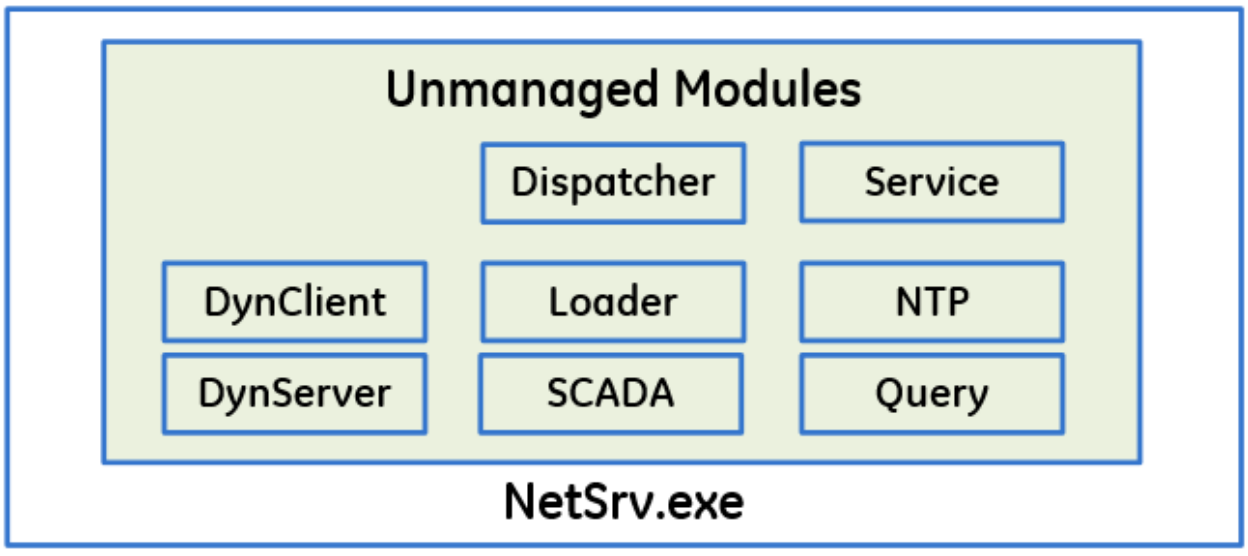


Figure 5. DMS Component

The previous figure includes the following modules:

- **Dispatcher:** Forwards all messages among modules.
- **Service:** Exposes DMS services to external systems through a set of service-based API functions. For NetSrv.exe, this module is used by the Tagging and Safety Documents applications to retrieve data from ADMS. For DnafSrv.exe, this module is used by the DNOM2HDB application to retrieve DNAF data from ADMS.
- **DynClient:** Creates fast dynamics buffers. Manages connections with other servers.
- **DynServer:** Interacts with clients through TCP/IP. Writes dynamics files.
- **SCADA:** Interacts with ProxySrv/Scada.
- **Loader:** Loads station models and publishes the "station map" to other modules through the dispatcher.
- **NTP:** Performs topology processing. Adds and removes temporary modifications (cuts, jumpers, faults, and grounds).
- **Query:** Resolves queries for tabulars, pop-ups, and search operations.

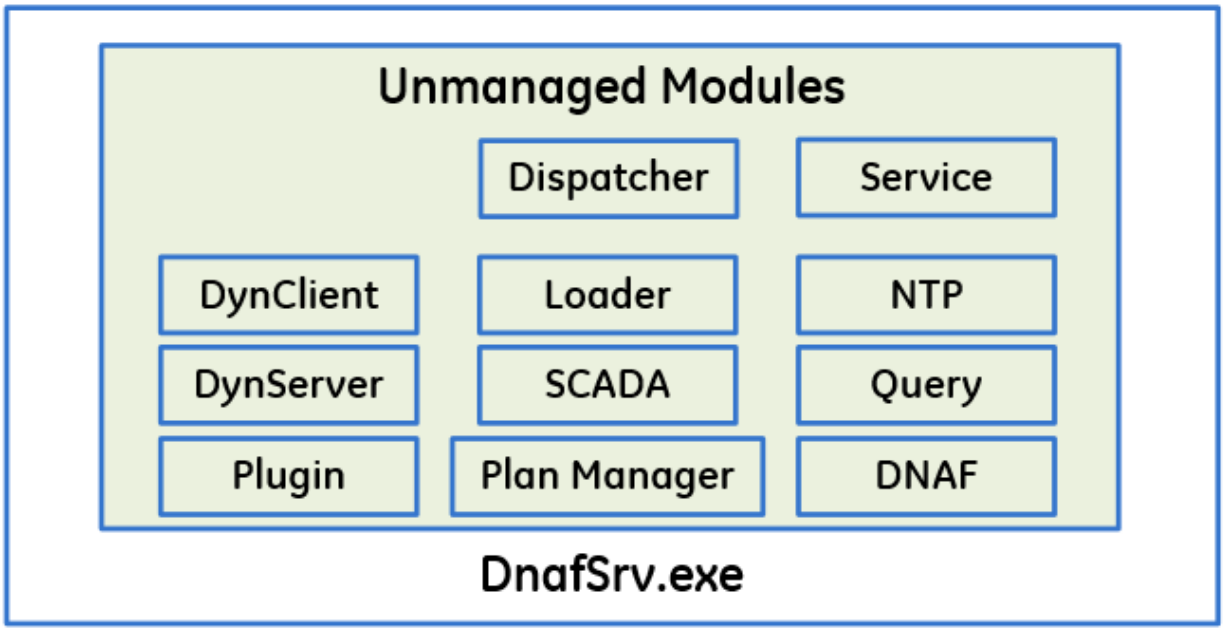


Figure 6. DNAF Component

The previous figure includes the following modules:

- **DNAF:** Reads .lst files and applies them to DNOM. Runs DNAF applications.
- **Plan Manager:** Manages DNAF advanced application plans.
- **Plugin:** Manages plugin applications.

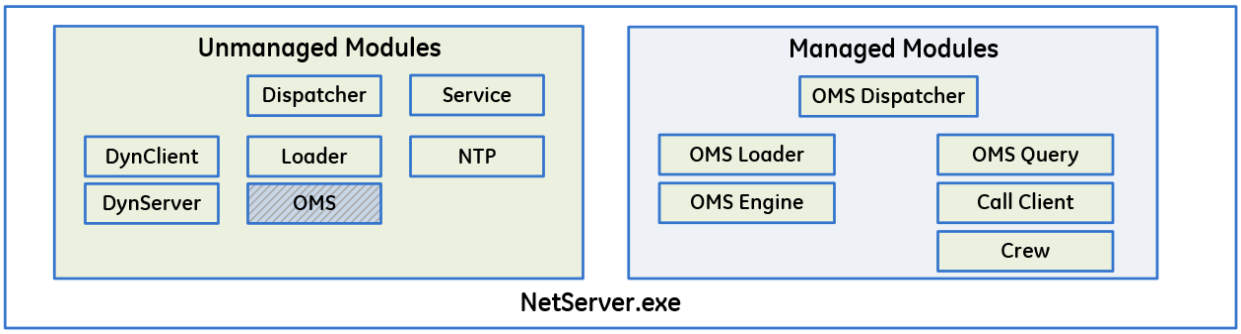


Figure 7. OMS Component

The previous figure includes the following modules:

- **OMS Module:** Holds a station map pointer to DNOM. Forwards messages among Dispatcher and OMS Dispatcher. Uses mixed managed/unmanaged C++.
- **OMS Dispatcher (C#):** Dispatches messages among managed modules and the OMS module.
- **OMS Loader (C#):** Creates the "OMS DB" and passes it to OMS Engine on startup and when a new customer file is loaded. Creates OMS Snapshots.
- **OMS Engine (C#):** Maps DNOM to the OMS DB. Writes journals (Call, Incidents, Crew, and Archive). Handles calls, AMI messages, NTP results, and power measurements.
- **OMS Query (C#):** Responds to OMS queries from the clients.
- **Call Client:** Interacts with Call Server.
- **Crew (C#)**: Interacts with Crew Server (mobile interface).

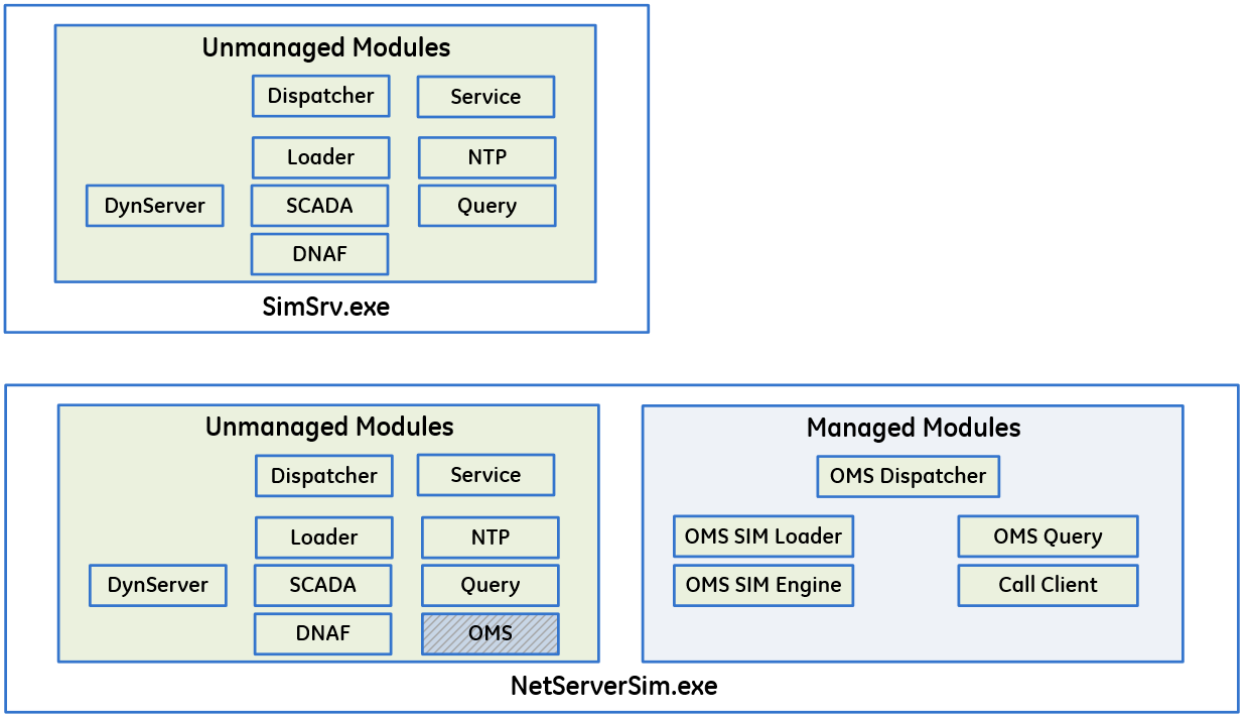


Figure 8. Simulator Components

The previous figure includes the following modules:

- **OMS Sim Loader (C#):** Creates the "OMS DB" and passes is to OMS Engine on startup and when a new customer file is loaded. Does not create snapshots.
- **OMS SIM Engine (C#):** Maps DNOM to the OMS DB. Does not create journals. Processes NTP results. Generates calls and AMI messages.

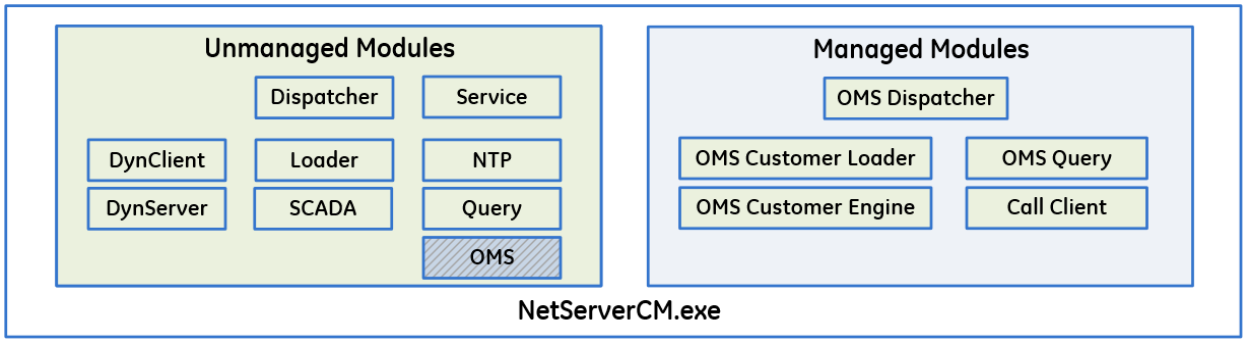


Figure 9. Customer-Only Mode Component

The previous figure includes the following modules:

- **OMS Customer Loader (C#):** Creates the "OMS DB" and passes is to OMS Engine on startup and when a new customer file is loaded. Does not create snapshots.
- **OMS Customer Engine (C# and managed C++):** Maps DNOM to the OMS DB. Writes Call journal only. Processes calls, AMI messages, and power measurements.

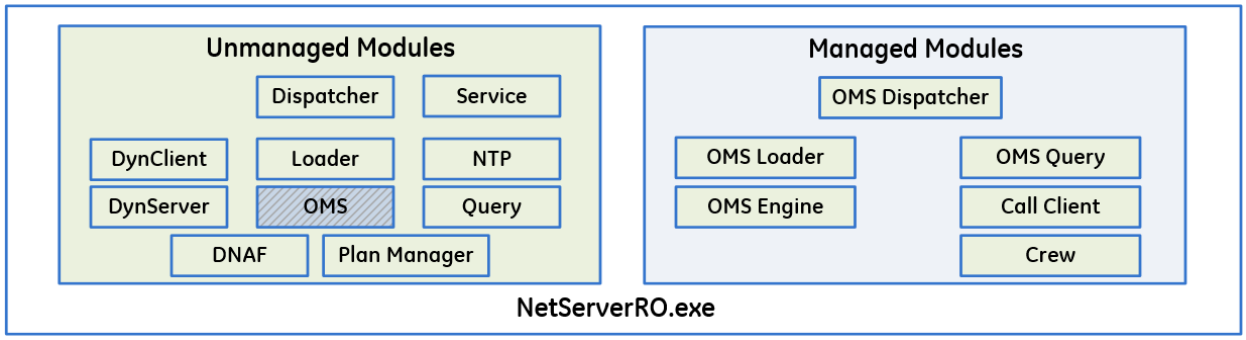


Figure 10. OMS Read-Only Mode Component

The previous figure includes the following modules:

- **Plan Manager:** This module is used on a read-only server to show advanced application plan data on tabular displays.

DMS read-only mode components are similar to OMS read-only mode components except for the OMS modules.

2.2.2. Dual Production DMS/DNAF

This is a product concept of Leader-Follower, which enables the capability of having more than one ADMS system to stay enabled concurrently. There are two types of Follower Modes: Independent and Dependent. The main difference between these two modes is:

- Dependent Follower mode DMS server forwards all the manual actions to Leader and synchronizes dynamics with the Leader DMS server while serves its own clients as usual.
- Independent Follower DMS server does not forward any manual actions directly to the Leader DMS server. It doesn't synchronize dynamics with the Leader and only accepts almost all the dynamics from the Leader.

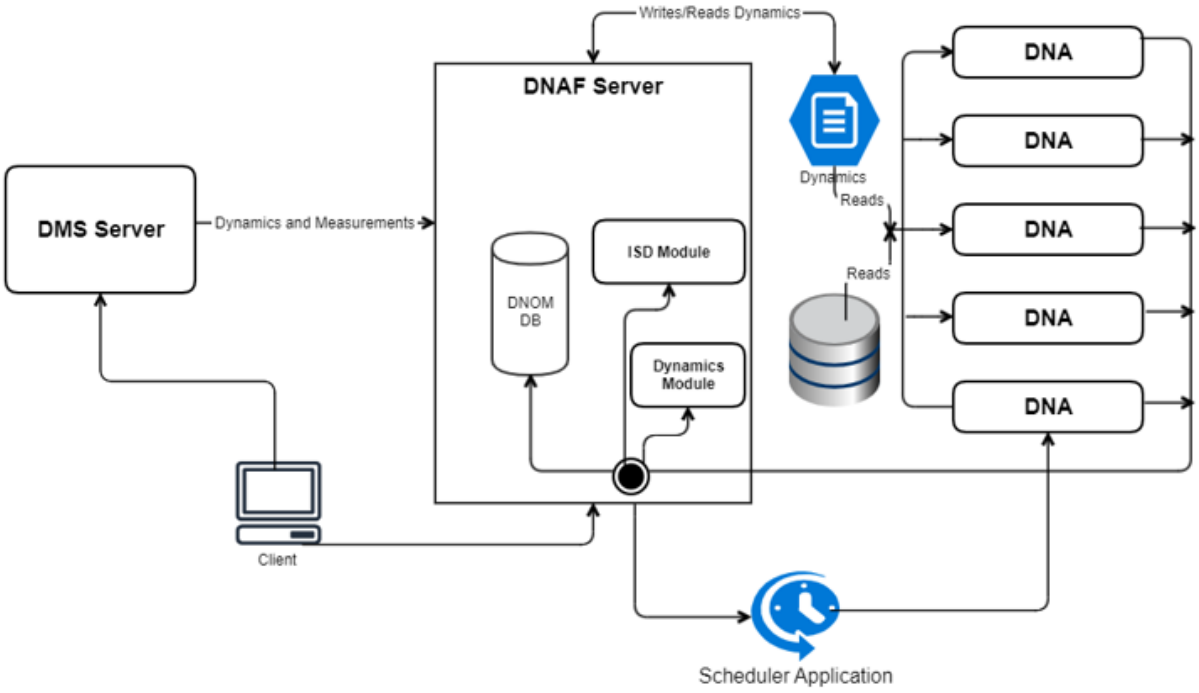
  The Independent DMS Follower can be leveraged by the utilities to try newer features in the Follower sub-system without making any changes to the production system software.

  It is required for the Follower servers and clients have a version that is at least the same as the Leader Server's version.

For detailed setup information, refer to the Dual Production configuration details in the *Distribution Software Intallation and Maintenance Guide*.

2.2.3. DNAF Decoupling

To improve DNAF scalability, DNAF modularization is being incorporated into the ADMS. The ultimate outcome is to transform the DNAF to serverless "Function as Service", providing horizontal scalability. This modularization will be implemented over multiple ADMS releases. The high-level architecture diagram for initial phase 1 is below.



The continuously running scheduler application helps to coordinate the DNAF Server and a list of Distribution Network Analysis (DNA) Servers. The basic role of the scheduler application is to receive requests from the DNAF Server and schedule the DNA Servers to carry out the requests. The main functionalities provided in phase 1 are as follows:

- The scheduler application has the intelligence to creates and monitor a number of DNA Server instances based upon configuration. For details, refer to the Distribution Software Intallation and Maintenance Guide.
- The application accepts tasks (requests) from the DNAF Server and sends each task to an idle DNA instance to carry out and monitors each DNA Server's status.
- The scheduler application is a container deployable application written in .Net C#.
- DNA is designed to solve DPF, Fault Location, and FISR in phase 1 release (other applications will be done later releases).
- The DNAF Server is configurable to solve the requests in its worker threads or send them to the scheduler application.
- The request and input data sent to the scheduler application (including dynamics, SCADA measurements, and DER measurements) are stored in Protobuffer messages.
- The scheduler application will manage the requests in a message queue and distribute the requests to a list of DNA Servers.
- Any communication failures between servers (DNAF and scheduler, and scheduler and DNA) are

handled with alarms and logs.

- The solution results in each DNA Server will be passed back to the DNAF Server directly.

2.2.4. Read-Only Server for OMS Data (Net Data Server)

Net Data Server supports near real-time history of OMS data. It provides the same OMS queries as NetServer, except that it includes current incidents as well as incidents that have been completed in the recent past (2–4 weeks). The intent is to provide read-only access to current OMS data and recent history for users inside and outside of the control room environment. Therefore, it supports replicating necessary data to other Net Data Server(s) in different firewall zones. It provides data to support light-weight client tools (such as dashboard software or reporting packages) that can run in a Web browser.

Data are provided via Web services. The Web services are implemented using Microsoft Windows Communication Foundation (WCF). The framework takes care of the communication protocol and formatting data objects. It uses Open Data protocol (OData) to expose services, which is based on Representational State Transfer (REST) architecture.

The figure below illustrates the data flow of a typical configuration servicing read-only clients. In the diagram, the replicated Net Data Server is installed in a different hardware than the Web server. However, the design does not preclude installing both on the same hardware.

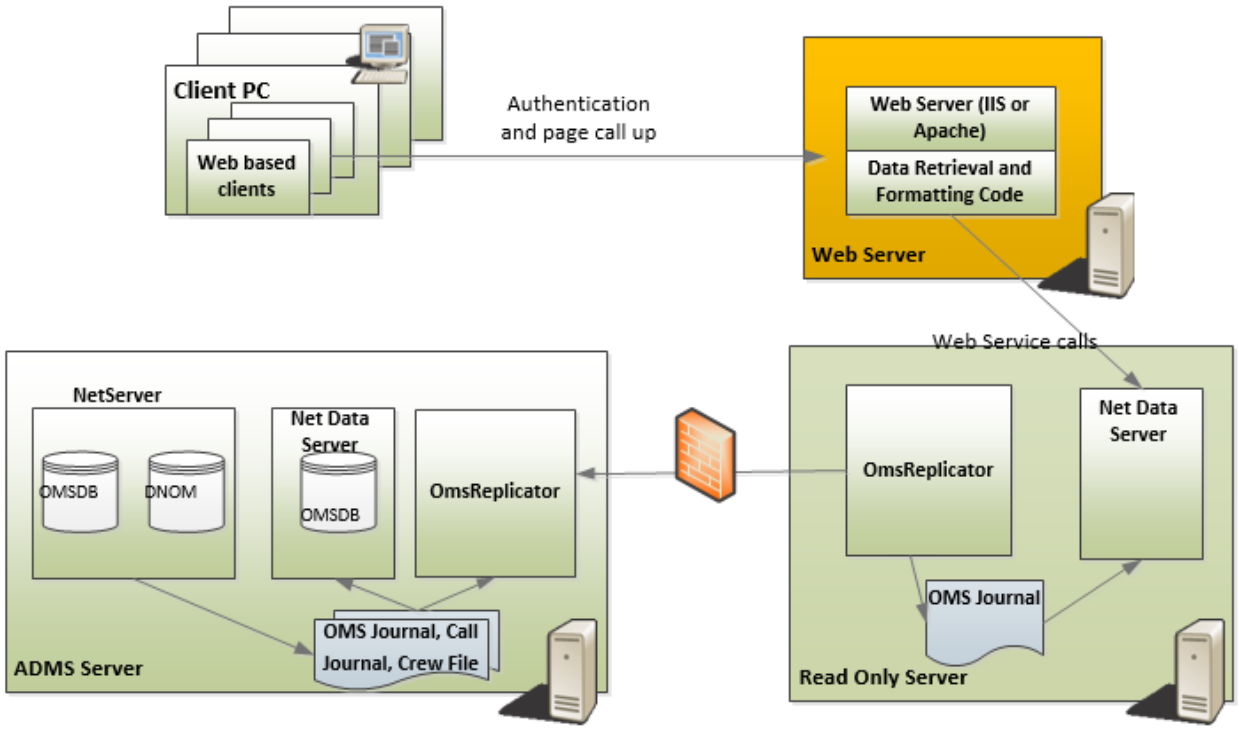


Figure 11. Servicing Read-Only Clients Using Self Hosted Service

Net Data Server also provides recently cleared incidents data to the recent history correction application (not shown in the diagram).

2.2.5. DMS Interface Server

The DMS interface server provides a standard web server interface to allow third-party applications to communicate with the DMS server (NetSrv). The DMS interface server acts as a client to the DMS Server, using the same communication protocol that the Browser client uses to talk to the DMS server. This enables access to the same set of DMS data as the client.

The DMS interface server can be managed by CFGMAN and PROCMAN to provide redundancy and high availability. The detailed configuration depends on the deployment needs.

2.2.6. DMS API Server

The DMS API server can receive inputs from external systems and forward them to the DMS server. It offers the flexibility to interact with the ADMS servers.

2.2.7. OMS Interface Server and OMS Adapter

The OMS Interface server provides a standard web server interface to allow third-party applications to communicate with the OMS server (NetServer).

The OMS Adapter provides a web services portal for messages among the OMS Interface server and external applications, including the Outage Assist solution (also known as the OMS Thin client). The OMS Adapter supports the Customer Experience, Automated Notification Paging Services, and View Outage Portal web services.

The OMS Interface server and OMS Adapter do not have a CFGRPROXY interface. They can be managed by CFGMAN and PROCMAN to provide redundancy and high availability. The detailed configuration depends on the deployment needs. For more information, refer to the *Habitat Configuration Manager User Guide* and the *PROCMAN User Guide*.

2.2.8. Switch Order Server

The switch order server application supports real-time switching operations. It serves as a document server for switching orders as well as daily logs (unplanned switching operations during an operator's shift). The switch orders and templates are stored as XML files. The switch order server application supports user actions for creating, editing, copying, and saving switch orders. The server also supports printing and exporting to Microsoft Excel or PDF formats.

When the user executes a switching step, the NetSrv application changes the state of the device, updates the network topology, and journals the actions. The switch order server application updates the switch order document.

When a switching step is completed, it is archived by the Archive application. The archive database provides general query capability for the archived switch orders.

The switch order server also provides general query capability for planned switch orders.

2.2.9. Call Server

The call server application implements the interface to the Customer Information System (CIS), the Interactive Voice Response (IVR) system, and the Advanced Metering Infrastructure (AMI) system, through an Enterprise Service Bus (ESB) infrastructure, which is a custom integration component between the call server and an external system. The call server application relays calls from a CIS, IVR, or AMI (a "call server") or from another call server (a "relay server") to another call consumer (call server, relay server, or NetServer). It also maintains a call journal file for speeding up the synchronization when the server starts up or recovers from temporary network disruption. (More discussion about interfacing with external services can be found in section Integration with External Services.)

2.2.10. Crew Server

The crew server application implements the interface to the crew management system through an Enterprise Service Bus (ESB) infrastructure. The crew server application relays the crew assignment messages from a workforce management system to NetServer. There is no user interface client connected to the crew server. The query capability for the crew data is provided by NetServer. (More discussion about interfacing with external services can be found in section Integration with External Services.)

2.2.11. Net Data Replicator

Net Data Replicator replicates OMS and Switch Order data between servers in a high-availability configuration. In such a configuration, one or more standby servers run in parallel to an enabled server, but do not actively process transactions. After a failover, client applications automatically connect to the newly enabled servers, which are aware of all transactions that were completed (saved) before the failover.

Net Data Replicator replicates the following files:

- Incident journal
- Crew journal
- Archive journal
- Call journal
- Call Server journal
- Relay Server journal
- Daily snapshots
- Switch Order files

Net Data Replicator can run in DMSONLY and READONLY modes. In DMSONLY mode, it copies only the switch order files. In READONLY mode, it connects to the enabled machine and copies the data to the read-only server. The application also interfaces with CFGMAN, which allows it to behave differently in case role assignments change. The interface to CFGMAN is not enabled in READONLY mode.

Net Data Replicator ensures that OMS data created or updated on the enabled server is replicated to the standby server(s), so that the standby server(s) can quickly assume the enabled role without loss of data in the event of a failover.

2.2.12. User Interface Component (NetDisplay)

NetDisplay is the main client component for ADMS. It runs as a plug-in within Browser. It can instantiate a single NetConfig object and multiple NetDisplay objects (DMSDisplay, OMSDisplay, SwitchOrder). The collections of controls include the geographic view, tabular displays, the area world view, pop-up dialog boxes, and navigation within Browser.

2.2.13. Simulation Tools

ADMS includes a few simulation utility applications that are only used in the simulation (DOTS) environment:

- Crew Simulator (RoboCrew): This application simulates the behavior of a field crew. It works together with crew server and operator simulator for automatic changing of crew status, crew position, and fault removal.
- Operator Simulator (RoboDispatch): This application simulates the behavior of an operator, automatically assigns crew to the incident, keeps track of crew work, and clears incidents.
- Event Scripter: Used for the creation and running of special scripts that simulate customer trouble calls, AMI outages, unassociated calls, and device faults.

2.3. Application Programming Interfaces

ADMS provides a number of interfaces using web service (REST) and TCP-based formats to enable integration with third-party products and enterprise applications, and support read-only access to ADMS data.

- Outage Management Read-Only Interface
- Outage Management External Interfaces
- Outage Management Interface
- Distribution Management Interface

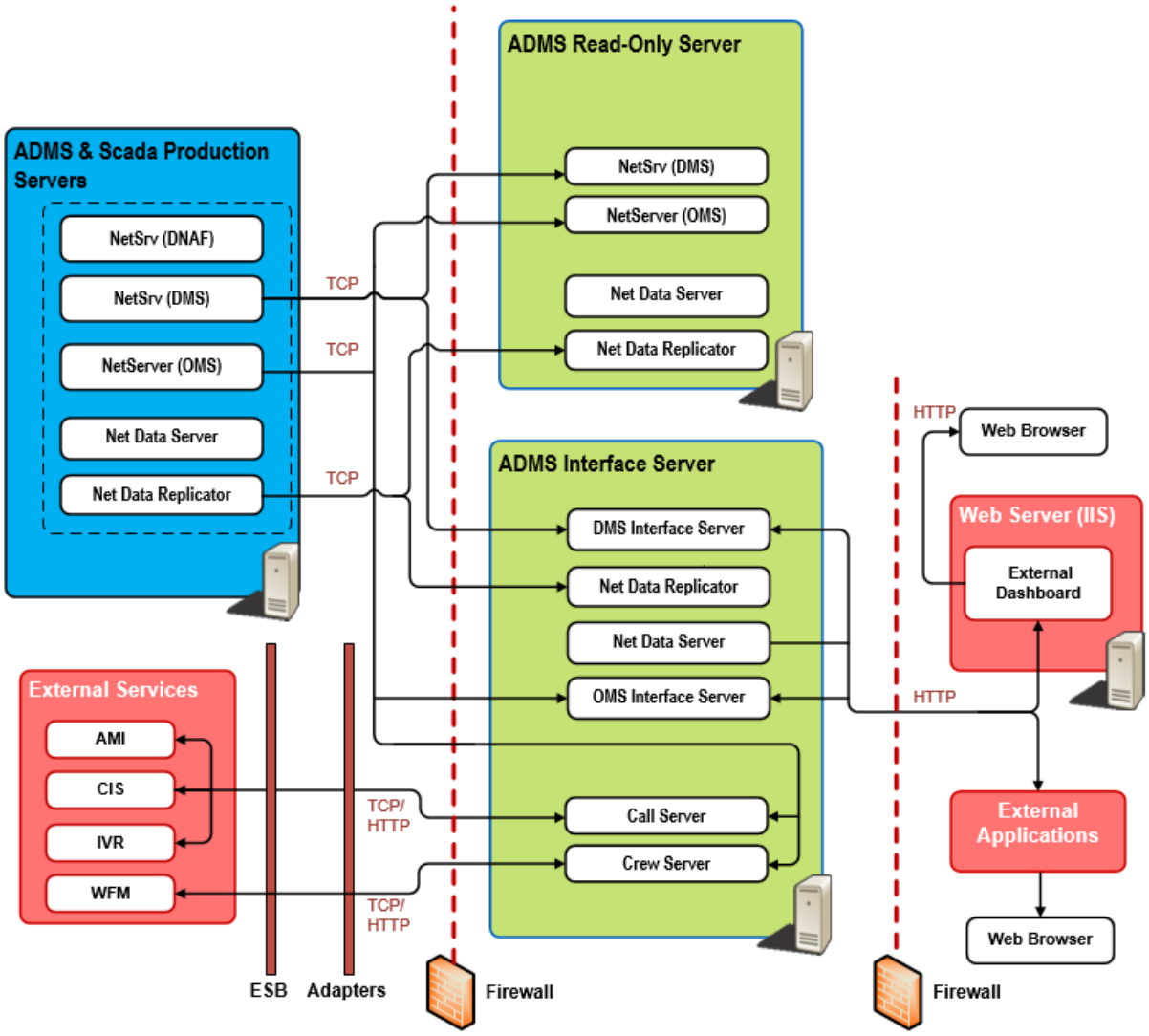


Figure 12. ADMS Interfaces

2.3.1. Outage Management Read-Only Interface

The outage management read-only web service interface is supported by Net Data Server (self-hosted). This interface reads snapshots, journals, crew files, and customer models to provide real time and history data.

The Net Data Server also creates daily snapshots of the system for the last 7 days (or as configured), which helps the system to start faster in case of a complete restart. It also allows system integrators retiring old OMS journal files from the system without losing history.

Interface resources are exposed based on conventions defined in the Open Data Protocol (OData), which is based on Representational State Transfer (REST) architecture. Queried data can be returned in AtomPub or JSON format.

For more information, refer to the *Outage Management Read-Only Interface Programmer Guide*.

2.3.2. Outage Management External Interfaces

These interfaces support communication between ADMS and external interfaces, such as:

- Workforce Management System (WFM)
- Customer Information System (CIS)
- Interactive Voice Response System (IVR)
- Advanced Metering Infrastructure (AMI)

The programming paradigm for these interfaces is TCP-based or HTML connections with XML messaging.

Integration with outage management interfaces is described in section Integration with External Services.

For more information, refer to the *Outage Management Integration Programmer Guide*.

2.3.3. Outage Management Interface

The Outage Management interface is based on the OMS Interface Server and OMS Adapter.

The OMS Interface server provides a standard web server interface to allow third-party applications to communicate with the OMS server (NetServer). The OMS Interface server acts as a client to the OMS server, using the same protocol that the Browser client uses to communicate with the OMS server. This enables access to the same set of OMS data (for example, incident and crew data) as the client. The OMS Interface server can also connect to the DMS Interface server to change device data and perform DMS queries.

The OMS Adapter provides a web services portal for messages among the OMS Interface server and external applications, including the Outage Assist solution (also known as OMS Thin Client). The OMS Adapter receives OMS data from the OMS Interface server. The OMS Adapter supports the Customer Experience, Automated Notification Paging Services, and View Outage Portal web services.

For more information, refer to the *Outage Management Interface Server Programmer Guide*.

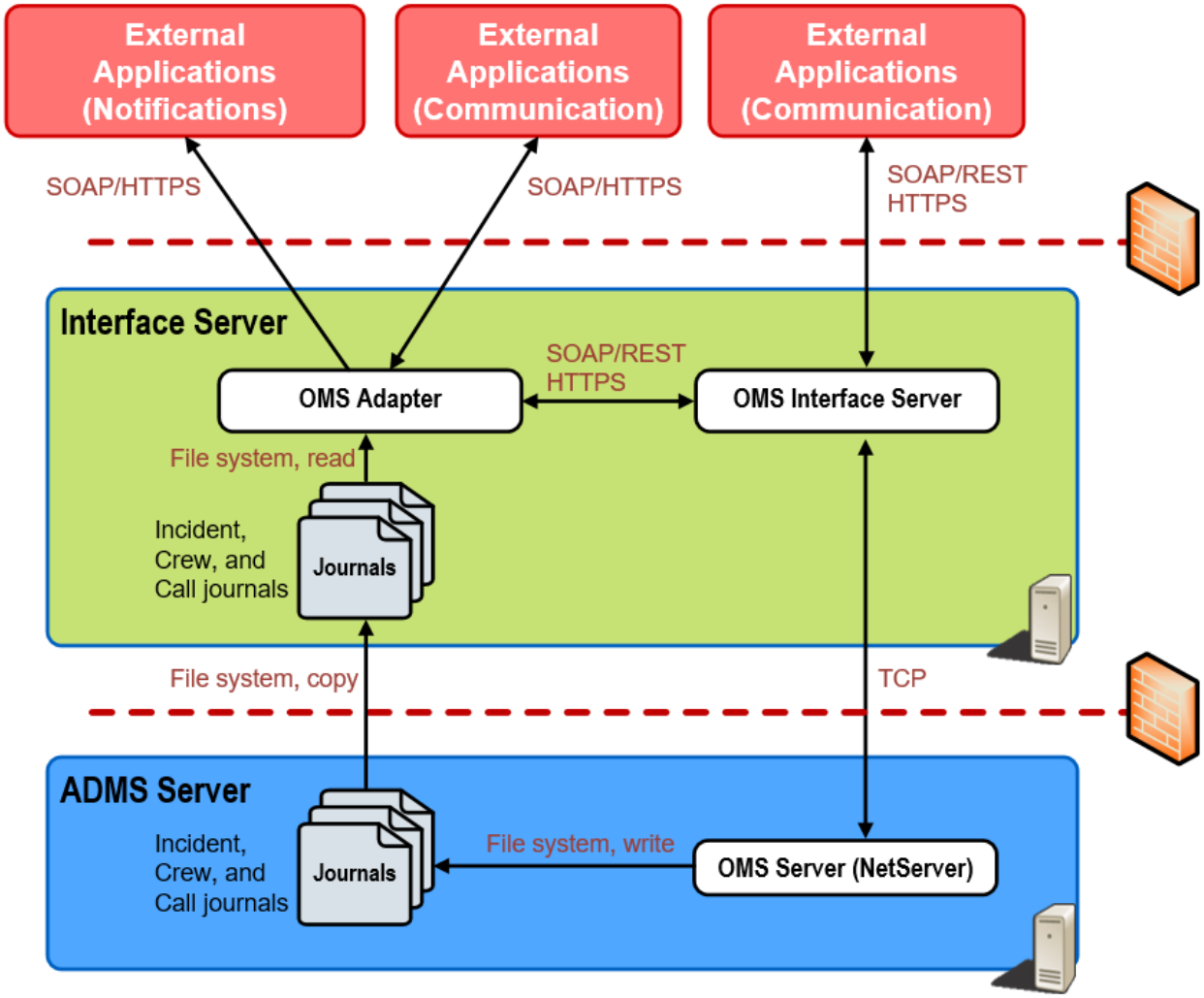


Figure 13. OMS Interface Server and OMS Adapter

2.3.4. Distribution Management Interface

The Distribution Management interface based on the DMS Interface Server that allows communicating with the DMS server. This web service interface supports custom integrations. Currently supported services are:

- **DMS Data Service:** Lists all the available data collections in the system and URLs to access the data, such as. DNAF solutions, annotations, abnormals, and exceptions.
- **DMS Query Service**: Allows external applications to send ad hoc DMS queries to the system. This query interface allows both GET and POST calls.
- **DMS Control Service:** Allows external applications to update DMS data (such as switch statuses and device data), send request to Study server, and update annotations. The service is implemented as a set of REST POST calls. Service calls support both XML and JSON formats.
- **Switching Order Web Service:** Allows external applications to update switching orders. This

Order web service is implemented as a set of REST POST calls. Service calls support both XML and JSON formats.

- **Annotation Web Service:** Allows showing data from external systems on the network view by creating, modifying, and deleting annotations. This web service is based on the DMS Control Service.
- **Schedules Web Service:** Allows external applications to send or update schedule data. This web service is based on the DMS Control Service.

The web interface is implemented using Windows Communication Foundation (WCF). DMS data collections are provided through the OData, which is based on REST architecture. Service calls support both XML and JSON formats.

For more information refer to the *Distribution Management Interface Server Programmer Guide*.

2.4. System Administration Tools

ADMS is supplied with a set of configuration and support tools, to allow system administrators to perform the initial installation, along with ongoing maintenance and restoration of the system.

2.4.1. Configuration Tools

The following sections give a summary of what configuration tools are available. The details are given in the corresponding applications' installation guides.

2.4.1.1. Net Dynamics Configuration Editor

This tool is used to define and modify the instances of the dynamic data that are used by ADMS. While ADMS uses a standard set of static data, a separate instance of dynamic data is defined for each mode of operation (i.e., real-time, study, and simulation).

The editor includes tools to create new dynamic instances, modify the paths for the location of dynamics files, set display background colors for each instance, configure files for saving annotation dynamic data, and define background land-based data files.

2.4.1.2. Viewer Configuration Editor

This tool provides a graphical user interface for modifying the look and feel of the ADMS client, the display of network objects (symbols, zoom, and de-clutter behavior), and the definition of function keys and mouse operations. The output of the editor is an XML file that is read by the ADMS client.

2.4.1.3. DNAF Configuration Editor

There are a large number of parameters that are used in the configuration of the distribution network analysis and network optimization functions. These parameters are all maintained in the DNAF_PARAMS.xml file, which is read by the DNAF function. This parameter file is edited by the DNAF Configuration Editor.

2.4.1.4. Switching Order Configuration Editor

This tool is used to define the switch order document configuration parameters, which include the look and behavior of the switch order displays, the document file name conventions, the server host name and port number, etc.

2.4.1.5. OMS Client Configuration Editor

This tool is used to configure the look and behavior of the outage management displays.

2.4.1.6. OMS Server Configuration Editor

This tool allows customization of the Estimated Time of Restoration (ETR) matrix, call actions, chronology message configuration, and the case note matrix. It puts all of these configurations into one output XML file needed by the server.

2.4.2. Restoration Tools

ADMS supports Snapshot Viewer to view and modify dynamics, snapshots, and journals. This is a highly controlled application, since it allows an administrator to modify data without the application's normal data entry validation checks.

2.4.2.1. Snapshot Viewer

The Snapshot Viewer application allows you to view the entries of dynamics, snapshot, and journal files in a grid view to modify and save them out again. For NetServer and Net Data server, it is possible to apply journal messages up until a specified point in time and then create a new snapshot file with the saved results.

The Snapshot Viewer requires that an administrator restarts the application with the corrected files. Since both the Snapshot Viewer and the applications open journal files for exclusive writing, the administrator must make sure that both applications do not run at the same time on the same journal files.

The Snapshot Viewer is intended only to be used as a debugging tool, or for recovery as a last resort. The system provides displays and commands to perform valid operations on the OMS

database.

The Snapshot Viewer also allows an administrator to view and edit dynamics files. Dynamics files are standalone, such that a dynamics file can be modified in isolation without requiring a modification in another file before the change is valid. The Snapshot Viewer allows an administrative user to view any dynamics file, modify any of the contents, and save the file.

For system restoration, it is necessary for the administrator to either copy the files to a separate location, or coordinate to stop the run-time application manually before saving the file.

It is possible for dynamics entries that no longer link to a valid device to be deleted from an administrative display. These do not require the Snapshot Viewer.

An example of where the display is used is for a cut that is placed on a line, but a GIS import occurs where the original line no longer exists in the system. In this case, the DMS server is unable to apply the cut. The cut can be deleted and potentially re-applied to the system using the normal display technology rather than the Snapshot Viewer.

> **Note**
>
> The Snapshot Viewer is designed for system administrators as a debugging tool. For operation staff to view and make corrections to the cleared incidents, use the History Correction display, which is part of the ADMS client.

2.5. System Integration

This section provides a high-level description of the approach for integration of the internal core ADMS components and of ADMS with external services.

2.5.1. ADMS Integration (Internal)

In general, functions executed entirely within ADMS are coordinated by the Habitat utilities PROCMAN (for startup, shutdown, or sequencing) and CFGMAN (monitoring and role assignment). The applications are integrated with the Habitat permission management server (PERMIT) to provide authentication and authorization for user access to the system.

The communication between server and clients uses a proprietary protocol developed for the ADMS solution. Specifically, ADMS has a defined set of message types that are used for communication. Messages are constructed and serialized into binary form and transferred over TCP/IP.

2.5.2. Scada Integration

When ADMS is integrated with GE's Scada product, SCADA acquires the measurement data and status of telemetered switches from the field, and allows remote control for devices such as circuit breakers. The Scada system also provides the alarming and event logging functionality that ADMS

applications can use to alert operators on events.

Permission checking and device tagging are the other two functions shared by both the Scada and ADMS applications. They are supported by most SCADA systems as part of the core functionality; therefore, ADMS relies on the Scada implementation and data repository for these two functions.

In a Scada/ADMS integrated system, both parties need to consider and honor permissions and tags as they are defined and modified (respectively placed and removed).

2.5.2.1. Real-Time Data and Control

In order to exchange real-time data with the Scada database, ADMS uses InterSite Data (ISD) protocol to communicate with Scada. The main data flow is directed from Scada to the ADMS network server as values and quality flags for analog points change.

Controls sent by the operator from the ADMS user interface are handled as follows:

1. Upon selection of a remote-controlled device (i.e., modeled in SCADA), the ADMS UI invokes the SCADA control pop-up display by its URL command with the appropriate device context information.
2. The actual control sequence is then handled by SCADA, just like controls issued from SCADA displays.
3. Once the control is executed in the field, the new status is obtained by SCADA and transferred to ADMS via the ISD interface.
4. The DMS server updates the DNOM model and sends notification of the changes to connected clients (dynamics transfer).

The figure below shows different types of interactions with SCADA and associated data flows.

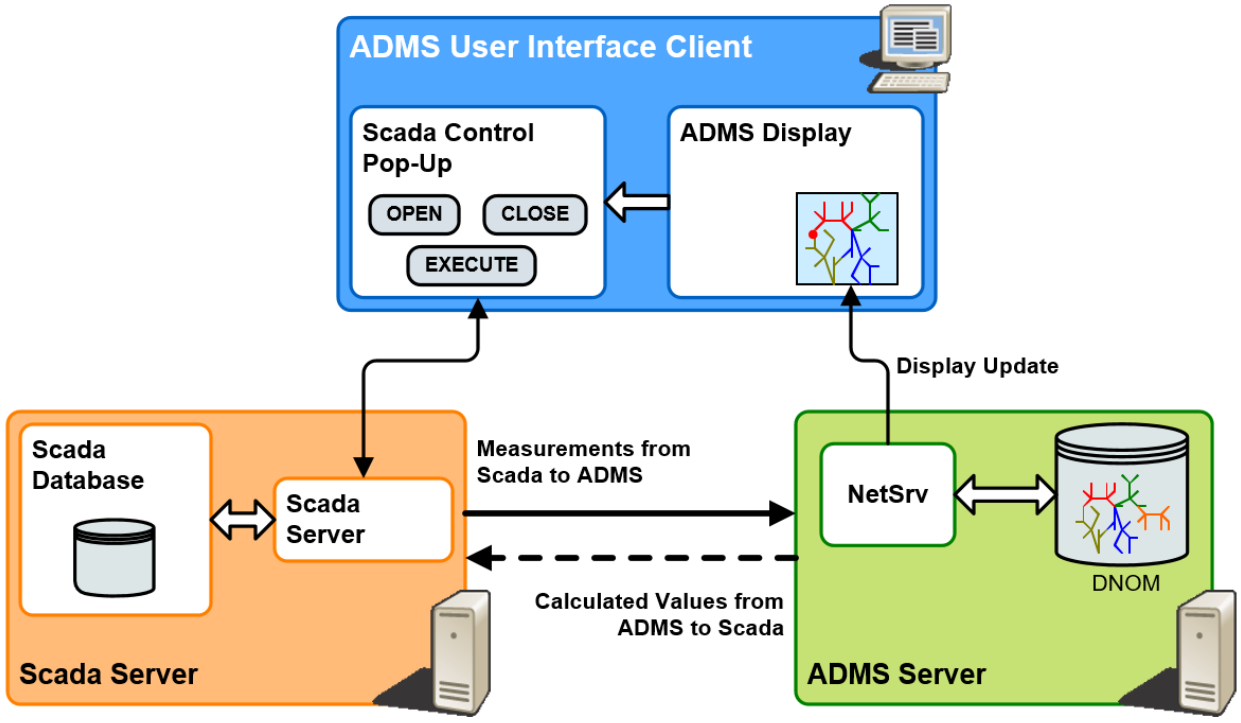


Figure 14. Interactions with Scada

2.5.2.2. Device Tagging

Placing tags and notes is a common service in control centers. In EMS, this is usually viewed as a SCADA function. In a distribution context, many equipment measurements are not telemetered and may not be modeled in the SCADA database, but they should still be tagged. In addition, the volume of tags placed in a distribution system is much larger. The tagging subsystem is enhanced to accommodate the distribution system's needs.

The following is a brief description of the interaction and data flow between the tagging and the ADMS component.

Tags are managed by the TAGGING application and stored in the tagging database (TAGGING). However, the ADMS user interface needs to show tags placed on distribution devices and to allow distribution operators to place new tags from there.

Existing tags, placed on distribution devices, are transferred from Tagging to ADMS using the InterSite Tagging protocol. An initial download is performed at connection; from there, any creation, modification, or removal of tags is transferred incrementally. Tag information transfer is always from Tagging to ADMS.

To allow users to place tags from the ADMS UI, an RUSER command is sent by the software to the Tagging application, which in response opens a tagging pop-up display. From there, the tag placement sequence is handled the same way as from SCADA displays.

Once the tag is entered, the DMS server receives an update via the InterSite Tagging link. In turn, it

updates the DNOM model and refreshes the view on the ADMS client.

For devices that are modeled in SCADA, the tag is placed on the device exactly as it is in SCADA and is placed based on SCADA composite keys. For devices outside the substation fence that are not modeled in SCADA (most of the distribution devices), tags are recorded as being associated with a tagged DNOM object and are flagged as DMSONLY.

The figure below shows the tag data flow and interactions among the Scada server, the DMS server, and the UI client.

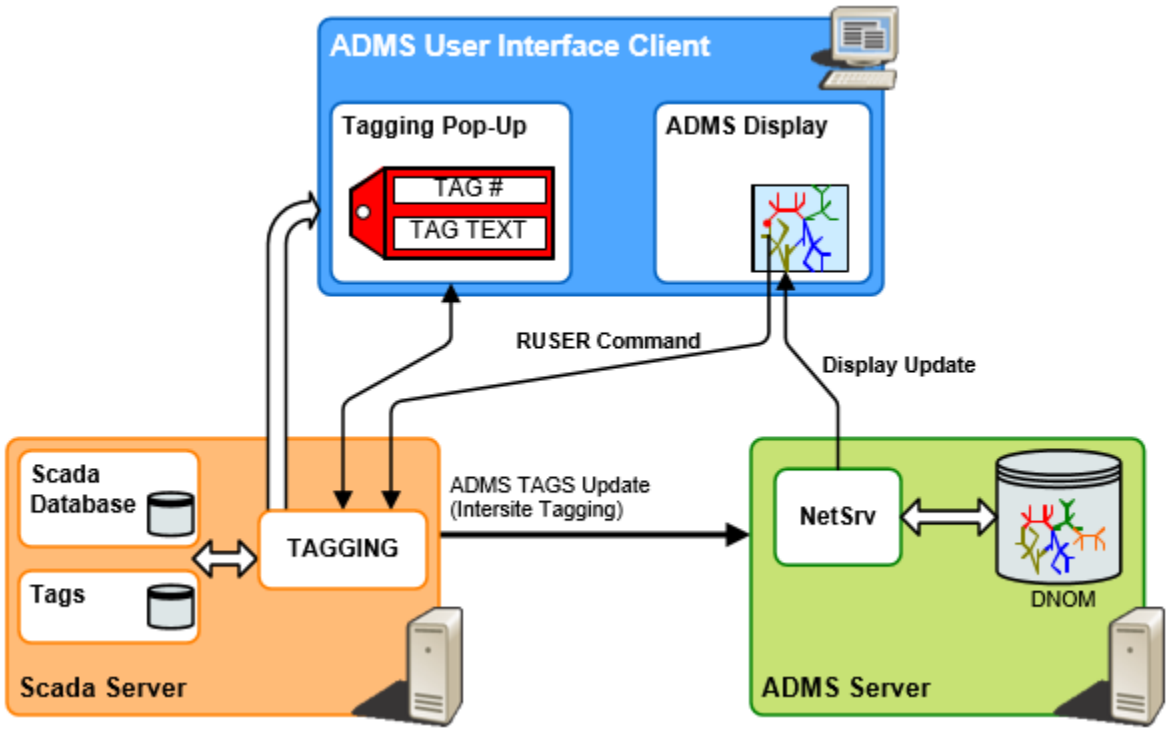


Figure 15. Tag Data Flow

2.5.2.3. Permission Checking

Defining permissions for ADMS is performed in the Scada environment using the traditional PERMIT application. All permission definitions are transferred to ADMS via a TCP/IP link by the Permission Manager proxy server (NETPERMSRV.exe), a Habitat–based application that runs on the Scada/EMS host where PERMIT is running. Upon connection, the content of the PERMIT database is downloaded to ADMS. Periodic updates are sent to make sure that permission modifications are reflected on the ADMS system in real time.

The DMS server is responsible for enforcing permissions as they are defined in PERMIT. Along with actions performed by users, the DMS server receives user identification from the clients, so that permissions can be checked and the action request denied if the user is not authorized.

The figure below illustrates permission checking principles.

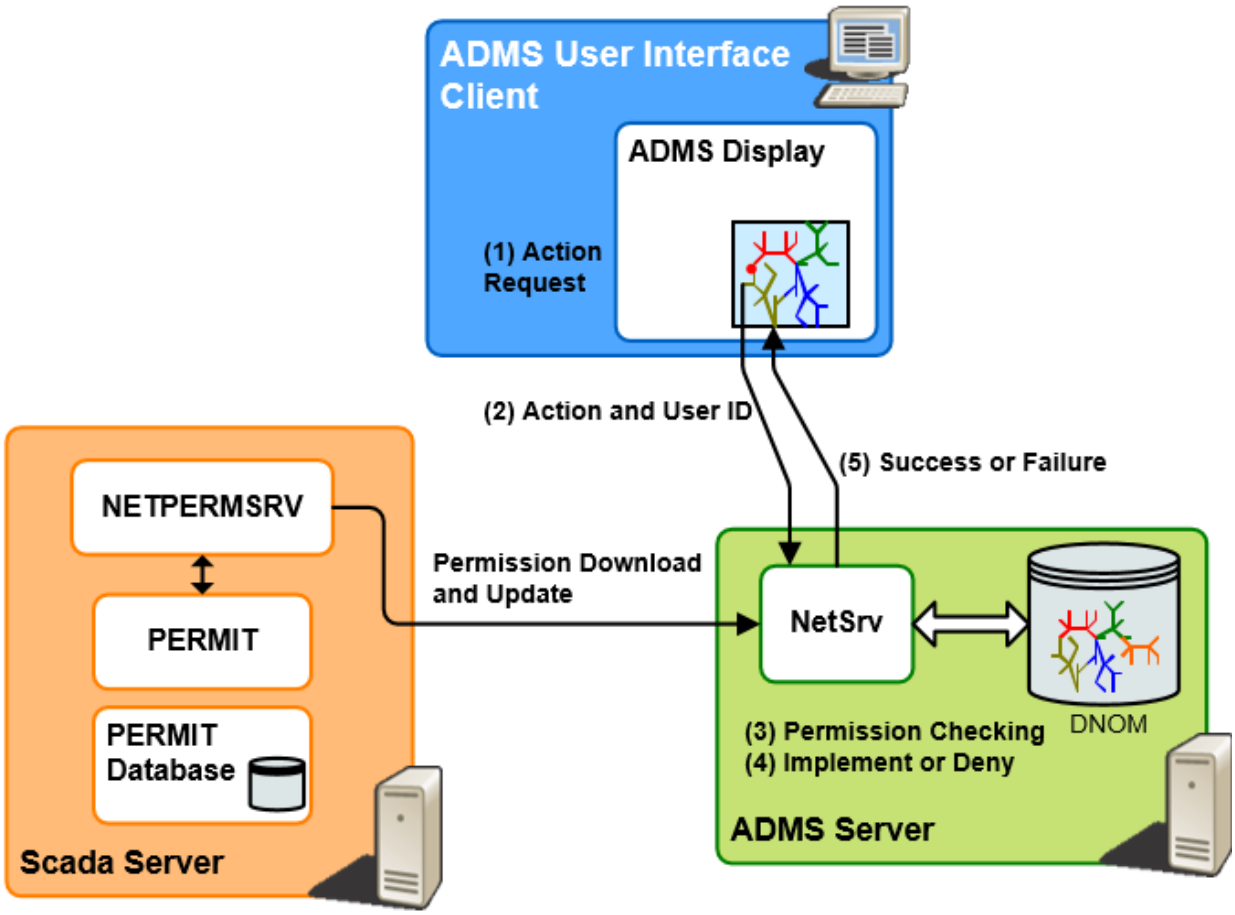


Figure 16. Permission Checking

2.5.3. Historical Data

Archiving ADMS historical data is a collaborative effort of multiple applications. The Historian Sampler application is responsible for archiving the tag and SCADA data, while the ADMS Archive application is responsible for capturing DMS, OMS, switching order, safety document, and reference data.

The Archive application has a separate thread for each data source, including:

- A thread to archive DMS data from the DMS archive journal
- A thread to archive OMS and switching order data from the OMS archive journal
- A thread to archive safety documents
- A thread to archive reference data (static codes and descriptions)

The general structures of the four threads are similar, but the implementation details are different because the nature of the source data and the triggers for archiving data vary.

The Archive application uses Entity Framework and an ADO.NET Data Provider for database access.

This enables a shared data access library to provide the data access logic for both Oracle and SQL Server from a single code set.

Data access to an Oracle database is provided by Oracle Data Provider for .NET (ODP.NET). Data access to a SQL Server database is provided by Microsoft .NET Framework Data Provider for SQL Server (which is a part of the Microsoft .NET Framework).

The figure below shows the data flow of capturing historical data from different data sources.

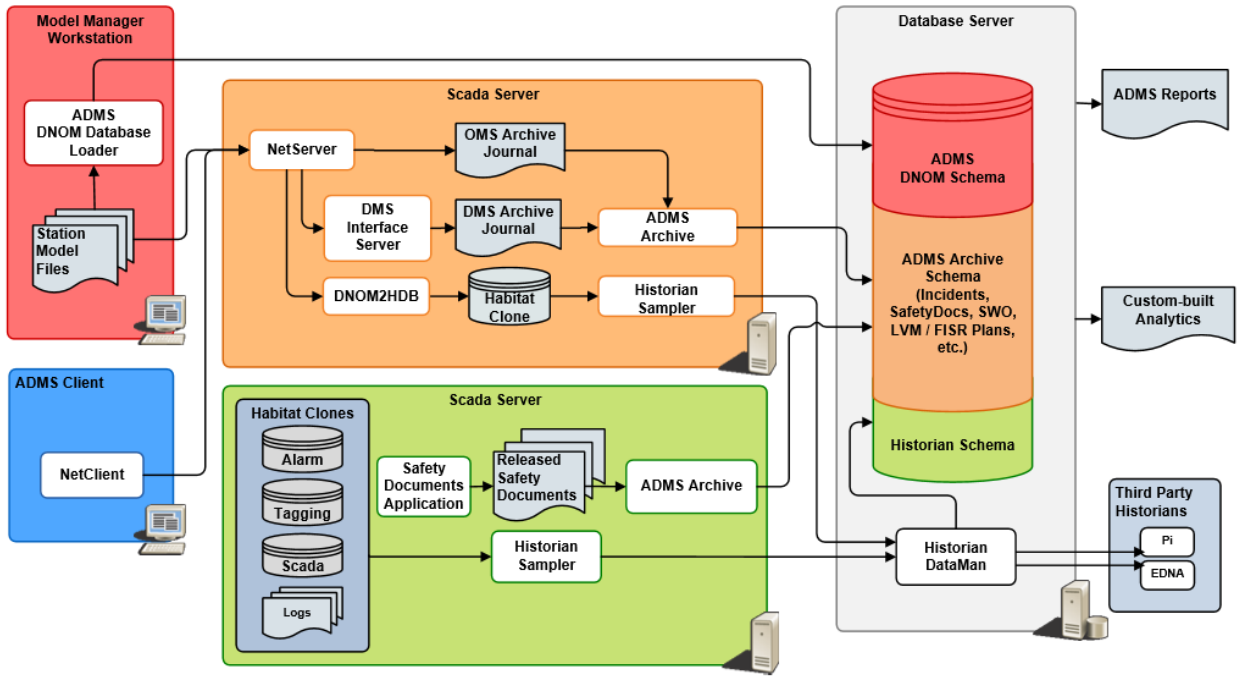


Figure 17. ADMS Archiving Solution

NetServer writes entries in the OMS archive journal when an incident is cleared or cancelled, a switching order is executed or completed, or a switching order step is executed. DMS Interface server writes entries in the DMS archive journal when various events occur, such as when a new FISR solution is available or a new station model file is loaded. The Archive application reads the archive journals and stores the data in database tables for long-term archiving. It also stores the timestamp of the last successful archive entry in the database. Upon startup or recovery from a temporary network connection disruption, the Archive application retrieves that timestamp and continues archiving data that was written to the archive journal after that time.

The Safety Document application writes released safety documents to a designated directory on the Scada server. If a separate instance of the Archive application is not running on the Scada server, File Updater copies the safety documents to a directory on the DMS server. The Archive application periodically scans the directory, and reads and archives the files residing in the directory. Archived safety documents are moved to a separate directory.

The Archive application is capable of updating multiple databases to allow for high availability implementations. Two alternatives are supported:

- Two instances of the Archive application run simultaneously, independently retrieving data

from the data source. Each instance of the application updates a separate archive database.

- A single instance of the Archive application updates a single database. Database replication is used to update a second standby database.

If there is no requirement to allow user updates to the archived data, the two approaches work equally well. The first approach may have an advantage of a lower license fee. However, if user updates are needed, the first approach has the drawback of requiring synchronization of user updates and may require double data entries.

2.5.4. Distribution Operator Training Simulator Integration

Integrating a Distribution Operator Training Simulator (DOTS) is similar to integrating a real-time network management system, since they use the same software components, interfaces, and user interfaces. DOTS is less complex in the sense that it does not deal with redundancy, network security, external services, multiple locations, etc. On the other hand, it is more complex because it requires configuring the servers for different modes of operations, uses different datasets for different simulations, and needs to integrate additional simulation tools. The installation and configuration aspects of the applications are documented in the installation guides.

The figure below illustrates the software modules and their interactions in a DOTS environment.

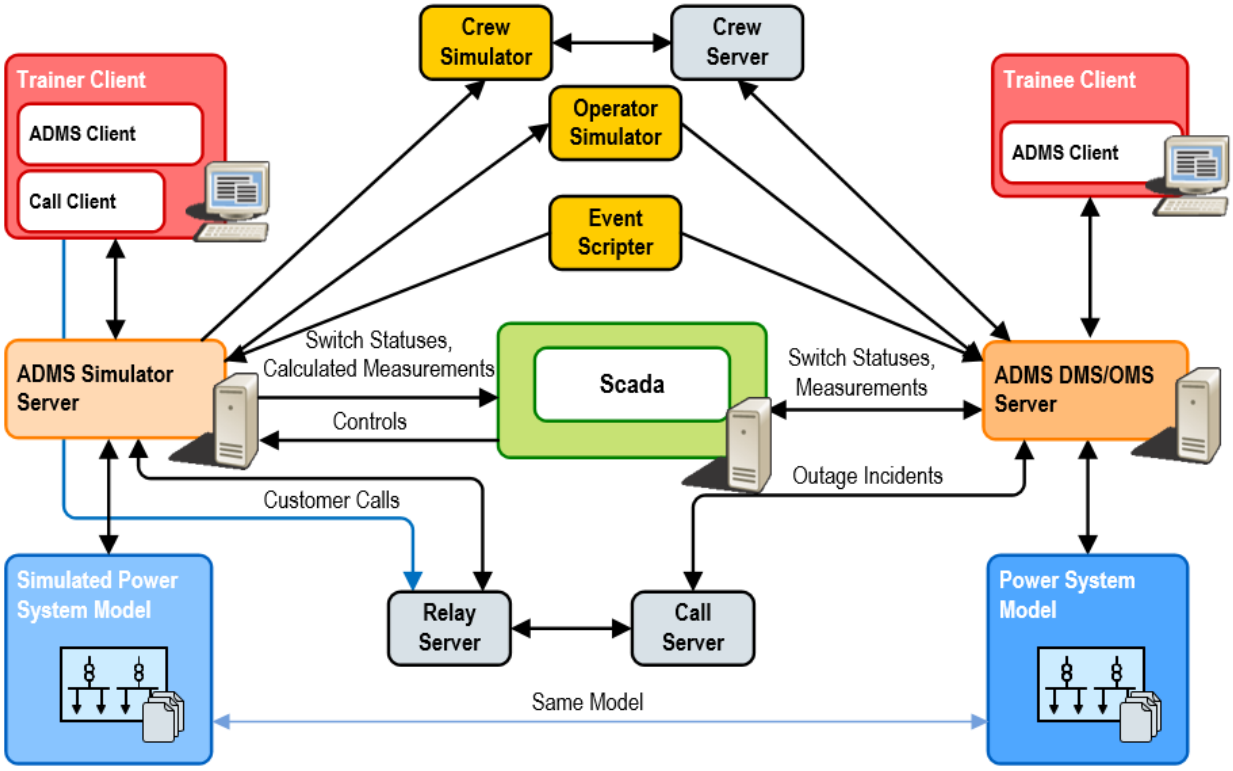


Figure 18. Distribution Operator Training Simulator

2.5.5. Integration with External Services

In addition to the components that are considered part of ADMS, ADMS also requires integration with a number of external services:

- Workforce Management System (WFM)
- Customer Information System (CSS or CIS)
- External Interactive Voice Response (IVR)
- Advanced Metering Infrastructure (AMI)
- Geographic Information System (GIS)

The interface to the GIS is an extraction process that provides model information about the distribution facilities and land-based information to the ADMS model management system. The model management process is described in section Model Management Process.

Within ADMS, two applications manage the interfaces with the WFM, CSS, IVR, and AMI:

- The crew server (Workforce Management System interface)
- The call server (CSS, IVR, and AMI interfaces)

The figure below shows the data flow and interaction between the servers and the external services.

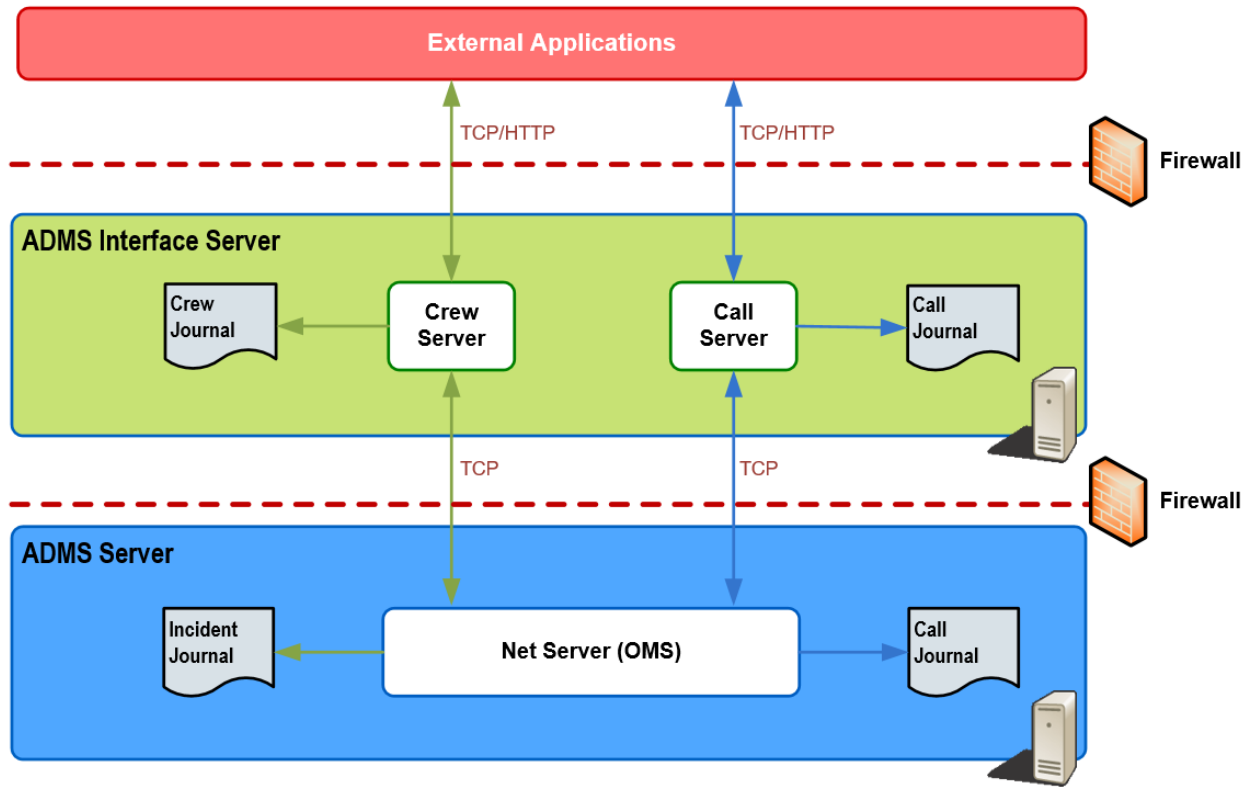


Figure 19. Call and Crew Server Interfaces

The interface among external applications and the call server and crew server is TCP/IP or HTML. Each message is defined by an XSD schema. The message structure includes five parts:

- A buffer header that describes the size and framing of the message.
- A security token based on the next three parts plus a secret shared key.

The security token is created using the SHA 256 hashing function, which generates a 256-bit hash value based on the XML message payload, user SID, current UTC time, and secret shared key.

- The XML message payload that the application uses to pass relevant information.
- A user SID.
- The time of the message, expressed in UTC format.

The messages supported by each interface are summarized below.

2.5.5.1. WFM Interface

The crew server application implements the interface to the workforce management system. It supports the following messages:

- Hello Request
- Hello Response
- Begin Synchronization
- End Synchronization
- Crew Status
- Incident Status
- Incident Update
- Crew Update
- Ack/Nack
- Vehicle Location
- Vehicle Query
- Vehicle Subscribe

Since the number of crews is relatively small, the crew server does not maintain a journal. In the event the server starts up or temporarily loses the network connection, the crew server requests synchronization with the workforce management system to download the latest crew information.

2.5.5.2. CSS and External IVR Interfaces

The call server application implements the interfaces to CSS and IVR. CSS and IVR support the same set of messages:

- Hello Request
- Hello Response
- Ack/Nack
- Call Query
- Call Message
- CallBack Request
- CallBack Response
- Case Note Query
- Case Note Response
- Customer Correction
- Customer Correction Acknowledgement

The call server maintains a call journal. In the event the call server starts up or reconnects to the custom integration component between ADMS and an external system (for example, channel adapter or enterprise service bus), the call server requests the calls since the last entry of the call journal. NetServer itself also maintains a call journal. Upon reconnection with the call server, it requests calls since the last entry of the call journal.

2.5.5.3. AMI Interface

The call server application implements the AMI interface. It supports the following set of messages:

- Hello Request
- Hello Response
- Positive/Negative Acknowledgement
- Meter Event Query
- Meter Event Response
- Meter Ping Request
- Meter Ping Response
- Meter Voltage Event Query
- Meter Voltage Event Response
- Meter Voltage Ping Request
- Meter Voltage Ping Response

Similar to the other interfaces implemented in the call server, the call server and NetServer use the call journals to speed up the synchronization upon startup or network reconnection.

2.6. Deployment Design

This section provides a brief discussion of the deployment of the ADMS software to a configuration of split primary and standby sites. In this configuration, the system is not redundant in each site. Failure of critical software in one site results in failing over the system to the other site.

The deployment design affects the intrasite and intersite failover strategy. Section Intrasite Failover versus Intersite Failover has a brief discussion about how the strategy is affected by the configurations.

2.6.1. Deployment Environments

The ADMS software components are deployed to the following environments.

2.6.1.1. Production (Real-Time) System

The production system is a full redundant system distributed to two sites. The high-level configuration consists of the following:

- **ADMS Servers:** These are the main servers for the ADMS server applications. They include DMS Server (NetSrv), OMS Server (NetServer), Simulator Server (NetServerSim or SimSrv), DNAF Server (DnafSrv), Net Data Server, Switch Order Server, Archive, and Net Data Replicator applications. These servers are fully redundant (one at each of the two sites).
- **Scada Servers:** These are the host servers for the Scada applications and Habitat utilities. They host the Scada applications and other shared Habitat applications such as ALARM, CFGMAN, PERMIT, TAGGING, SAFETY, etc. These servers are fully redundant (one at each of the two sites).
- **Web and File Update Server(s):** These servers provide Browser displays along with file and data serving services to the real-time ADMS UI clients. The number of servers needed depends on the loading and the size of each server. These servers are fully redundant (one at each of the two sites).
- **Model Management Server:** This server runs the model management software, including the model management application and other conversion tools and compilers. It also serves as a study and model review environment. This server can be redundant but not critical.
- **Interface Servers:** These host the interface applications: Call server and Crew server. These servers are fully redundant (one at each of the two sites).
- **Archive Database Servers:** These are Oracle or SQL Server database servers hosting the ADMS archive database and the Historian database. These servers are fully redundant (one at each of the two sites).
- **Read-Only Servers:** These include Read-Only server (NetServerRO) and Net Data server and provide read-only OMS data to thin clients. These servers are not redundant, but multiple read-only servers can be installed in different locations depending on the read-only access needed.
- **Citrix Servers:** These servers host multiple ADMS UI clients in each server. These servers are

not redundant. The number of clients per server depends on the size of each server. The total number of clients depends on the operations needed.

- **Control Center and Support Consoles:** These consoles run the ADMS UI clients. These consoles are not redundant. The number of consoles depends on the operations needed.

The figure below shows a simplified configuration of one production site.

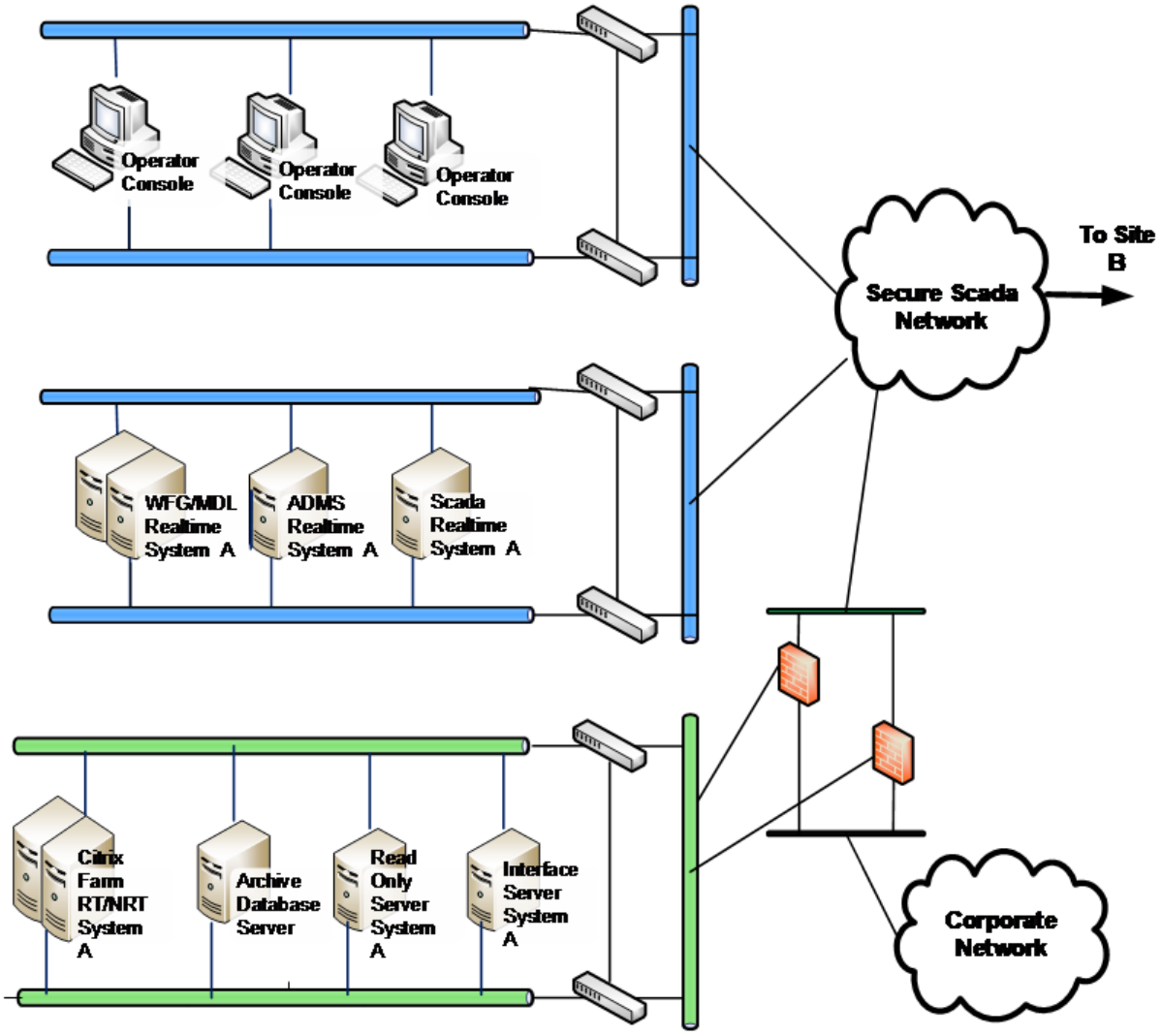


Figure 20. Hardware Configuration of One Production Site

2.6.1.2. QA (Testing) System

The QA system is a full redundant system residing on the primary site. For quality assurance purposes, the software and hardware configuration of the QA system should be as close to the production system as practical.

2.6.1.3. Training System (DOTS)

The training simulator is a non-redundant system. It encompasses all of the real-time software as well as the simulation tools.

- **DOTS RT ADMS Server:** The ADMS server running in real-time mode. It hosts the DMS server (NetSrv), OMS server (NetServer), Simulator server (NetServerSim or SimSrv), DNAF Serve (DnafSrv), and Switch Order server applications.
- **DOTS Simulation ADMS Server:** The ADMS server running in simulation mode. It hosts the NetServerSim (or SimSrv) and Switch Order server applications. It also hosts the simulation tools and the interface servers: Crew Simulator (RoboCrew), Operator Simulator (RoboDispatch), Event Scripter, Relay server, Call server, and Crew server.
- **Scada Servers:** The host server for the Scada applications and Habitat utilities. It hosts the Scada applications and the other shared Habitat applications such as ALARM, CFGMAN, PERMIT, TAGGING, SAFETY, etc.
- **Citrix Servers:** These servers host the ADMS UI clients for training non–control room dispatchers.
- **Trainee Consoles:** These consoles host the ADMS UI clients. The number of consoles depends on the training needed.
- **Trainer Consoles:** These consoles host the ADMS UI client and call client.

3. Availability

3.1. Overview

This chapter describes the high-level availability and resiliency aspects of ADMS, provided to give context to the high availability and resiliency features of the solution.

Discussion of model synchronization and data persistence is deferred to chapter Data Management.

3.2. High-Availability Architecture

The server applications in ADMS are redundant. They run under the control of the Process Manager (PROCMAN) and Configuration Manager (CFGMAN) applications. The PROCMAN application controls startup and shutdown of the applications and execution sequence of the tasks, while the Configuration Manager Proxy server (CFGPROXY) and the CFGMAN application give the ADMS server applications their roles (primary/active, secondary/standby, or offline/disabled). Details on how to install and configure PROCMAN, CFGPROXY, and CFGMAN can be found in the *ADMS Installation Guide*.

The figure below illustrates the major processes and network connections in a typical redundant system.

The NetSrv, NetServer, DnafSrv, Net Data server, Call server, Crew server, and Net Data Replicator applications run in either primary mode or standby mode, as directed by CFGMAN.

There are two reasons for these applications to run in "hot" standby mode: One is to request replication of dynamic data from the primary copy of the application, and the other is to perform all possible initialization so that failover from the primary to the standby server is as fast as possible.

Switch Order server applications can complete their initialization very quickly, and do not require "hot" standby mode. The Switch Order server application can be configured to either run on the same ADMS server machine or run on a separate server machine.

The server applications have connections with the Habitat utilities PROCMAN, CFGMAN, and CFGPROXY (not shown in the figure below). The connections between the Browser clients and the Browser servers are also omitted.

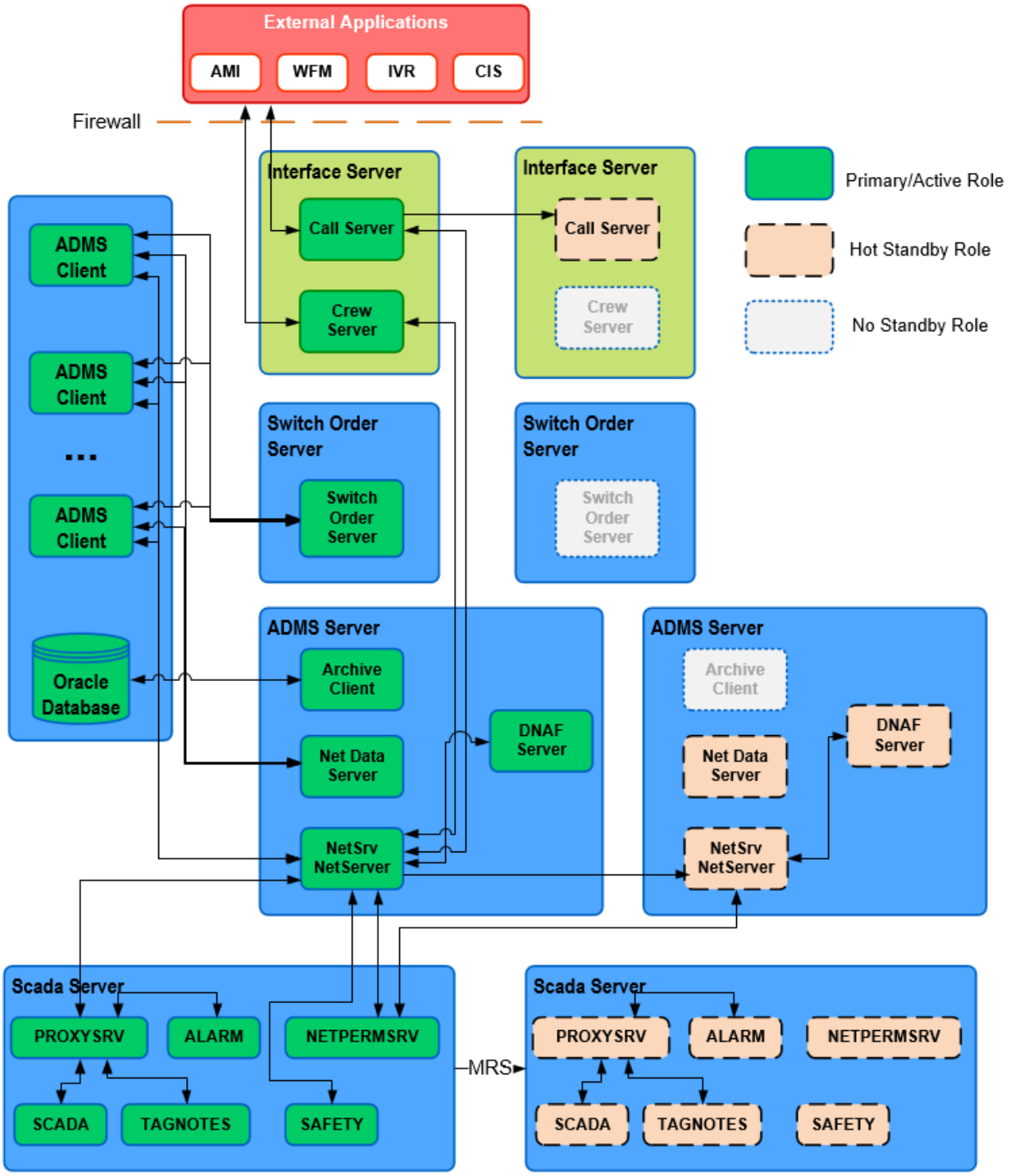


Figure 21. ADMS Redundant Configuration Details

The redundant management of the Habitat and Scada components is detailed in the Habitat and Scada product documentation sets and is not repeated here.

The previous figure does not include the possible additional copies of these applications. For example, additional read-only Net Data server applications can be installed at different locations or

call servers running in relay server mode to facilitate additional call clients. These can be configured to run in either redundant or standalone mode.

The previous figure also does not show the redundant configuration of external components such as Oracle or SQL Server databases and channel adapters. These external components are likely to be found in a different security zone.

3.2.1. Configuration and Availability Features of ADMS Components

This section provides a summary of the configuration options and key availability features of the ADMS components.

Discussion of data replication and recovery is deferred to chapter Data Management.

3.2.1.1. DMS Main Network Server Application (NetSrv)

Main Function: The main ADMS server application performing the core DMS functions.

Executable: NetSrv.exe

Network Connections: It listens for clients as a server. It makes client connections to PROXYSRV for ISD, alarm, and tagging exchange with SCADA, and it makes a client connection for permissions. It connects as a client to CFGPROXY. It listens as a server for a connection for safety documents.

Static Data: DNOM model files.

Dynamic Data: Real-time dynamics, annotation dynamics, fast dynamics, DNAF result file (if configured as DNAF server).

Redundancy and Failover: It runs in dual redundant or multi-site (one primary and many standbys) modes. Dynamic data is automatically replicated.

Failure of the NetSrv application triggers the ADMS server failover. In the event the ADMS server fails over, the network server on the new primary server transitions to the primary role.

3.2.1.2. OMS Server Application (NetServer)

Main Function: Supports OMS functions.

Executable: NetServer.exe

Network Connections: The Network server application listens for clients as a server. It connects as a client to CFGPROXY. It connects to the call server and the crew server as a client. OMS server does not allow SCADA controls to be forwarded to the DMS server. This check prevents access to SCADA controls by OMS users.

Static Data: Customer file.

Dynamic Data: Call journal, incident journal, real-time incident journal, snapshot files, crew file.

Redundancy and Failover: It runs in dual redundant mode with the current role supplied by CFGMAN. It can also run in a multi-site role (one primary and many standby instances). Dynamic data is automatically replicated.

Failure of the NetServer application triggers the ADMS server failover. In the event the ADMS server fails over, the network server on the new primary server transitions to the primary role.

3.2.1.3. DNAF Server Application (DnafSrv)

Main Function: Supports DNAF functions.

Executable: DnafSrv.exe

Network Connections: The Network server application listens for clients as a server. It connects as a client to CFGPROXY.

Static Data: Application input files.

Dynamic Data: Solution files.

Redundancy and Failover: It runs in dual-redundant mode with the current role supplied by CFGMAN. Dynamic data is automatically replicated.

Failure of the DnafSrv application does not trigger the ADMS server failover. The application can be configured to restart automatically after failure.

3.2.1.4. DNA Server (DnaSrv)

Main Function: The Distribution Network Applications (DNA) server supports a modularized architecture for distribution network applications. If enabled, the DNA server runs the Fault Location and Short Circuit Calculation solutions for the DNAF Server with a smaller performance impact.

Executable: DnaSrv.exe

Network Connections: The DNA Server communicates with the DNAF server. The DNA server does not connect to CFGPROXY.

Static Data: Application input files.

Dynamic Data: Solution files.

Redundancy and Failover: It is a standalone application, not redundant.

For more details about the DNA server, refer to the *ADMS Installation Guide*.

3.2.1.5. ADMS Client

Main Function: The main client component for ADMS. It runs as a plug-in within Browser. It includes the geographic view, tabular displays, area world view, pop-up dialog boxes, and navigation within Browser.

Component: NetDisplay

Network Connections: Connects as a client to any number of NetSrv servers. It connects as a client to a switch order server. It connects as a client to Net Data server.

Static Data: DNOM model files, navigation overview file, customer file, and geographic background files.

Dynamic Data: Real-time dynamics, annotation dynamics, fast dynamics, and customer mapping cache.

Redundancy and Failover: Browser is not redundant. As a client, it can support connections to redundant servers (up to four IP addresses).

The failure of the UI client does not cause failover of any server it connects to. The UI client needs to restart manually. In the event of any server failover, the connection is automatically re-established.

3.2.1.6. Switch Order Server Application

Main Function: A document server for the switch orders function.

Executable: SwitchOrderServer.exe

Network Connections: Listens as a server to any number of clients.

Static Data: None.

Dynamic Data: Switch order XML documents.

Redundancy and Failover: It runs as a passive task (with no CFGMAN interface) in a dual redundant environment, with a shared disk.

> **Note**
>
> This may not be possible in a split primary and standby sites configuration. See the discussion in section Intrasite Failover versus Intersite Failover.

The application can be configured to either run on the same ADMS server machine or run on a separate server machine. If the application is running on the ADMS server, it is configured such that failure of the application does not trigger ADMS server failover, and the application restarts. If the application is running on a separate server machine, it is configurable such that failure of the application either triggers the switch order server to failover or just restarts the application. In the event of ADMS server or switch order server failover, the switch order server application in the new

primary server starts up and takes on the active role.

3.2.1.7. Archive Application

Main Function: Archives OMS data to an RDBMS-based archive database.

Executable: ArchiveApp.exe

Network Connections: Connects to an Oracle or SQL Server database.

Static Data: None.

Dynamic Data: Archive journal (read-only) and safety documents (read-only).

Redundancy and Failover: Runs in an N-primary environment. It is a light-weight application and is probably configured to run on the ADMS server together with NetServer. Depending on the redundant configuration of the archive database, there may be more than one instance of the Archive application running on the same server.

Failure of the Archive application does not cause the ADMS server to fail over. The Archive application can be configured to restart automatically after failure. The application does not interface with CFGMAN to get a role assignment. When the Archive application starts up, it connects to the database and assumes that it can start archiving. It queries the archive database to get the last archive entries and continues the archiving from there.

3.2.1.8. Call Entry Application

Main Function: Allows a non–control room user to enter an outage call (for example, in the Trouble Call Backup (TCBU) environment or in DOTS).

Executable: CallEntry.exe

Network Connections: Connects as a client to a relay server (a call server in relay server mode).

Static Data: Customer file (optional; it may also query customer data from a relay server at a slight performance cost).

Dynamic Data: None.

Redundancy and Failover: It is a standalone client application, not redundant, but it can connect to redundant relay servers.

Failure of the Call Entry application does not cause the relay server to fail over. If the Call Entry application fails, the user needs to manually restart the application. In the event of relay server failover, the Call Entry application reconnects to the new primary relay server.

3.2.1.9. Call Server Application

Main Function: Relays calls from a CIS, IVR, or AMI (a "call server"), or relays calls from another call

server (a "relay server") to another call consumer (call server, relay server, or NetServer). It supports the CIS, IVR, and AMI interfaces.

Executable: CallServer.exe

Network Connections: As a server, it listens for clients to send calls or query for calls. It also listens for a connection from the channel adapter and from NetServer. A relay server connects to another call server as a client. A standby call server connects to a primary call server as a client.

Static Data: Customer file.

Dynamic Data: Call journal.

Redundancy and Failover: As a server in the real-time environment, the Call server application runs in a redundant environment. In standby mode, it receives data from the primary. It is configured such that failure of the application either triggers the interface server to fail over or just restarts the application. In the event the interface server fails over, the standby call server transitions to the primary role.

3.2.1.10. Crew Server Application

Main Function: Transfers crew and incident information between NetServer and a workforce management system. It implements the workforce management interface to the channel adapter.

Executable: CrewServer.exe

Network Connections: Listens as a server for a connection from the channel adapter, and from NetServer.

Static Data: None.

Dynamic Data: None.

Redundancy and Failover: As a server in the real-time environment, it runs in a redundant environment. In standby mode, it initializes the port for listening but does not accept any connections. No replication is required (the interface protocol supports a full resynchronization on startup, since the amount of data is small). It is configured such that failure of the application either triggers the interface server to fail over or just restarts the application. In the event the interface server fails over, the standby crew server transitions to a primary role.

3.2.1.11. NetDataServer – Read-Only OMS Server

Main Function: Read-only server for OMS data.

Executable: NetDataServer.exe

Network Connections: It supports a Web service for connection to clients.

Static Data: Customer file.

Dynamic Data: Call journal (read-only), incident journal (read-only), real-time incident journal, archive journal.

Redundancy and Failover: When configured as a primary-standby pair, it obtains a role from CFGMAN (via CFGPROXY). As a standalone replicated server, it always assumes the primary role.

In the primary ADMS server, the primary Net Data server keeps its OMSDB up to date by reading the incident journal file written by Net Data Replicator.

In the standby ADMS server, the standby Net Data server keeps its OMSDB up to date by reading the incident journal file written by the standby Net Data Replicator.

Failure of the Network server application does not trigger the ADMS server failover. The application can be configured to restart automatically after failure. If running in the primary-standby configuration, in the event of ADMS server failover, Net Data server on the new primary ADMS server transitions to the primary role.

3.2.1.12. Net Data Replicator Application

Main Function: Replicates OMS data from enabled servers to read-only and standby servers.

Executable: Net Data Replicator.exe.

Network Connections: On the enabled server, Net Data Replicator listens for connections from Net Data Replicator clients on the standby servers.

Static Data: None.

Dynamic Data: Snapshot, journal, and switch order files.

Redundancy and Failover: Net Data Replicator runs in dual redundant mode with the current role supplied by CFGMAN. It can also run in a multi-site role (one primary and many standby instances). It does not have its own dynamic data; instead, it replicates the dynamic data of other applications (NetSrv, NetServer, Call server, Crew server, and Net Data server).

Standby Net Data Replicator instances periodically send replication query requests to the enabled Net Data Replicator instance, which sends responses containing data to be replicated. Standby Net Data Replicators update their own databases with the replicated data.

When Net Data Replicator starts, if it receives the standby role assignment from CFGMAN, it replicates all data from the enabled Net Data Replicator before confirming its standby role assignment. This ensures that it is not assigned the enabled role before it is ready to become enabled, and therefore data is not lost during the failover.

Failure of the Net Data Replicator application simply results in Net Data Replicator restarting without triggering an ADMS server failover.

> **Note**
>
> Net Data Replicator replicates only OMS data. DMS data is replicated by NetSrv, and SCADA

data is replicated by Memory Replication Services (a component of Habitat).

3.3. Intrasite Failover versus Intersite Failover

There are two possible scenarios for the deployment of the ADMS/Scada servers:

- Primary and Disaster Prevention and Recovery (DPR) Sites: In this configuration, the primary site houses redundant DMS/OMS servers. The servers are connected in a Local Area Network (LAN). At the DPR site, additional DMS/OMS servers (either redundant or non-redundant) reside on and connect to the primary site servers via a Wide Area Network (WAN). In this scenario, intrasite failover is triggered automatically, but intersite failover is probably triggered by a manual intervention.
- Split Primary and Standby Sites: In this configuration, the primary and standby ADMS/Scada servers reside in different locations connected via the WAN. Failure of a primary server automatically triggers a failover and results in a standby server (which is physically located in a different site) taking over the primary role.

> **Note**
>
> ADMS does not support Platform or Scada with the Parallel Site redundancy configuration.

The current communication link between the two sites should have a bandwidth of at least 50 MB/s for large systems.

With the first configuration, intrasite failovers are flexible. In the primary site, the interface server pair, the ADMS server pair, and the Scada server pair can fail over independently. For the applications that maintain documents (such as SAFETY), a file share is a viable option (although it still requires file replication to the DPR site). UI client response time is probably not affected by which servers are running as primary. To keep the DPR site up to date, the servers have to configure as one primary and multiple standby, while only the standby at the primary site can automatically take over the primary role.

The intersite failover is triggered manually. It needs to have some scripts that can help the administrator to restore the system rapidly.

With the second configuration, the system integrator needs to decide whether the interface server pair, the ADMS server pair, and the Scada server pair can fail over independently. In a scenario where the ADMS server is primary at one site while the Scada server is primary at the other site, one concern is whether the communication link can handle the traffic during the heavy load condition. Another concern is whether it affects the UI client response time when an operation involves both ADMS and Scada applications. It is more critical that there is a reliable file replication mechanism in place for the document servers, such as the Switch Order or SAFETY applications.

The first configuration provides higher redundancy, while the second configuration has lower hardware and administration costs.

4. Data Management

4.1. Overview

One of the challenges in administering ADMS is the amount of data exchange along with the number of data files stored in the system. This chapter includes an introduction of the memory-resident databases, a summary of the model management process, a discussion of data persistence, and brief descriptions of the static and dynamic data files in the system.

4.2. Memory-Resident Databases

There are two major parts of the model for a NetSrv and NetServer pair: the Distribution Network Object Model (DNOM) and the OMS information contained in a combination of in-memory model and static files (often referred to as the "OMS database"). The design criteria of these two databases include supporting the following:

- A large model (exceeding 25 million network objects).

  In a 32-bit platform, the model size is subject to the 2 GB per process address space limitation. With a 64-bit operating system, the limitation is more dependent on the CPU clock speed, the number of cores, and the available memory of the server hardware.

- Frequent updates (every day).
- A high volume of planned and unplanned switching activities.
- A wide range of network configurations, including temporary modifications.

Brief descriptions of the two databases are given below.

4.2.1. Distribution Network Object Model (DNOM)

DNOM is the foundation for all of the ADMS applications. It is a phase-domain, memory-resident, object-oriented model. It maintains both the static and the dynamic data describing the distribution network.

The DNOM data structure supports the following types of data:

- Static data: Nominal network model (feeders, transformers, switches, etc. in a nominal state) and UI display object data
- Dynamic data: Non-nominal states of network equipment, including temporary modifications and manual entries
- Output results from a topology processor and DNAF applications

The model includes all electrical equipment and the connectivity of this equipment. While the

memory model is really a graph, with arbitrary object relations, the model is actually persisted in a tree structure for the sake of simplicity.

The root of the DNOM tree is the substation. All other queryable objects are in collections that belong to a substation. Each substation contains a collection of switches, a collection of transformers, a collection of feeders, etc. (as illustrated in the figure below).

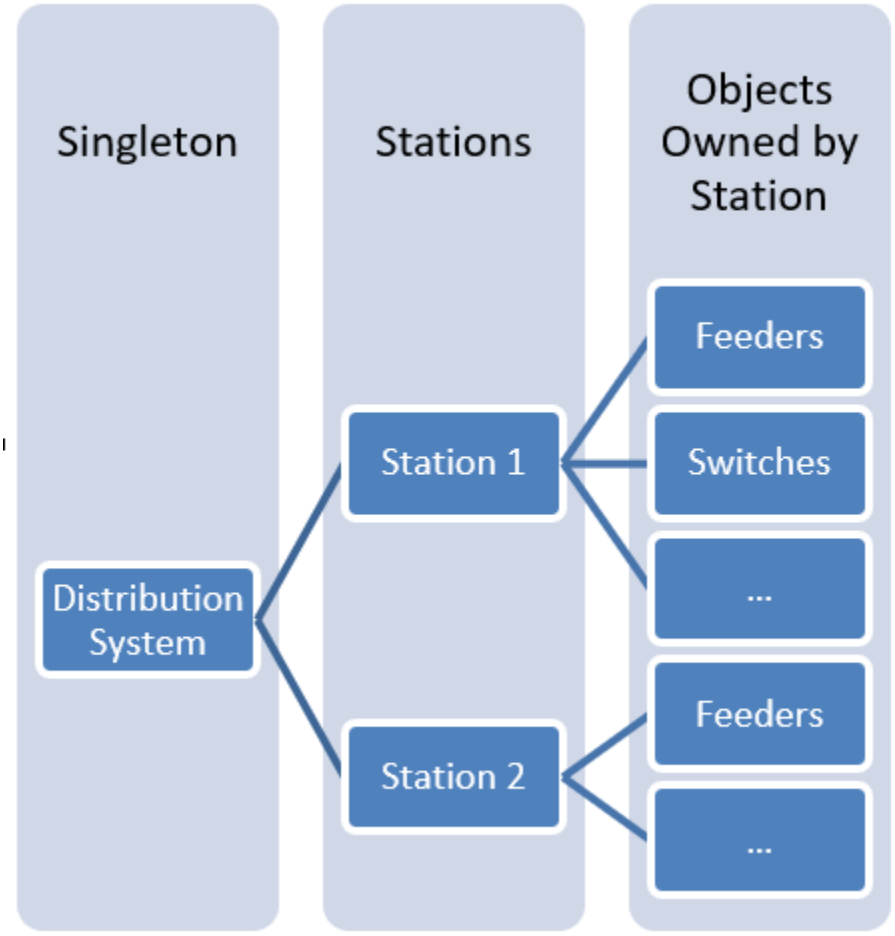


Figure 22. Collections of Objects in DNOM

4.2.2. Outage Management Database (OMSDB)

OMSDB contains data structures that support the outage management activities and switching operations:

- Static data: Mainly the customer information.
- Dynamic data: Trouble calls; planned outages, safety tags, documents; crew information; incidents; AMI; and chronologies.
- Output results from incidents and crew management modules.

4.3. Model Management Process

A diagram of the overall model management process is shown in the figure below.

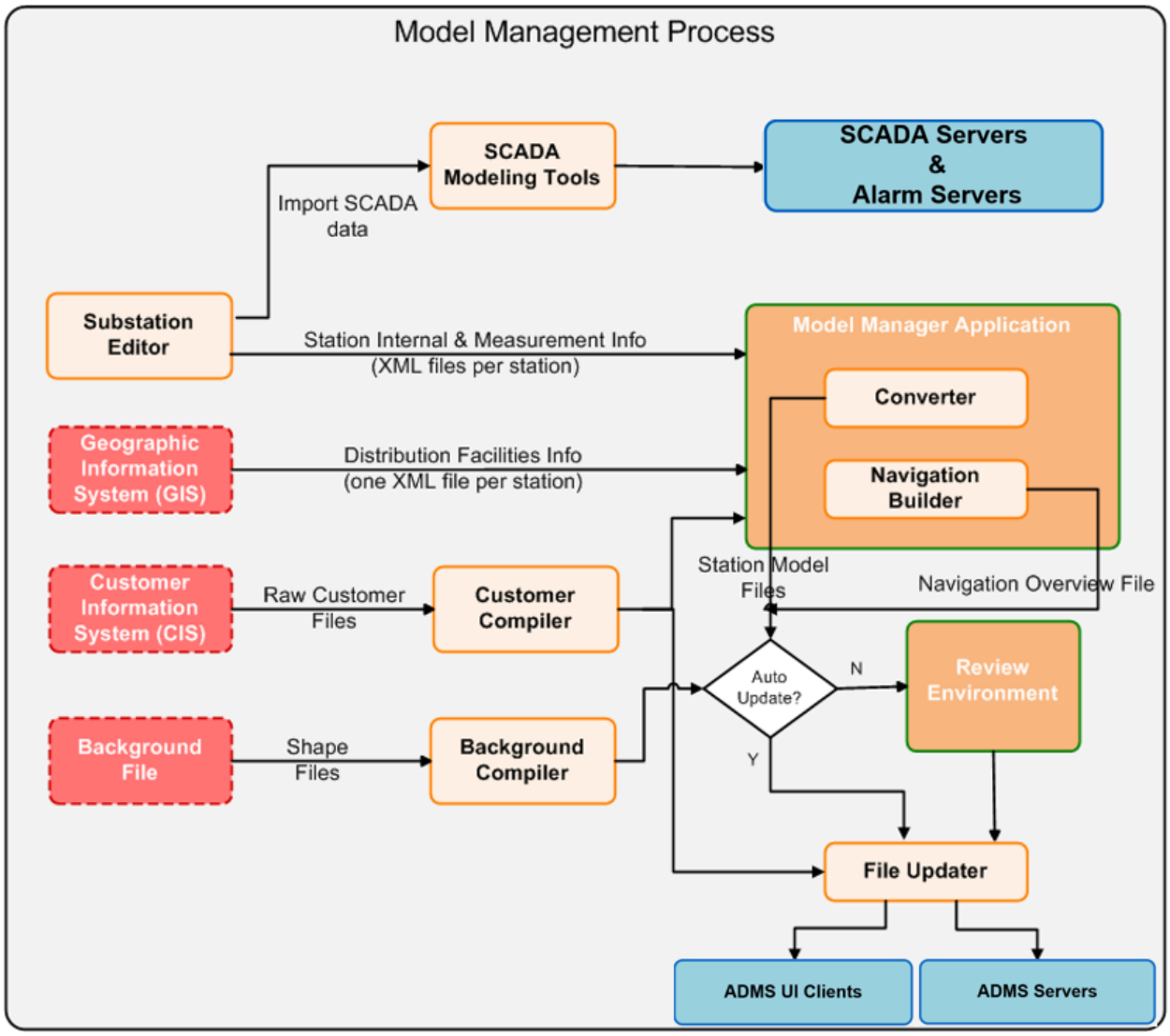


Figure 23. Model Management Process

The input data sources for the ADMS model management environment include:

- Substation Editor (SSE): SSE is the data source for substation internal devices and other devices not modeled in the asset management system (usually GIS). The topology information, facility data, and measurement associations for all devices (inside and outside the substation fence) are defined in the SSE.
- Geographic Information System (GIS): GIS provides data for all feeder devices (devices outside the substation fence). Data extracted from GIS includes network device information, connectivity, topology, nominal status, and non-electrical data such as road and buildings.
- Customer Information System (CIS): CIS provides raw customer files that contain the well-

known fields that are used by applications (for example, meter number and electrical address) and administrator-defined fields (for example, driving directions).

- Background files: The geographic background files contain geographic reference data that describes the land base. These files are either extracted from the GIS or purchased from third parties.

The model data processing steps involve the following:

1. Use SSE to enter model data for devices internal to the substation and devices external to the substation but not modeled in the GIS; to build the network topology; and to create schematic or geoschematic displays. The SSE generates multiple station files, all with an **.**xml file extension, which contain the information required by the model management application. SSE also provides model data files and display definition files that can be used by other Scada modeling tools to maintain the Scada and ALARM models. The data entries and model data export are performed manually.
2. Extract distribution circuit model data from the GIS database. The extraction includes multiple substation files and a substation group file. A substation file contains the facility data for the substation. The name of the file has the station name, an underscore, a unique substation extract number, and an **.**xml file extension (for example, "STATIONNAME_153418.xml"). A substation group file name has the unique extracted number, an underscore, the term "SSG", and an **.**xml file extension (for example, "153418_ssg.xml"). This file contains the group of stations (one or more) that must be validated and placed in the online system at the same time. The file also sets an auto-process flag that determines if the station model file can be placed directly into the online system or must be passed to the review environment for further analysis/validation. The data extraction process can be automated or done manually.
3. Extract and compile customer data from the CIS database. The raw data files from the CIS are usually flat files with a fixed length for each field. The extraction can be a bulk extraction or an incremental extraction. The compiler processes the bulk file and then applies the updates from the incremental file. The output of the customer compiler is a single binary customer file (with the file extension **.**bpcl). This step can be automated or done manually.
4. Compile the geographic background files. The compiler takes the source files in shapefile format and generates the binary background files (*.geo) and the binary search file (*.gsch). The resulting files are moved to the appropriate output folder based on validation and the SSG file auto-process flag. This step is usually done manually.
5. Input the station XML files, the station group XML files, the compiled customer file, and the compiled background files to the model management application. The model management application consists of two major functions (the converter and the navigation builder). It reads the SSG file and determines the stations that are to be converted. Based on the individual station validation status, the auto-process flag (from the SSG file), and the list of stations that must be processed together, the converter generates the station model files (*.mod) and the resulting converter log and analyst files, and copies them to the appropriate folders. Then the navigation builder generates the navoverview.nvw file and copies it to the appropriate folder. The application can continue monitoring the incoming files, and it performs the conversion automatically or executes on demand. The folder structure is configurable.

6. Depending on the validation statuses and the auto-process flags, the model files are either moved to the review environment or transferred (together with the customer file, navigation overview, and land base files) to a folder using FTP or SFTP for the File Updater application. The File Updater application provides file serving services to the ADMS server applications and UI clients and is responsible for keeping the servers and clients up to date in terms of data files.

4.4. Data Persistence

4.4.1. DNOM Persistent Data Storage

The general paradigm of startup for DNOM is for a server or a client to read static model data from disk into memory first, then to read and apply dynamic data to modify the classes initialized with static data to produce the current state of the system.

The application of switch dynamics to DNOM is illustrated in the figure below. The content and storage of the static and dynamic data are described in subsequent sections.

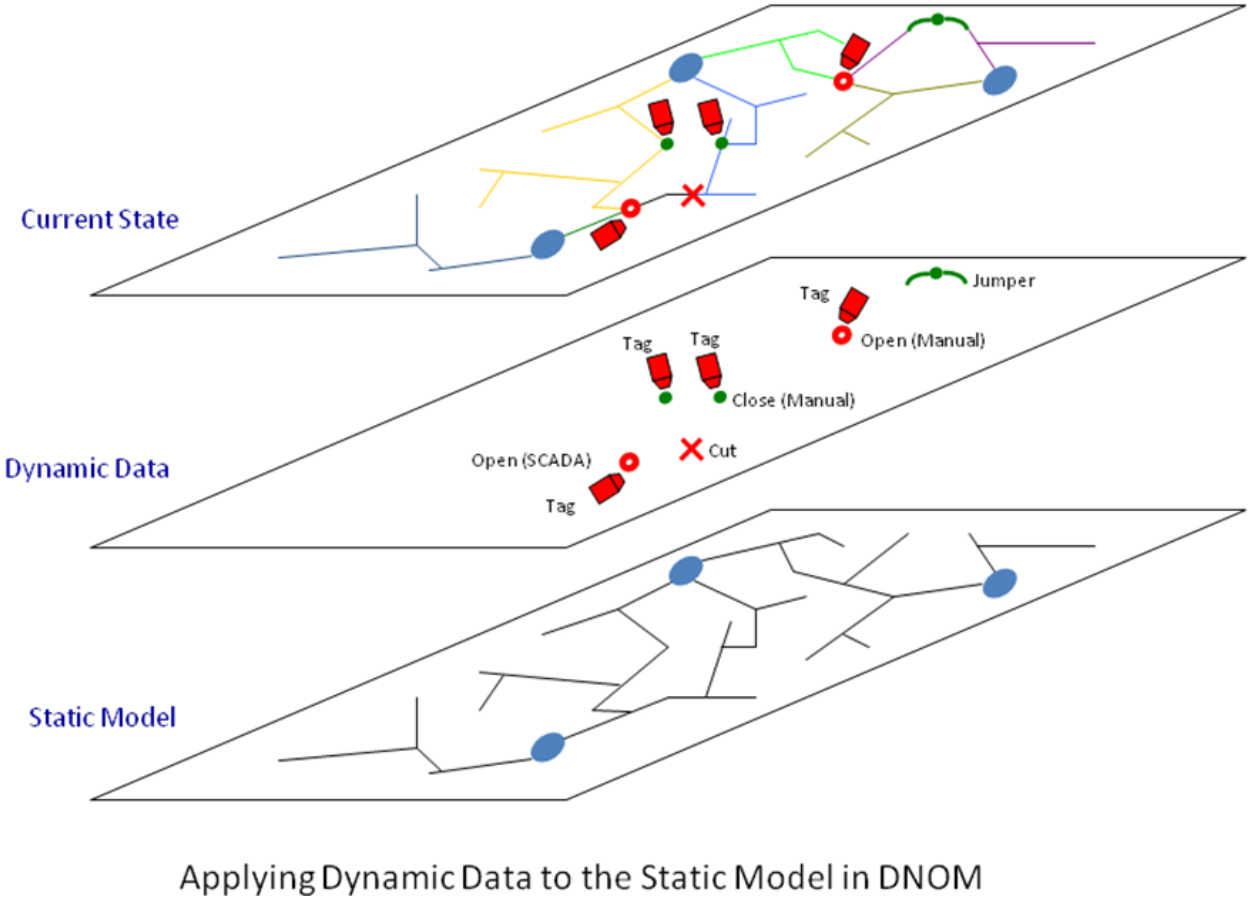


Figure 24. Applying Dynamic Data to the Static Model in DNOM

The static model can be compiled and copied into a directory for the server or client to read it,

independent of the dynamic data. If this occurs, the server or client application reads the new static model as if it were new, applies the dynamic information, and then begins using the new copy of the model. This occurs on both the primary and standby servers independent of the applications that use the static model.

Dynamic data is written under the application's control. While the application is active, this file is locked (for another application to write it). The dynamic data is replicated automatically between NetSrv applications, from NetSrv running in the primary role to any NetSrv applications running in a standby role. The static data is not automatically replicated by NetSrv; it is posted to a third server and independently updated at a slower rate.

A recovery application operates on the dynamic data file only. The new dynamics file can be replaced with a backup file if the application is briefly taken down.

Static data can be placed on disk one time and used by any number of applications running on the same server or workstation, since static data is read-only at run time.

A dynamic data file must be placed on disk in a folder reserved for a single instance of an application (these folders cannot be shared between applications). It is important to configure the system so that dynamic data files are isolated by application; if they are not, the application fails or exhibits unpredictable behavior.

In a Citrix server farm environment, it is possible to configure sharing of one copy of static model files between all of the clients. For dynamic data files, one set of files per user is required.

It is possible for the system to start without any dynamic data. In this case, the power system is initialized to normal, with all switches in their normal state, and no incidents or calls.

4.4.2. OMS Persistent Data Storage

NetServer uses a write-ahead logging scheme for data persistence for OMS. That is, OMS modifications are first written to disk (the code flushes data to disk after every message) before being applied to the OMS database. These modifications are represented by journal messages, which are atomic changes to the system. Example journal messages are an outage call, or a request by an operator to change the Estimated Time of Restoration (ETR) of an incident.

The general data flow for OMS data in real time is shown in the figure below.

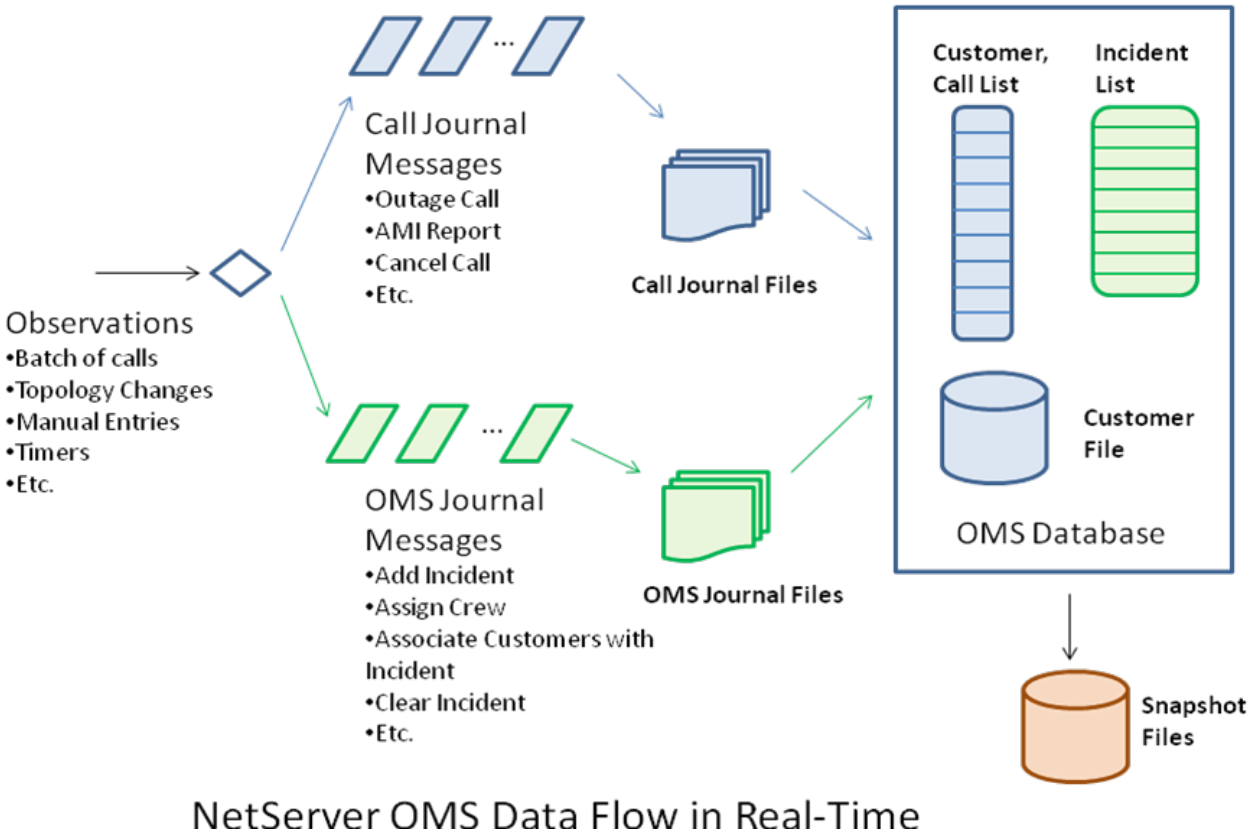
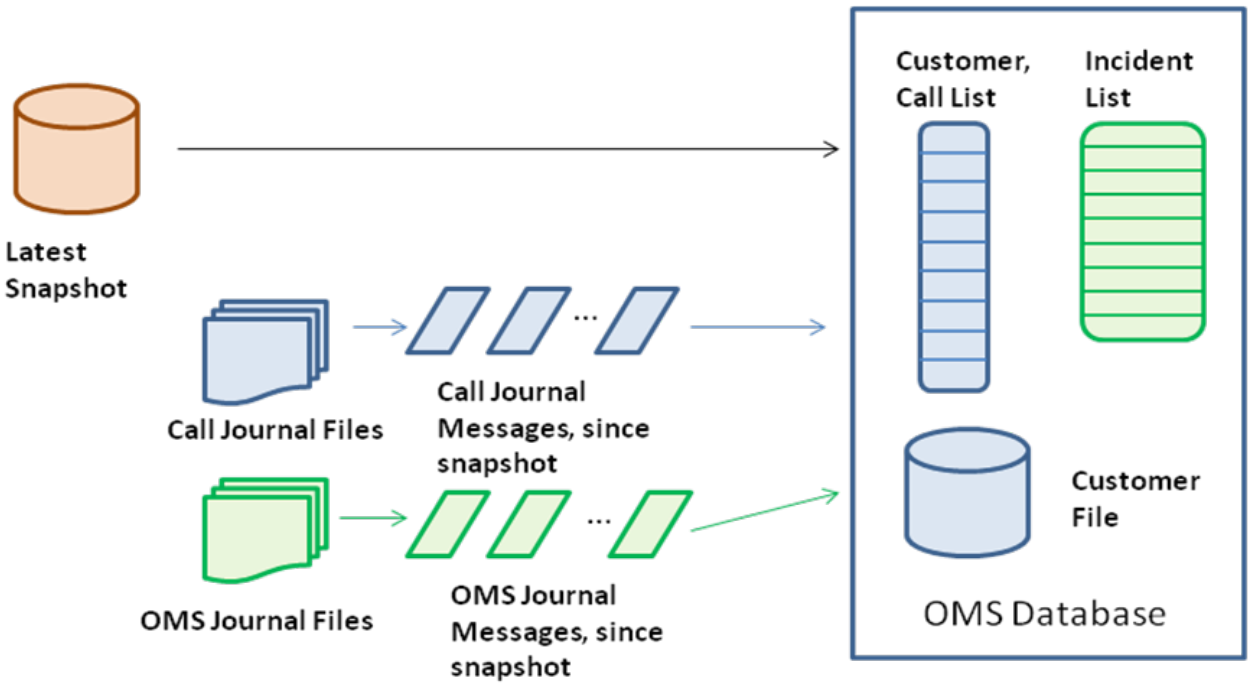


Figure 25. NetServer OMS Data Flow in Real Time

Inputs to the system are serialized through a set of code that calculates one or more journal messages. The journal messages are written to disk, and then the journal messages are applied to the OMS database.

Periodically, snapshots are created that contain a consistent state of all information in the OMS database. These are used in startup only, as shown in the figure below.



OMS Data Flow on NetServer Startup

Figure 26. OMS Data Flow on NetServer Startup

When NetServer starts, it creates a nominal OMS database (all customers are energized, and no incidents). Then NetServer finds the latest snapshot file and applies it to the OMS database to create a starting point. The snapshot file stores the time of the latest entries in the call and incident journals, and these are used to query the call and incident journals for all messages since the snapshot. If no snapshot file exists, only the journal files are used for initialization.

For more information, refer to Managing OMS Journals and Snapshots.

4.5. Static Data Files

The static model files are usually created and maintained in the model management environment and copied to the real-time environment for read-only usage. The static model files are required for the ADMS server applications as well as the ADMS UI clients.

Updated files need to be copied to all of the server machines and the UI workstations (including the Citrix servers, where one copy of these files serves all users of the server). The File Updater application performs the static model file copying activities. Depending on where the machines reside, there is a slight time difference when the files arrive at the machines. It becomes a requirement for the ADMS applications to handle this temporary model out-of-sync condition.

4.5.1. DNOM Model Files

Model files have a file extension of **.**mod. Model files are created by the converter application, in the offline model management environment. Model files are read by NetSrv and NetServer and the client component (NetDisplay).

Model files are partitioned so that each file contains a single substation's worth of electrical model, including the equipment within the station and all of the circuit information to the tie points with other stations. When the static data is replaced, it is possible to replace a single substation or the entire model, or anywhere in between.

Model files are serialized by the converter and de-serialized (from disk into memory) by applications that need the data. Model files include a header to describe the fields contained in the file, so that it is possible to have upward and backward compatibility between model files and the applications that use them (at least to the extent that applications can function if fields are missing).

4.5.2. Navigation Overview File

The navigation overview file is named "overview.nvw". This file is created by the NavBuilder application, in the offline model management environment. This file is read by the client.

The navigation overview file contains polygons representing the geographic extent of each substation, and a graph of potential connectivity between each station and its immediate neighbors, referred to as the station's "adjacency".

The polygons in the navigation overview file allow the client to call up a geographic display given an X,Y location, and also for the client to show the navigation overview display. The adjacency information allows the client to select additional stations for view, using a partial model view algorithm.

4.5.3. Customer Model File

The customer file has a file extension of **.**bpcl. This file is created by the customer compiler in the offline model management environment. The customer file is read by NetSrv, NetServer, Net Data server, Call server, and NetDisplay. The Call Entry and Converter applications can optionally read the customer file.

The customer model file contains all customer information in the system. The data in the customer file is a combination of fields and index structures, to more quickly find information in the file. The fields are a combination of well-known fields that are used by the application (for example, meter number and electrical address) and administrator-defined fields (for example, driving directions).

When an application "opens" the customer file, it does not read its entire contents into memory as it typically does with other files. Some queries to the file cause the code to seek in the file for some calculated location, and to read a few bytes.

4.5.4. Geographic Land Base Files

Geographic land base files have a file extension of **.**geo; these files are split into arbitrary grids. A single file with an extension of **.**gsch is used for searching the land base. Additionally there are **.**xml files that are used for searching the land base that contain polygons of areas to limit the search.

Land base files are created using the geographic background compiler application in the offline model management environment. The files are read by the NetDisplay component when the user calls up a geographic display.

The file name of the **.**geo file includes the X,Y extent of the information contained in the file, so that it is not necessary for the client to read all land base information for the system into memory when calling up a display.

4.5.5. OMS Polygon Files

Polygon files used for the area world view are stored in XML files (for example, oparea_polygongroup.xml).

Polygons for substations and feeders (used by the area world view display) are read from the model files. Polygons that are not part of the electrical model are included in the OMS polygon files (for example, operating center polygons).

4.6. Dynamic Data Files

The term "dynamic data" refers to all of the actions and events that are applied on top of the static description (or nominal state) to reflect the current state of the system. ADMS supports the following dynamic data.

4.6.1. Dynamics Files

Dynamics files have a file extension as specified in the configuration files (typically, **.**sav). These are created by NetSrv, NetServer, and the ADMS UI client (NetDisplay). Client NetDisplay components and standby NetSrv and NetServer applications query for the latest dynamic data from the primary NetSrv and NetServer, which save increments in their own copy of the dynamics file. Annotation dynamics files can be created offline by using the annotation importer application.

There are dynamics files for switches and temporary objects such as cuts, jumpers, grounds (the "main" dynamics file), manual entries, and mobile substations. There are also a number of annotation dynamics files, as specified by an administrator. The annotations can be split into multiple files to allow for some of them to be imported from an outside source. For example, hospitals and police station annotations can be imported from an outside database, but operator notes are only created in the online system.

Dynamics files are used in conjunction with a serialized block class, which is the memory

representation of the data. A serialized block has the property that it can support queries of all changes since a specified sequence. Dynamics files also support a "deleted" flag on each dynamics entry to say that the entry is not needed anymore.

A dynamics change for a single device is only stored once in the dynamics file. For example, if a normally closed switch is changed to open, the server creates an entry for the device and stores its new state as open. If the switch is closed again, the entry is marked as deleted.

The server saves the dynamics at the end of each day to a file with the date in the name of the file, compresses the dynamics file by writing a new file with only non-deleted entries in it, and increments the sequence ID. It is possible for an administrator to restore the system by renaming one of the saved dynamics files (or this file can be used to initialize a DOTS environment).

In addition to the daily compressed dynamics file, the dynamics file is also backed up periodically (for example, every 15 minutes, configured by an administrator). In the unlikely event that file corruption occurs, an administrator can manually replace the latest dynamics file with the "last good" file.

A client initially reads a local copy of the dynamics file and applies it. This way, the client can restore the "last known state" even if the connection to the server is not available. Then, upon connection to the server, the client queries for all changes since the current client's sequence number. The server returns all incremental dynamics entries that occurred while the client was down. The client requests include a Global Unique ID (GUID) stamp for the dynamics. If there is a mismatch between the client's and the server's GUIDs, the server returns all dynamic entries since sequence 0, and the client abandons its old, outdated dynamics files. The client then queries based on the latest sequence number of the returned dynamics (which typically return a null result if no changes have occurred). If a new dynamics change occurs, only the new changes are sent to the client.

At startup (or on display call-up in the client), dynamics are applied to the DNOM model, and the topology processor is initiated. The dynamics files are used to persist the system state after a server restart.

Dynamics files include a header that specifies the records and fields in the file to allow for backward and forward compatibility between files and the applications that use them.

Location:

- DMS dynamics files are located in the dynamics\server folder.
- OMS dynamics files are located in the dynamics\server\OMSServer folder.
- Client dynamics files are located in the dynamics\client folder.

Replication: Replicated automatically by NetSrv and NetServer, from NetSrv and NetServer running in the primary role to the NetSrv and NetServer applications running in a standby role.

Dynamics Files	
dynamics.sav	Main dynamics file where the dynamic data for switches and temporary objects is written. This file is periodically backed up with the name dynamics_<yyyymmdd>_<hhmm>_bckp.sav.

Dynamics Files	
Annotation_<Annotation_Type>.sav	Annotation dynamics files. A separate file is created for each annotation type. For example, Annotation_KeyCustomer.sav. The files are periodically backed up with the name Annotation_<Annotation Type>_<yyyymmdd>.sav.
manuals.sav	Stores data that was manually entered via data entry pop-ups or manual input tabulars (for example, Manual Entry Parameters pop-up, Line Data Entry pop-up, and Line Manual Input grid tabular display). The file is periodically backed up with the name manuals_<yyyymmdd_hhmm>.sav.
Templatedynamics.sav	A file that contains dynamics header data. Used by the Snapshot Viewer application.
preservedstates.sti	Stores the states of switches at the time of dynamic compression. When dynamic compression occurs, the existing file is backed up with the name preservedstates_<yyyymmdd_hhmm>.sti and a new file with the current states of the switches is created. The preservedstates.sti and dynamics.sav files are related. The content of preservedstates.sti must correspond to the content of dynamics.sav for correct switch orphan processing operations, such as load-shift and phase-shift.
all_flow.fldr	Stores the NTP flow direction data. The flow information is used in case of a server restart or failover to correctly determine the exact device that opened to cause a network outage (in particular, this is relevant to boundary switches). When NTP processes the network, it sets the "upstream node" information on every branch in the DNOM model. Each time NTP runs on a station, the upstream indicators are reset for every processed part of the network, except for the de-energized parts. This allows representing the system's state in terms of the flow before the section of the network lost power.
dynSchedule_year_<month>_<day>*<hour*><minute.dsc>	Stores dynamic schedules data for the specified time. Dynamic schedules are loaded by an external system using the DMS Interface server functionality.
all_loads.outage	Stores load outage and restoration times for cold load pickup effect calculations. This file is written by DnafSrv.
all_meas.meas	Stores Scada measurements modeled in DNOM. This file is written by NetSrv.
all_der_meas.dmeas	Stores Scada measurements for Distributed Energy Resources (DERs). This file is written by NetSrv.
ns_meas.meas	Stores Scada measurements not modeled in DNOM. (These measurements are considered simulate-only measurements; they can be modified in simulator mode using the Event Scripter.) The ns_meas.meas file is written by NetServer Simulator and stored in \dynamics\server\SimServer\.

4.6.2. Journal Files

Journal files have a file extension of **.**txt. Although the file extension **.**txt is overused in a Windows environment, it has some advantages when importing a file to the Microsoft Excel application.

Journal files are used by the NetServer, Net Data server, Call server, and Archive applications. There are four instances of journals:

- A call journal (read/write access by the call server and NetServer, read-only access by Net Data server)
- An incident journal (read/write access by NetServer, read-only access by Net Data server)
- A real-time incident journal (read/write access by NetServer or Net Data server)
- An archive journal (read/write access by NetServer and DMS Interface server, read-only access by Net Data server and the Archive application)

Journal files are the persistent storage for a NetJournal class. A NetJournal is similar to a stream where messages can be read in time order. A NetJournal object can only be queried by a timestamp, and the query returns all messages, in order, since the given timestamp.

Journal files include header records (the first record in the journal tile) that specify the layout of the fields in the file, for backward and forward compatibility. Journal entries contain a timestamp that is guaranteed to be always increasing. Since the header record defines what fields are required for each message type, the reader applications use this information to detect partial records. Reader applications periodically check for new files and new messages into files. If there is a new message, the reader reads it. When a message is read, the fields present in the message are compared to the header so that incomplete messages are detected. When that happens in the middle of a file, the reader discards the message. If the message is at the end of the file, the reader tries reading it again on the next scheduled read.

A journal file has a programmatically controlled maximum number of entries (currently, 10000). When a journal is "opened" for write, the journaling code creates a new file with header information. When an entry is written to the journal and the existing file has the maximum number of entries contained in it, a new file is created. A new file is also opened for the first entry written after midnight.

The file names contain the date and time based on the file open time, and any new files are guaranteed to have a later time than all previous files.

After an administrative-defined time, the previous journal files are moved to an archive folder, where they can be archived and/or deleted.

Location: Folders with journal files are located in the dynamics\server and dynamics\server\OMSServer directories.

Replication: All journal files are replicated from the primary to a standby server by Net Data Replicator.

Journal Files and Folders	
IncidentJournal_<yyyy_mm_dd_hh_mm_ss>.txt	Contains incident related data (name, location, device, status restoration steps, crews, etc.). IncidentJournal files are written by NetServer.
RealTimeIncidentJournal_<yyyy_mm_dd_hh_mm_ss>.txt	Contains Near Real-Time (NRT) outage management and historical data, such as incident, planned outage, and area note information. RealTimeIncidentJournal files are written by NetServer or Net Data server.
CrewJournal_<yyyy_mm_dd_hh_mm_ss>.txt	Contains crew assignment messages from CrewServer to NetServer. CrewJournal files are written by NetServer.
OrderJournal_<yyyy_mm_dd_hh_mm_ss>.txt	Contains entries for follow-up work orders. OrderJournal files are written by NetServer.
ArchiveJournal_<yyyy_mm_dd_hh_mm_ss>.txt	Text files that contain DMS and OMS data (for example, incidents, switching orders, safety documents, and DNAF application results) to be archived in a relational database by the Archive application. There are two types of Archive journals: OMS and DMS. The OMS Archive journal is written by NetServer. It contains historical data that NetServer no longer needs (for example, cleared incidents and/or completed switching orders). When NetServer starts, it does not load the OMS Archive journal data into memory. The OMS Archive journal is stored in \dynamics\server\OMSServer\ArchiveJournal. The DMS Archive journal is written by DMS Interface server. It contains DMS and DNAF data including DNAF Solution results, station and feeder exceptions, and network model file information. The DMS Archive journal is stored in \dynamics\server\InterfaceServer\ArchiveJournal.
CallJournal_<yyyy_mm_dd_hh_mm_ss>.txt	Contains call information from the CallServer application (CIS, IVR, and AMI interfaces), and RelayServer (Relay Call server). CallJournal files are written by NetServer.
CallServerJournal_<yyyy_mm_dd_hh_mm_ss>.txt	Contains call information from CallServer (CIS, IVR, and AMI interfaces), and RelayServer. CallServerJournal files are written by CallServer. CallServerJournal files can contain the same information as CallJournal files, but they are used by different processes, and therefore need to be separated.

Journal Files and Folders	
RelayServerJournal_<yyyy_mm_dd_hh_mm_ss>.txt	Contains calls from the Call Entry client. Relay Call server (RelayServer) journal files are written by CallServer. In Simulator mode, RelayServerJournal files contain data that comes from the simulator call server (CIS, IVR, and AMI interfaces), or from another call server (for example, RelayServer) to another call consumer (CallServer, RelayServer, or NetServer). RelayServerJournal files can contain the same information as CallJournal files, but they are used by different processes and therefore need to be separated.
\dynamics\server\JournalWS	This directory contains old journal files that are no longer used by the system.
DeenergizedLineJournal_<yyyy_mm_dd_hh_mm_ss>.txt	Contains data about de-energized lines, predicted out lines, and cuts by substation. **Note** Deenergized lines without affected customers are not written to the de-energized line journal. DeenergizedLineJournal files are written by NetServer.
\dynamics\server\InterfaceServer\GisNotificationJournal	Contains GIS notification data (switch status changes to the Pending/Permanent state). GisNotificationJournal files are written by DMS Interface server.
\dynamics\server\RTServer\AreaNotesJournal	Deprecated.

4.6.3. Crew File

Crew information is stored as a .CREW file, as defined in the NetServer configuration file. This is considered to be dynamic data because it is possible to add, update, or delete objects incrementally while the system is running. Any online update of the crew model is stored as a journal file, which is stored as part of the daily snapshot. The main entities are crews and employees. This file format supports importing and exporting.

The crew file is like a snapshot of the crew and employee information. Incremental changes to crew and employee information are saved in the incident journal. NetServer writes the crew file. NetServer and Net Data server read the file during startup. From a recovery point of view, if a snapshot file and all the other journal files are available, it is possible to delete the crew file without loss of functionality (except for a slower initialization).

Location: \models\crew

Replication: Crew file can be replicated by the File Updater.

Crew Files	
<Company_Name>_CrewModel.Crew	A crew file contains information about crews and employees registered in the system. Crew data are used in the OMS. A crew file is created by an analyst using the Crew Modeling Tool.

4.6.4. Snapshot Files

A snapshot file has an extension of **.**snapshot. The file name includes the date and time when the file was created. Regular snapshots are written and read by NetServer, and daily snapshots are written and read by Net Data server.

A snapshot file contains all OMS information from the system for a given time. It stores information that is normally initialized from the call, OMS, and archive journals, and from the crew file. The snapshot includes all the previous history, so older journal files can be retired. Snapshot files are compressed and can be viewed with the Snapshot Viewer.

Snapshot files are created daily and periodically (for example, once per thirty minutes) as defined in the NetServer configuration file, or upon request by an administrator.

On startup, NetServer and Net Data server look for the latest snapshot file. If a snapshot file is found, NetServer and Net Data server open and read the snapshot file to initialize its in-memory data, and then NetServer and Net Data server read all journal entries since the snapshot file was made to complete the initialization.

A stand-by NetServer uses snapshot files on startup of the system.

Location: \dynamics\server\OMSServer\Snapshot

Replication: Only daily snapshots are replicated by the Net Data Replicator.

Snapshot Files	
Snapshot[<*yyyy*_*mm*_*dd*>[<hh*mm*_*ss*>].snapshot CrewSnapshot[<*yyyy*_*mm*_*dd*>[<hh*mm*_*ss*>].snapshot	Snapshot files, which contain OMS information from the system for a given time. These snapshot files are created periodically by NetServer.
DailySnapshot_<yyyy_mm_dd>.snapshot	Snapshot files, which contain the data for the previous day consolidated in a single file. Daily snapshots are written by Net Data server once a day at midnight.

4.6.5. Fast Dynamics Files

Fast dynamics files are used by the server to pass load flow information between servers and clients. Fast dynamics files are stored as a single file per substation to match the static model files.

Fast dynamics files are considered to be "fast" because they can be applied without looking up IDs. This means that fast dynamics are not compatible for any station's model file that is different from

the model file they were created from.

The following fast dynamics file is supported:

- Measurement, SCADA quality, and status (**.**nmea extension)

The typical uses of fast dynamics files are to pass data from a main NetSrv to a DnafSrv to pass data from the real-time client to a study server application running on the client machine.

The measurement, SCADA quality, and status information in fast dynamics files can be re-created from other sources. From a recovery point of view, it is possible to delete these files without loss of functionality.

Location: \dynamics\server

Replication: Fast dynamics files are replicated by NetSrv.

Fast Dynamic Files	
<Substation_Name>.nmea <Substation_Name>.mea (old file format)	Files that store measurement, SCADA quality, and status data. The server uses these files during startup after a restart or unexpected shutdown to restore the data. The *.mea files are used only by the old study server (version 3.1 and earlier) in study savecases. New study servers create and use *.mea files only if they are configured to support old format (FD_SUPPORT_OLD_FORMAT parameter is set to 1). These files are written by NetSrv.

4.6.6. Switch Order Files

The switch order server stores the switch orders as XML files on disk. The document file name can be configured using the configuration editor. The user can select control center, user name, switch order status, etc. as part of the file name. Completed switching steps are archived.

The switch order files are dynamic data. The files are replicated from the primary server to the standby server(s).

Location: \dynamics\server\SwitchOrderServer

Replication: Switch Order Server files are replicated by Net Data Replicator.

Switch Order Files	
<Switch_Order_Name>.XML	Approved, completed, and saved switch orders and daily logs. These files are written by Switch Order server.

4.6.7. Safety Document Files

The safety documents are dynamic data. The Safety Documents application stores the released

safety documents as XML files on disk. The Archive application periodically checks the folder and archives all of the documents in the folder.

Location: \dynamics\server\SafetyDocuments

Replication: The files are replicated from the primary server to the standby server(s) for archiving purposes only (Habitat replication services (MRS) are used for data replication to the standby).

Safety Document Files	
<Safety_Document_ID>.XML	Saved and released safety documents. The safety documents are created by the Safety application.

XML files generated by the SAFETY application in the Scada server are required to replicate to the ADMS server(s) so that the Archive application can pick them up for archiving. The current design assumes that the system has network storage such as Storage Area Network (SAN) or file replication capability such as Microsoft Distributed File System (DFS). The two above-mentioned technologies have been used in the EMS and MMS solutions and have proven to be effective. Some projects have also used the Habitat utility SAMPCOPY[1] and the third-party file transfer product SecureFX to meet similar file synchronization requirements.

4.6.8. Network Analysis Result Files

The DNAF server (DnafSrv) generates binary solution contents.

The result files (*.lst) are dynamic data. The files can be re-created by re-executing the DNAF functions. From a recovery point of view, it is possible to delete these files without loss of functionality.

The solution results communication is based on the DNAF_RESULTS_BY_TCP parameter in the NetServerDnafConfig.txt file:

- **DNAF_RESULTS_BY_TCP = 1:** This is the recommended configuration. In this case, DNAF server sends the solution data through the network to the real-time server. The real-time server applies the data to the system and creates a binary file with the most recent content. This binary file can be used for data restoration after a server restart. The real-time server also forwards the solution contents through the network to the standby or read-only servers if they are connected.
- **DNAF_RESULT_BY_TCP = 0:** The DNAF server writes the solution contents to binary files. The real-time server watches the containing folder and applies contents from newly created files to the network system. For this configuration, the real-time server does not send solution contents to connected standby and read-only servers.

> **Note**
>
> Communicating solution contents between DNAF and real-time servers is not necessary. For example, it does not occur for simulator and study server.

Location: \dynamics\server\DNAF

Replication: Network analysis result files are replicated by NetSrv.

Network Analysis Results Files	
<Substation_Name>_<DNAFappID>_DNAFSRV.LST	Files that contain results of DNAF applications executed for substations. The files are written by DnafSrv.

4.7. File Replication

Dynamic data files replication is performed by NetSrv and Net Data Replicator. For more information, refer to section Dynamic Data Files.

Static data files are copied using the File Updater tool, as described in section Static Data Files.

4.8. Data Flow and Failure Recovery Examples

This section presents a few concrete examples illustrating data flow between components, how the system handles failure events, and how the system recovers from those events.

The first example includes descriptions of each process step and recoveries from potential failures. The subsequent examples highlight those features that have not been covered in the first example.

There is a small sample of failure scenarios. The goal is to provide some insight into the interactions and resilient features of the ADMS applications.

4.8.1. Example: A Single Customer Call and Incident

In this example, a single customer call is received, and the operator assigns the incident to a crew. The crew travels to the site, clears the trouble, and restores the power.

The figure below shows the message and data flows of this example. The numbers shown in the diagram correspond to the step numbers in the descriptions that follow.

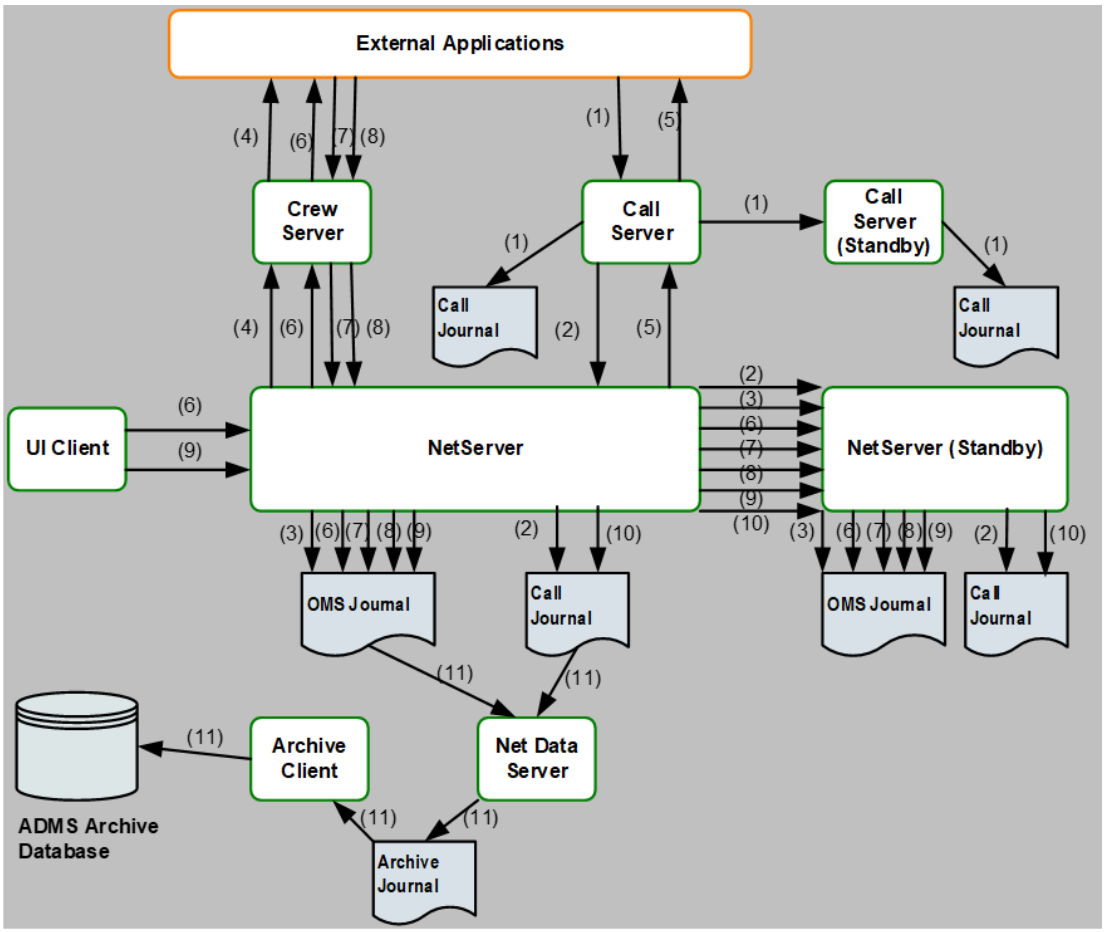


Figure 27. Message and Data Flows of Example 1

The following describes the process steps and recoveries of potential failures:

1. The call server queries the CSS for new calls since the last periodic query. The CSS returns the new outage call message to the call server. The call server writes the message to the call journal, and then updates the changes in memory.

   > **Note**
   >
   > The standby Net Data Replicator periodically queries the primary Net Data Replicator for the changes and updates its own call journal.

   Failure Scenarios:

   - The call server fails before it completes writing the journal. When the call server restarts, it initializes by reading its own call journal. Then it queries the CSS for any calls since the last time entry of the call journal. The CSS resends the new call message and the process continues.
   - The call server fails after it completes writing the journal. When the call server restarts, it initializes by reading its own call journal. Querying the CSS finds no new calls, and the process continues.

2. The primary NetServer periodically queries the call server for the changes. The call server sends the new outage call messages to NetServer. NetServer writes the messages to its call journal; then updates the changes in memory.

   > **Note**
   >
   > The standby Net Data Replicator also periodically queries the primary Net Data Replicator for the changes and updates its own call journal file.

   Failure Scenarios:

   - If the call server fails before NetServer receives the message, it delays the process until the call server restarts. Call server failure after NetServer receives the message does not disrupt the process.
   - NetServer failure causes failover. If failover occurs before NetServer completes writing the journal, then when the new primary NetServer takes on the active role, it queries the call server for any calls since the last time entry of its own call journal. The call server resends the new call message and the process continues.

3. NetServer processes the outage. Since the service point has only one customer, after the "simmer timer" (the time between when a call comes in and an incident is created) expires, the OMS engine creates a single customer incident, writes the add incident message to the incident journal, and then applies the journal message to the OMS database in memory.

   > **Note**
   >
   > The standby Net Data Replicator periodically queries the journal messages from the primary Net Data Replicator and updates its own incident journal file.

   Failure Scenarios:

   - NetServer failure causes failover. If failover occurs before the standby NetServer updates its incident journal, then when NetServer takes on the active role, it sees a new call message that has no incident. It adds the incident and continues the process.
   - If failover occurs after NetServer completes writing the journal, then when the new primary NetServer takes on the active role, it updates the OMS database in memory from the journal and continues the process.

4. NetServer sends the incident update message to the crew server, which relays the message to the channel adapter to inform the work management about the new incident.

   Failure Scenarios:

   - If the crew server fails and restarts (or loses the network connection), it exchanges sync messages with NetServer and the channel adapter to get all of the parties in sync with incident updates, incident statuses, crew assignments, and crew statuses. The work management receives the incident update message if it did not get it before the failure.

5. The call server periodically queries NetServer if there are any updates to case notes. The call

server keeps all of the case notes in memory. The channel adapter may query for case notes related to the new call. The call server responds with the latest note related to the call.

Failure Scenarios:

- If the call server fails and restarts, it requests all of the case notes from NetServer; however, it does not send the case notes to the channel adapter until the channel adapter queries the case notes again.

6. When an operator assigns a crew to work on an incident, NetServer writes a crew assignment message to the incident journal, and then applies the journal message to the OMS database in memory. Then NetServer sends the crew update message to the crew server, which relays the message to the channel adapter to inform the work management about the crew assignment associated with the incident.

   Failure Scenarios:

   - If the crew server fails and restarts (or loses the network connection), it resynchronizes with NetServer and the channel adapter to get all of the parties in sync with incident updates, incident statuses, crew assignments, and crew statuses. The work management receives the crew assignment message if it did not get it before the failure.
   - NetServer failure causes failover. If failover occurs before the standby NetServer updates its incident journal and the crew assignment message has not been sent to the channel adapter, then when NetServer takes on the active role, the operator sees that the incident has no crew assignment and needs to assign the crew again.
   - If failover occurs after the crew message has already been sent to the work management but before the standby NetServer updates its incident journal, then when NetServer takes on the active role and resynchronizes with the crew server and the channel adapter, it notices the crew assignment of the incident and updates the incident journal and the OMS database.

7. After a crew is successfully assigned, an incident status message is sent by the work management (via the channel adapter) to the crew server, which relays the message to NetServer. NetServer writes the message to the incident journal, and then applies the update to the OMS database in memory. As work progresses, the work management continues sending crew status messages (for example, "en route", "onsite") to the crew server, which in turn sends the messages to NetServer. The updates are recorded in the incident journal and then applied to the OMS database.

   Failure Scenarios:

   - When the crew server restarts or reconnects after a network disruption, it resynchronizes with NetServer and the channel adapter to get all of the parties in sync with incident updates, incident statuses, crew assignments, and crew statuses. At this process step, the work management has the latest information. The resynchronization allows ADMS to recover from either crew server or NetServer failure.

8. When the crew restores power, the work order is complete. The work management sends the incident complete message to the crew server, which relays the message to NetServer. The

message is written to the incident journal, the incident is flagged as completed, and the crew is set to standby.

Failure Scenarios:

- When the crew server restarts or reconnects after a network disruption, it resynchronizes with NetServer and the channel adapter to get all of the parties in sync with incident updates, incident statuses, crew assignments, and crew statuses. At this process step, the work management has the latest information. The resynchronization allows ADMS to recover from either crew server or NetServer failure.

9. An operator manually clears the incident. The change is recorded in the incident journal and then applied to the OMS database.

   Failure Scenarios:

   - NetServer failure causes failover. If failover occurs before the standby NetServer updates its incident journal, then when NetServer takes on the active role, the operator sees that the incident has not been cleared and needs to clear it again.

10. Clearing the incident triggers clearing the call. NetServer writes a message to clear the call into its call journal. The standby NetServer queries the update and writes the entry to its call journal.

    Failure Scenarios:

    - NetServer failure causes failover. If failover occurs before the standby NetServer updates its incident journal, then when NetServer takes on the active role, the call is re-applied and the operator needs to cancel the call or incident.

11. Net Data server periodically reads the incident journal to update its OMS database. It writes out the cleared incident and its chronology logs to the archive journal. The ADMS Archive application periodically reads the archive journal and archives the information about the cleared incident to the archive database.

    > **Note**
    >
    > Since Net Data server and the ADMS Archive application are continuously reading the journal files, any corruption of the journal files is quickly noticed.

    Failure Scenarios:

    - If Net Data server fails and restarts, it initializes the OMS database with the latest snapshot file, and then it applies the call journal and the incident journal from NetServer. With this approach, Net Data server recovers the OMS database to the latest state and continues the process.
    - If the ADMS Archive application fails and restarts, it queries the archive database for the timestamp of the last archive completion, and then it reads the archive journal to continue archiving from that time.

[1] In **e-terra***habitat* 5.8, SAMPCOPY has been improved to securely copy various types of files around the system.

5. Security

This chapter provides a brief introduction to the ADMS security aspects.

5.1. Overview

GE's security goals for ADMS include the following:

- To provide an ADMS that is secure upon delivery per industry best practices and regulations, such as NERC CIP
- To equip the system with core technology and management tools to allow the system to be maintained and operated in a secure manner
- To provide documentation and recommendations to help customers address their specific business needs while deploying GE's ADMS system securely within their enterprises
- To provide a scalable and extensible system that meets the new smart grid cyber security standards without a major rebuild

Attaining the goal of a secure computing environment requires an integrated, consistent approach to be effective. This chapter discusses the following essential elements of a comprehensive ADMS cyber security architecture:

- Key security principles
- Security policy
- Network security
- Host security
- Application security
- User security
- System Maintenance

5.2. Key Security Principles

The security objectives for ADMS are providing high system availability, information confidentiality, and system and data integrity during normal operation and unintentional misuse, as well as under malicious cyber attack.

It is critical that key security principles are applied throughout the system life cycle: defense in depth, minimize the attack surface, and separate duties.

Defense in Depth

The principle of defense in depth is that, where one control can be reasonable, more controls that approach risks in different fashions are better. Multiple layers of defenses can make severe

vulnerabilities extraordinarily difficult to exploit and thus unlikely to occur.

GE takes the approach of four layers of defense in ADMS: network, host, application, and user security.

Customers need to have a security policy and a management process to make sure that the security posture of ADMS is not degraded after system deployment.

Minimize the Attack Surface

Minimizing the attack surface includes actions such as removal of unnecessary services and applications, shutdown of unused network ports, removal of unused accounts, and other system hardening practices.

GE delivers hardened ADMS in FAT, and provides documentation and tools to help customers maintain secured systems.

Separate Duties

Separation of duties ensures that people only have access to the cyber assets required to perform their job functions. There are many roles in an ADMS; ADMS offers role-based authorization.

5.3. Security Policy

An essential prerequisite for keeping ADMS secure is a clear, explicit, and relevant security policy. This must in turn be based on a careful risk assessment of your enterprise.

The development and maintenance of a security policy for your organization is outside the scope of this document, but some fundamental building blocks include:

- Asset identification and classification
- Asset protection
- Asset management
- Acceptable use
- Vulnerability assessment and management
- Threat assessment and monitoring

GE suggests that customers review and update their security policies for ADMS before commissioning a system.

5.4. Network Security

This section describes the network security, including the security zones and the network design principles.

5.4.1. Security Zones

The ADMS software components are deployed to a network extended from the existing infrastructure that houses the Scada system.

The production (real-time) network is isolated from the network through the use of firewalls. The system is logically divided into the following zones:

- **Level 0 – Red Zone: External Domains**

  IT infrastructures that are not under direct or delegated control must be placed in the Red Zone. All external systems in the Red Zone should be considered insecure until they have been deemed trusted. These include the Internet, a business partner's network, and the Public Switch Telephone Network (PSTN).

- **Level 1 – Orange Zone: Production Corporate DMZ**

  IT infrastructures that contain and expose the corporate business services to a larger, untrusted network (usually the Internet) are placed in the Orange Zone, also called the "Corporate Demilitarized Zone (DMZ)". It contains systems that provide secure stepping stones between Level 0 and Level 2, as well as between Level 1 and Level 3 domains. These secured systems include DNS servers, exchange servers, corporate web servers, and corporate authentication servers.

  Applications that exchange data with the network should reside in this zone. Firewall rules allow specific traffic to cross between this zone and the network.

- **Level 2 – Yellow Zone: Internal Domains**

  IT infrastructures that are under direct or delegated control and that are not directly related to managing, monitoring, or controlling elements of the distribution system are placed in the Yellow Zone.

  IT infrastructures in the Yellow zone include corporate business networks, end-user systems, file and print servers, HR and finance servers and associated network infrastructures, and internal IT administrative systems related to the IT management administration of IT infrastructures that are directly related to running the market and associated network infrastructures.

- **Level 3 – Green Zone: Production Control System DMZ**

  IT infrastructures that contain and expose the control system services to a larger, untrusted network (usually a partner's network) are placed in the Green Zone. The Green Zone is also called the "Control System DMZ". It contains systems that provide secure stepping stones between Level 2 and Level 4, as well as Level 1 and Level 3 domains. These secured systems include Intercontrol Center Communications Protocol (ICCP) servers, historian servers, control system web servers, and control system authentication servers.

  The interface servers (call server and crew server), replicated read-only server (Net Data server), archive database servers, Citrix servers, and Comm servers are usually located in this zone. The QA servers and support consoles are located in this zone.

Training facilities, including Distribution Operator Training Simulator (DOTS) servers, are typically located in the Green Zone. However, depending on the organization's business requirements, they may be placed in the Blue Zone.

Specific firewall rules control the access to other zones as well as to the internet.

- **Level 4 – Blue Zone: Production Control System Domains**

  IT infrastructures that are under direct or delegated control and that are directly related to managing, monitoring, or controlling elements of the distribution system must be placed in the Blue Zone.

  This is the most secure zone. It has no direct communication with the Internet. Infrastructures in the Blue Zone include the ADMS servers, Scada servers, and control center consoles.

  This zone may also include Grid Monitoring and Control (GMC) systems (such as Scada/EMS servers and clients); frequency and voltage monitoring systems; substation infrastructures, including local substation control systems, metering systems, and building services systems; secured IT administrative systems related to the IT management/administration of IT infrastructures in Level 3 and Level 4; and associated network infrastructures.

The figure below is a high-level view of the network deployment.

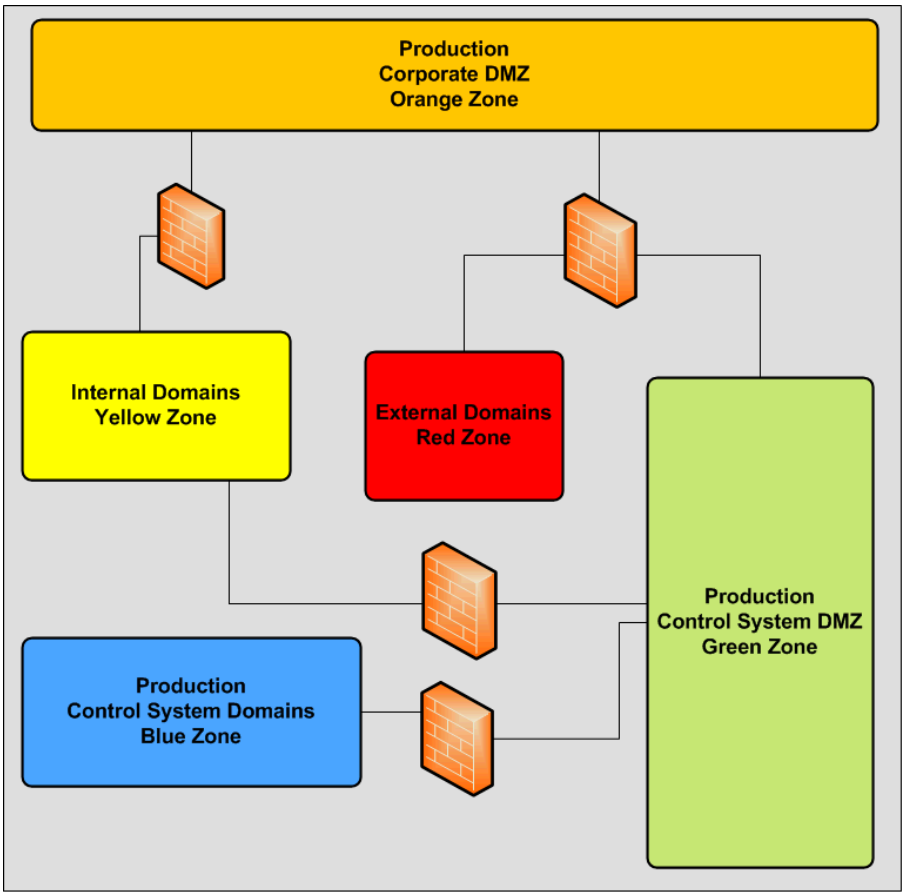


Figure 28. High-Level View of Production System

The figure below illustrates a high-level view of the DMS+OMS ADMS network architecture, where colors represent security zones.

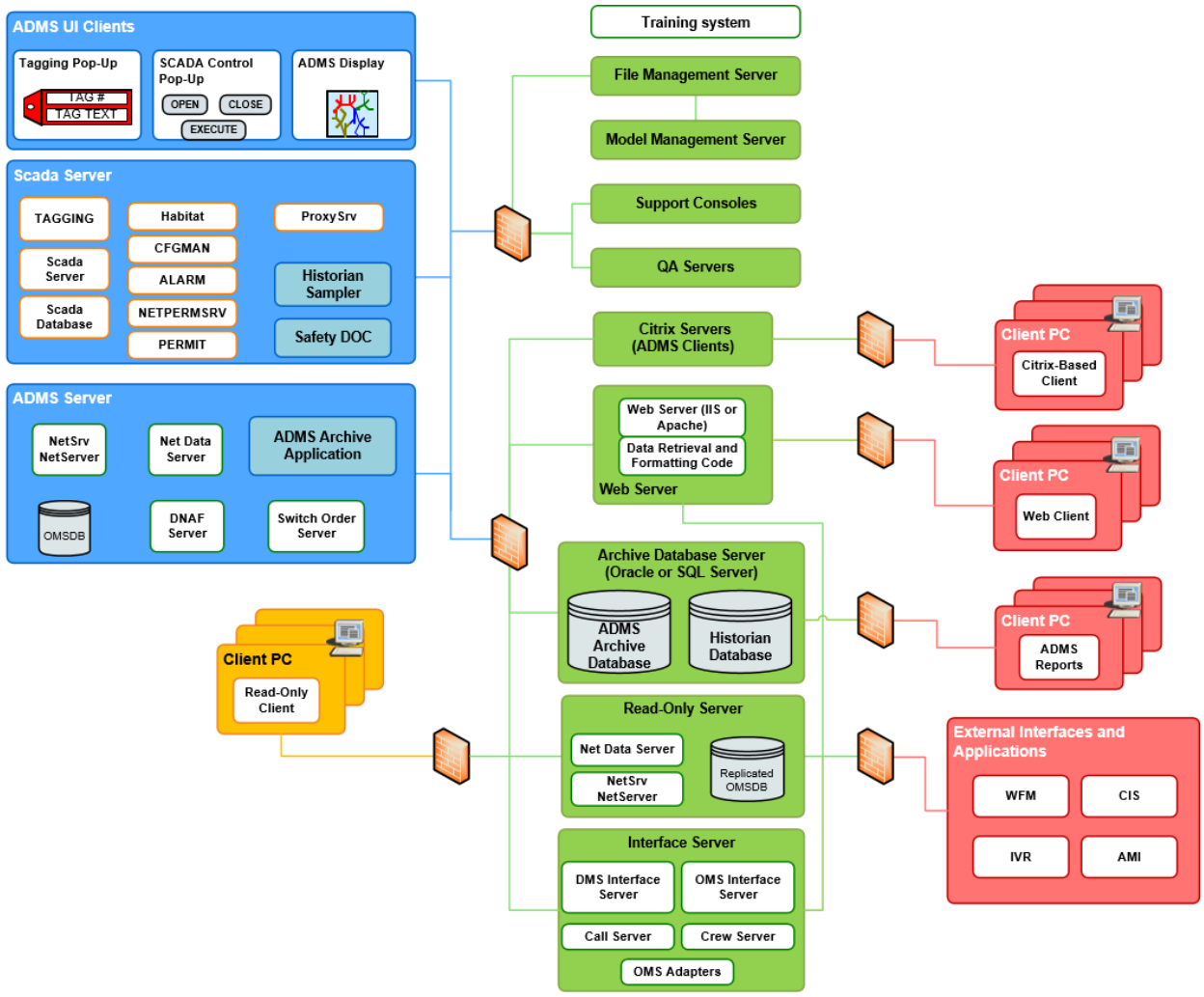


Figure 29. High-Level View of ADMS Network Architecture

Network segmentation provides clearly defined electronic perimeters. Perimeter protection is critical to mitigate cyber-attacks. To minimize the attack surface, unused ports are closed at the network-based firewalls. Firewall rules are configured to allow the minimum traffic between different network zones. A list of required ports and protocols is part of the system delivery. Network Address Translation (NAT) is used to reduce the exposure of the internal network IP addresses to the lower-level security zone.

Network-based Intrusion Detection Systems (IDS) are deployed at the perimeter coupled with the firewalls. An IDS detects network scans and exploit attempts. The rules are fine-tuned based on the protocols of the cross-network zone traffic pattern.

For additional information about network security, refer to the *Network Security Guide*.

5.4.2. Network Design Principles

The key security design principles are defense in depth, minimizing the attack surface, and separation of duties.

The design principles for network partitions include the following:

- A system must not be a member of a domain if the member has a higher protection requirement for confidentiality or mission assurance than is offered by the domain.
- Applications/systems are not permitted to make direct connections where the destination end point is in a Level 2 or Level 4 domain and the source end point is more than one level below the destination end point. For example, Level 1 can connect to Level 2, but Level 0 cannot connect to Level 2, and Level 1 can connect to Level 3, but not to Level 4.

	Level 0 (Red Zone)	Level 1 (Orange Zone)	Level 2 (Yellow Zone)	Level 3 (Green Zone)	Level 4 (Blue Zone)
Level 0 (Red Zone)	✓	✓		✓	
Level 1 (Orange Zone)	✓	✓	✓	✓	
Level 2 (Yellow Zone)		✓	✓	✓	
Level 3 (Green Zone)	✓	✓	✓	✓	✓
Level 4 (Blue Zone)				✓	✓

✓ Connection is allowed.

- Controls should be designed to allow each domain to protect itself where possible (i.e., reliance on transitive trust[1] should be minimized).
- Based on the principle of least privilege, a domain perimeter should allow Policy Enforcement Points (PEP) rules/policies to be established such that they support the most specific capability to fulfill the required functionality.

  Policy Enforcement Points (PEP) are used to mediate inter-domain traffic. Examples of PEPs are routers, switches, firewalls, Intrusion Detection Systems (IDS), Intrusion Prevention Systems (IPS), and Virtual Private Network (VPN) systems.

5.5. Host Security

Host security provides another layer of defense for ADMS. Host security includes the following:

- Install minimum server roles during operating systems installation.
- Disable unused services if they are installed as part of the server roles required by ADMS.
-

Install and run host-based anti-virus software to detect and quarantine viruses and prevent malware.

- Harden the system by adjusting the security settings using the principle of least privileges.
- Keep up with the latest operating system security patches and anti-virus software signatures.

5.6. Application Security

GE's Software Security Development Lifecycle (SSDL) mandates that cyber security be an integral component in every development stage. Security requirements are specified at the same time functional requirements are specified.

GE actively participates in and monitors the development of security standards in the industry. GE stays updated on the cutting edge of cyber security technologies in information technology, striving to use the latest and best cyber security technology in ADMS.

A security flaw in design is much harder to correct after a product is released. Threat modeling helps to identify any security shortages in design, and to find the mitigations.

To minimize security vulnerabilities in the source code, GE uses a Static Code Analyzer (SCA). The static code analyzer examines every line of code and every program path to identify any potentially exploitable vulnerabilities in the source code. The SCA is used to scan ADMS's source code on a half-week basis. GE is committed to fixing any high and critical issues identified by the SCA.

To adhere to the principle of least privileges at the application layer, each application runs with the minimum privileges required by the functionality. GE makes the best effort to eliminate the need to run an application under system administrative privileges. The system security guides contain the required privileges and system hardening instructions.

5.7. User Security

A user is typically one of the weakest links in a system. Social engineering and password dictionary attacks manipulate the natural human tendency to trust and how humans deal with complexity. Phishing and cross-site scripting rely on social engineering to carry out attacks. Multiple measures must be in place to mitigate these cyber attacks and traditional frauds. Security training, password policy, and strong authentication and authorization should be interwoven into another layer of defense.

A company can have users in control centers, corporate offices, remote sites, call centers, and even parking lots during storm restoration efforts. This variation of the user base presents many challenges.

ADMS offers user authentication at multiple levels, along with fine-grained role-based authorization.

5.7.1. User Roles

This section describes typical user roles in ADMS, which represent generic employee types of a distribution utility.

Using the PERMIT application and the Permission Management Plug-In, the users, user types, and permissions can be added, modified, and removed to match the user's role and responsibility and to meet the needs of a utility with specific employee roles and responsibilities.

System Administrators

System administrators are the primary system support staff. They have full access to Scada, DMS, and OMS functionality, subject to areas of responsibility. Users are granted Windows administrator privileges.

The workstations provided to this user role are located in the production (Real Time Network) Blue zone. GE ADMS clients are installed on these workstations. The workstations are configured specifically for ADMS functionality.

System Operators

System operators are the primary operators, located inside the distribution operations centers. They have full access to Scada, DMS, and OMS functionality, subject to area of responsibility. System operators are not granted Windows administrator privileges.

The workstations provided to this user role are located in the Blue Zone network. These workstations have the GE ADMS client installed and are configured specifically for ADMS functionality.

System Dispatchers

System dispatchers have limited access to Scada and DMS. Specifically, they are allowed to change the state of a switch in DMS, but are not allowed to perform tagging operations or issue SCADA controls. They are located inside the distribution operations centers.

The workstations provided to this user role are located in the production Blue zone. These workstations have the GE ADMS client installed and are configured specifically for ADMS functionality. System dispatchers are not granted Windows administrator privileges on the workstations.

Limited access is provided at the ADMS authorization level.

Remote System Operators

Remote system operators have all of the same capabilities of the system operators. However, they are not located in the distribution operation centers, but elsewhere, such as a division office, crew headquarters, or a staging area during storm restoration.

The workstations provided to this user role are located in other network zones outside of the production Blue, Green, and Yellow zones. These workstations (or laptops) do not have the GE ADMS client installed, but they do have the Citrix client installed. The ADMS client is installed on the

Citrix servers in the production Green zone.

Remote system operators access the ADMS client through their Citrix client. The Citrix technology provides another layer of user access control. Remote system operators are not granted Windows administrator privileges on the Citrix client workstations.

Remote System Dispatchers

Remote system dispatchers have limited access to Scada and DMS. Specifically, they are allowed to change the state of a switch in DMS, but they are not allowed to perform tagging operations or issue SCADA controls. Other than that, the remote system dispatcher role is the same as the remote system operator role.

The difference between remote system operators and remote system dispatchers is handled by the ADMS permission system.

Remote Limited Dispatchers

Remote limited dispatchers have a limited subset of the capabilities of the remote system dispatchers. They are not located in the distribution operation centers, but elsewhere, such as a division office, crew headquarters, or a staging area during storm restoration.

The workstations provided to this user role are located in other network zones outside of the production Blue, Green, and Yellow zones. The workstations (or laptops) do not have the GE ADMS client installed, but they do have the Citrix client installed. The ADMS client is installed on the Citrix servers in the production Green zone.

Remote limited dispatchers access the ADMS client through their Citrix client. The Citrix technology provides another layer of user access control. Remote limited dispatchers are not granted Windows administrator privileges on the Citrix client workstations.

The limited capability is controlled by the ADMS authorization.

Read-Only Users

The read-only system configuration replicates a copy of the production DMS/OMS system data so that users who require read-only access to the data can view it without interfacing directly with the production system. Typically the DMS/OMS read-only system is placed in the Green Zone "DMZ" located outside the production system's network firewall. Read-only users have the ability to view real-time SCADA, DMS, and OMS data via a read-only client located in the corporate network (Orange zone).

In addition, users anywhere in the network can have read-only access via a web service interface based on the Distribution Interface server and the Net Data server.

Call Entry Users

Call entry users only have access to the call entry client application. They have no access to the real-time ADMS servers.

The workstations for call entry users are located in the call centers in other network zones outside

of the production Blue, Green, and Yellow zones. Call entry users are not granted Windows administrator privileges.

5.7.2. Authentication

Authentication is the process of determining whether someone or something is, in fact, who or what it is declared to be.

There are three layers of user authentication: the operating system level, the Citrix level (for Citrix-based clients), and the ADMS level.

- Users who are located in the production Blue, Green, and Orange zones (see section Security Zones) and have the ADMS client installed log in to the workstation at the operating system level and log in to ADMS.
- Users who use the Citrix client to access the real-time ADMS servers log in to the operating system, the Citrix server, and to ADMS.

> **Note**
>
> Web-based users log in to the operating system and use web-based authentication.

ADMS authentication is performed through Habitat's centralized permission management application called PERMIT. The PERMIT application provides both authentication capability (who is attempting to log on) and authorization capability (what an identified user is allowed to do).

Browser supports a variety of authentication options to control access to the system. Any time an operator console opens a new connection to a given server, the server authenticates the connection. Once a connection from a client to a server is established, the operator can open any number of additional client windows without having to repeat the login sequence.

The login procedure checks the PERMIT user model to authenticate the user. Users are notified if invalid usernames or passwords are entered. Messages and alarms are issued to mark the authentication event and record the results.

The PERMIT authentication can also be supplemented with the Kerberos network authentication protocol, which provides strong authentication for client/server applications by using secret-key cryptography.

5.7.3. Authorization

Authorization is the process of giving someone permission to do or have something. Role-based authorization grants access rights to service operations or resources based on the functional role(s) of a user.

ADMS integrates with the PERMIT server to provide role-based authorization. User roles (a.k.a. "types") are defined in the PERMIT database. For more details, refer to sections Scada Integration

and Permission Checking.

5.8. ADMS System Maintenance

Security management should include change control, testing, and auditing. Change control includes the application updates.

GE notifies customers via e-mail when a security patch is available. The details about the security patches can be found on GE's online customer support web portal (https://gegrid.force.com/sws), in the Security Bulletins area of the Security Corner.

It is important to conduct a post-installation security test before deploying a system in production. The following sections provide the initial set of tasks required for ongoing system security maintenance. Once the system is deployed, proper maintenance is critical for keeping the security posture of the system and for continued compliance.

6. User Interface

6.1. Overview

The figure below provides an overview of the high-level component relationships and data flow for the ADMS user interface.

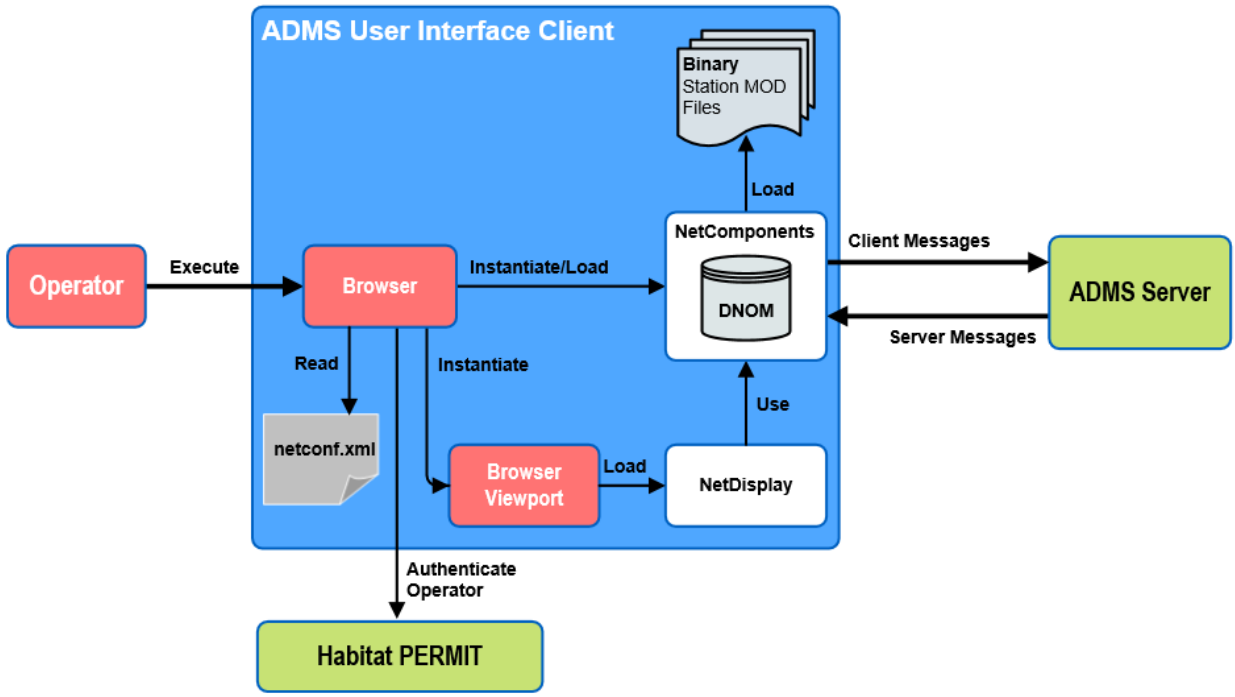


Figure 30. ADMS UI Client Overview

A fundamental architectural principal of the ADMS user interface lies in the choice to design it as a plug-in to Browser. The key Browser features are:

- Provides an integrated user interface environment that enables navigation to and interaction with other GE displays and applications (for example, Scada/Habitat applications).
- Provides base-level functions to manage:
  - Screen real estate
  - Handle events from input devices
  - Toolbars and menus
  - Keyboard mappings and definitions
  - Configuration and user preferences
- Provides integration with EMS functions dealing with redundancy, security, alarming, logging, and so on.

6.2. Plug-In Components

The application code for the ADMS user interface includes multiple components in the form of Microsoft Dynamic Link Libraries (DLLs). These DLLs are loaded in at run time and perform the following functions:

- **OMS Display Plug-In:** This plug-in is directly loaded by Browser and provides the core functionality for the OMS aspects of the system. The plug-in is available for customization.
- **SWOEmployeeInfoPlugin.dll:** This plug-in allows the customer-specific crew and employee information to be used in the Switching Order application.
- **NetAddIns.dll:** The system uses this plug-in for loading some of the pop-up displays.
- **app_loadshed.dll:** This plug-in supports the Surgical Load Shed (SLS) DNAF function that prioritizes and sheds circuits to obtain a desired target reduction in system load.
- **app_lvmmonitor.dll:** This plug-in supports the LVM Monitor DNAF function that provides a summary for monitoring system-wide LVM control events.
- **app_fsd.dll:** This plug-in supports the Faulted Section Determination (FSD) application, which identifies and isolates a faulted section using a switching-based approach.

6.3. Inputs

The ADMS components rely on inputs provided by the following sources:

- Configuration files: System, behavioral, and look-and-feel.
- Default files: Provided at installation time, but potentially modifiable by customers.
- Run-time files: Contain dynamic data, propagated to the client from the server using the File Updater application.
- Run-time data: Data that is pulled from the server via network connections.

6.3.1. Configuration Files

These files are used to configure system parameters, application behaviors, and general look-and-feel aspects of the user interface. The configuration files are XML-based, although modifications are typically made using the following configuration editors (the editors are described in more detail in section Configuration Tools):

- Net Dynamics Configuration Editor: Produces the "master" ADMS configuration file (`NetConf.xml`) and provides values for system settings, application behavior, and locations for other configuration information.
- Viewer Configuration Editor: Used to configure the appearance and effects of geographic network displays as part of the Distribution Management System. Produces the `ViewerConfig.xml` file.

- OMS client Configuration Editor: Used to configure the appearance and effects of geographic network displays as part of the Outage Management System (OMS). Produces the `OmsClientConfig.xml` file.
- OMS server Configuration Editor: Used to configure the system behavior for certain conditions of OMS events, such as incidents, case notes, and customer calls. Produces the `OmsServerConfig.xml` file.
- Switching Order Configuration Editor: Used to define the switch order document configuration parameters, which include the look and behavior of the switch order displays, the document file name conventions, the server host name and port number, and so on.

The files are read in during initialization of the various plug-in components.

6.3.2. Default Files

The ADMS client installation provides a set of default files that are read in at run time. Examples of these files are bitmaps, toolbar files, and tabular files. Customers can modify some of these files as needed, and modified files can be distributed to client machines using the File Updater application.

6.3.3. Run-Time Files

Static model files are generated external to the ADMS client machines. These files are distributed at run time as needed by using the File Updater application, as shown in the figure below.

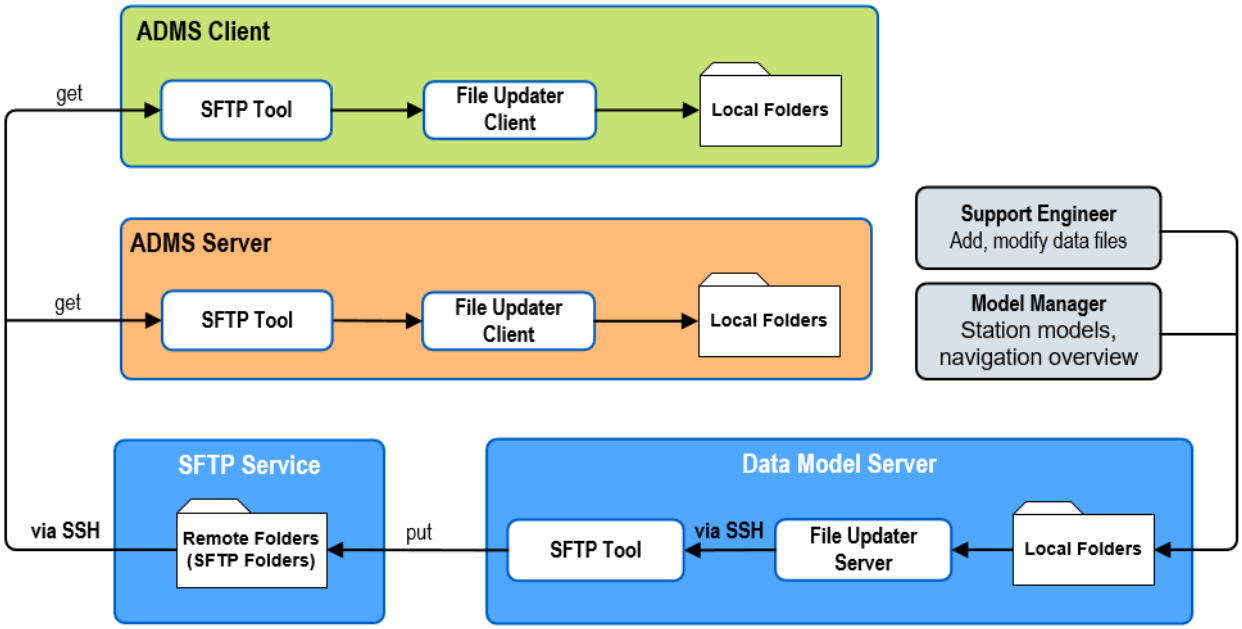


Figure 31. File Updater Application

The key files in this category are:

- Station Model: These contain the modeling information for portions of a geographic area that correspond to a substation and its related feeders.
- Customer: These files contain customer data.
- Crew: Information about crew locations and activities.
- Pictures: These are tabular displays.

6.3.4. Run-Time Data

The ADMS UI client maintains the network connections to real-time servers. The unified UI client is the key component that integrates the underlying functionalities.

The figure below shows the interaction and information flows between the UI client and the servers.

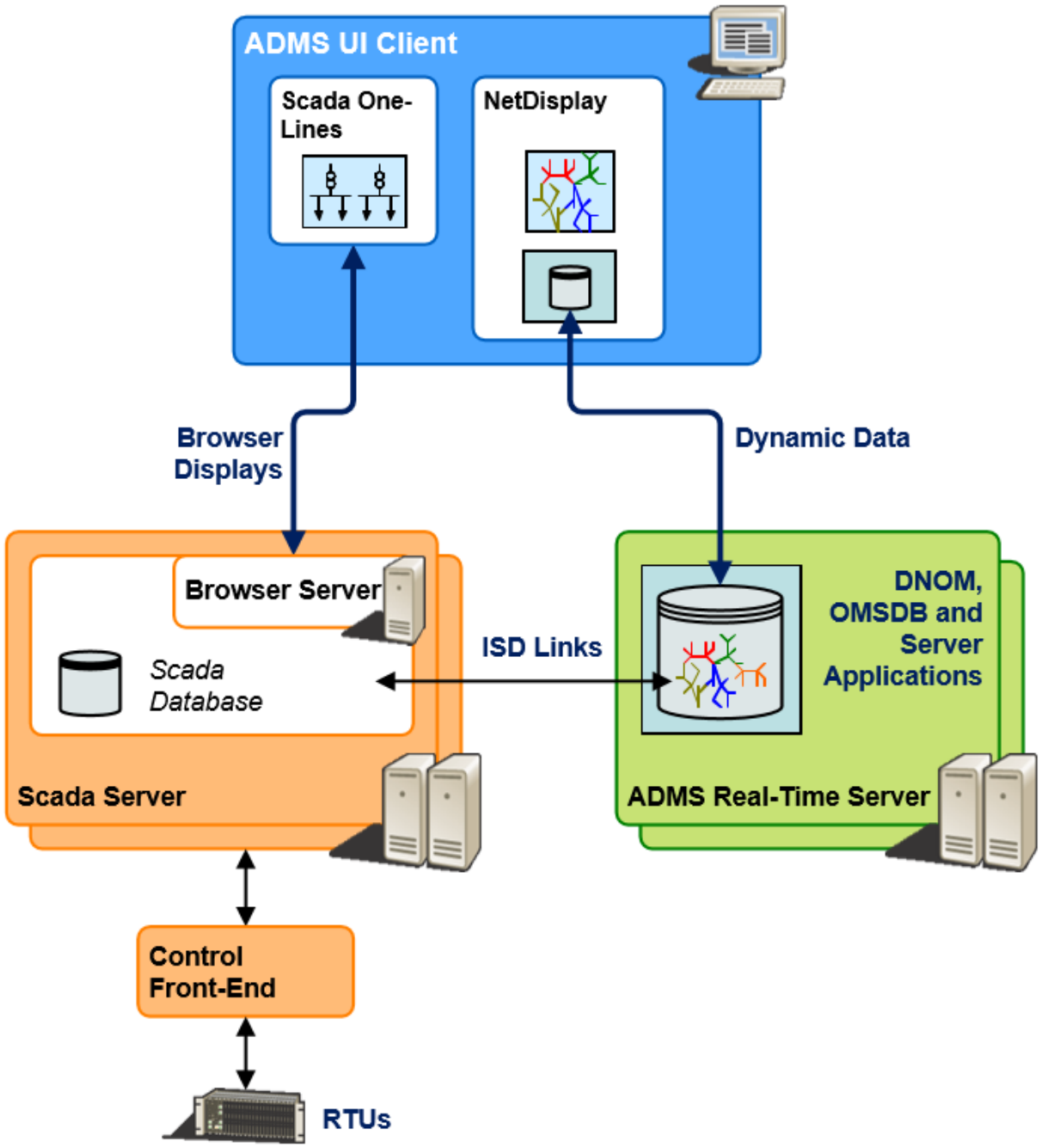


Figure 32. Information Flows between UI Client and Servers

6.4. Citrix Environment

Citrix provides virtual desktop and application virtualization technology that can simplify application deployments as well as secure remote access. For many customers, it is an appealing solution for centrally managing the UI client software and user access. This section gives a brief description of how the ADMS client supports operating in this deployment environment.

The security gateway, load balancing, redundancy, and client license management are not discussed here, as they are project-specific and depend on customers' IT and business process requirements. The *ADMS Installation Guide* documents the deployment procedures to meet these requirements.

The design discussed here assumes that the Citrix environment is Citrix XenApp 6 running on Microsoft Windows 2008 R2. The Microsoft Distributed File System can be used to provide shared folders for user data.

Through the Citrix servers, the ADMS client software, along with a Launch ADMS client application, is available to the remote operators. Users log on, and applications exchanges are carried over HyperText Transfer Protocol Secure (HTTPS). Additional firewall rules are implemented to allow the Citrix servers to communicate with the ADMS servers and Scada servers via the required ports.

The Launch ADMS client application provides the operator with a simple interface to start the ADMS UI client. It has a configuration file that guides the application performing the initialization before starting the ADMS UI client. The configuration includes the directory of the static files (shared by all users), the base directory of the dynamics files, whether the user group allows performing studies, and the range of ports for study servers.

During initialization, the Launch ADMS client application picks up the local user's ID and an open port for the study server (if allowed). It uses the information to create an ADMS client configuration file (`NetConf.xml`), which contains the server connection information, locations of the shared folders, user-specific folders, and the study server port. The shared folders and the user-specific folders should be located in a different drive than the system drive. The shared folders store static files that include the DNOM model files, the navigation overview file, the customer file, the geographic background files, and the default configuration files.

The ADMS client application expects that the folders and files already exist. Folder paths for the user-specific files are derived from the user's ID; the application creates the folders when necessary. The dynamics files are created and maintained by the ADMS UI client; the files include real-time dynamics, annotation dynamics, fast dynamics, and the customer mapping cache.

When the ADMS UI client starts, it connects to the servers and performs the permission checking based on the user's credentials. If needed, the ADMS UI client also starts a study server and creates the study model and study result folders.

When the user logs off, the study server and the ADMS UI client shut down, and the study server port is freed up.

7. Managing OMS Journals and Snapshots

This chapter describes how ADMS uses snapshots and incident and call journal files for registration, preservation, and replication of OMS data.

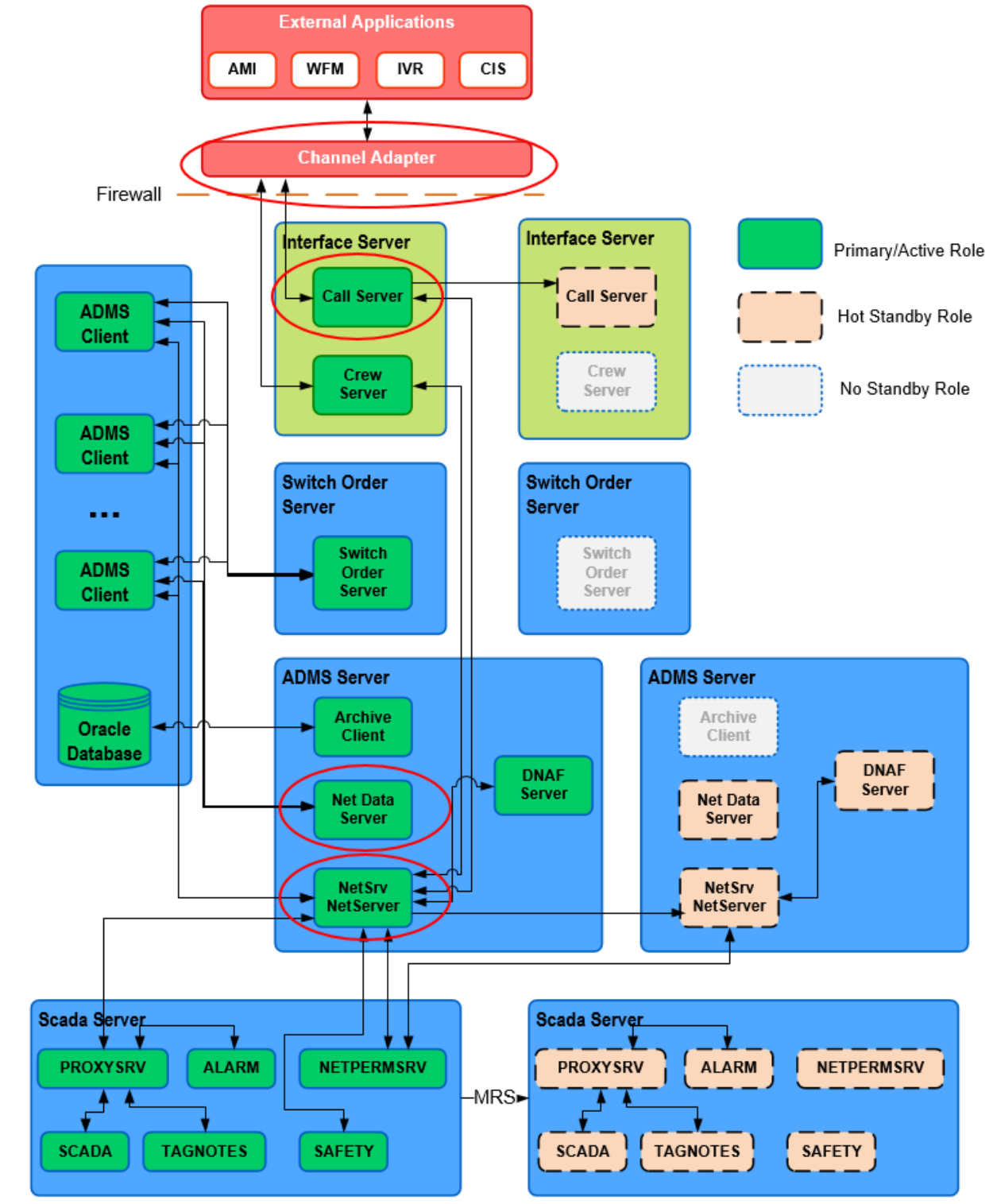


Figure 33. Integrated Distribution Management System

7.1. Journal Files

NetServer and Call server record call and incident data in journal files. For more details about the journal files, refer to section Journal Files.

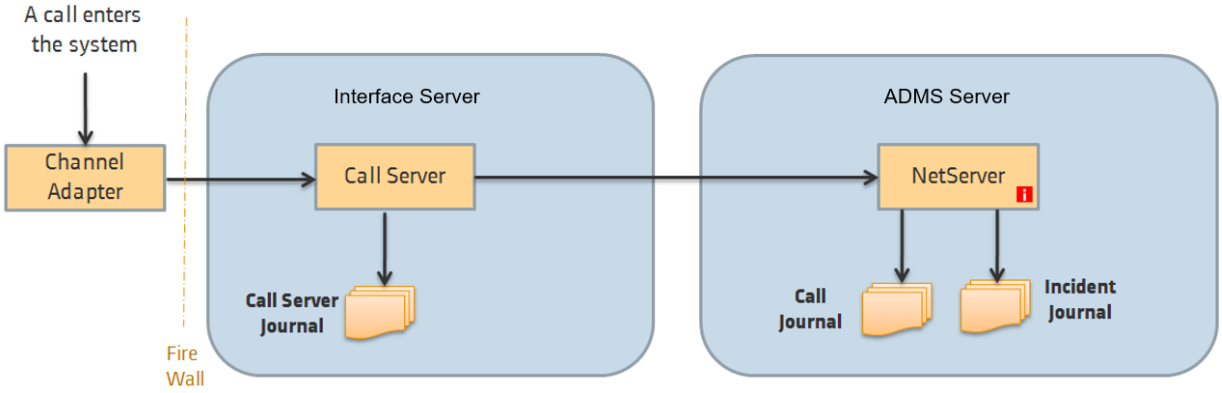


Figure 34. Registering OMS Data

If a system restarts, each server loads persisted data from journal files to restore the system.

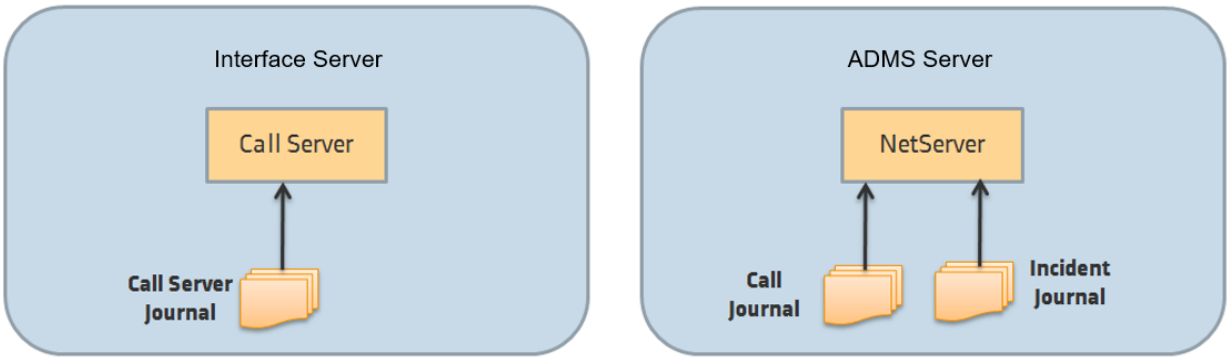


Figure 35. Restoring OMS Data from Journals

In order to prevent system slowing down due to a large number of journal files, the NetServer and other journal writers "retire" files once a day. The retirement is done at 1:00 AM as it is deemed to be the least busy time of the day. The number of days to keep journal files is configurable.

> **Note**
>
> Retirement means moving journal files to the specified folder. Journal files in the system are never deleted.

7.2. Snapshot Files

At a configure time interval NetServer creates snapshot files that include all customer calls and currently active incidents for a given time. For more details about the snapshot files, refer to section Snapshot Files.

Snapshots are used to speed up the startup of the system after restart. When the system restarts, NetServer loads only the latest available snapshot and the journal files that were created after the snapshot. Snapshots remove the need to load a large amount of journal files, many of which may already be obsolete.

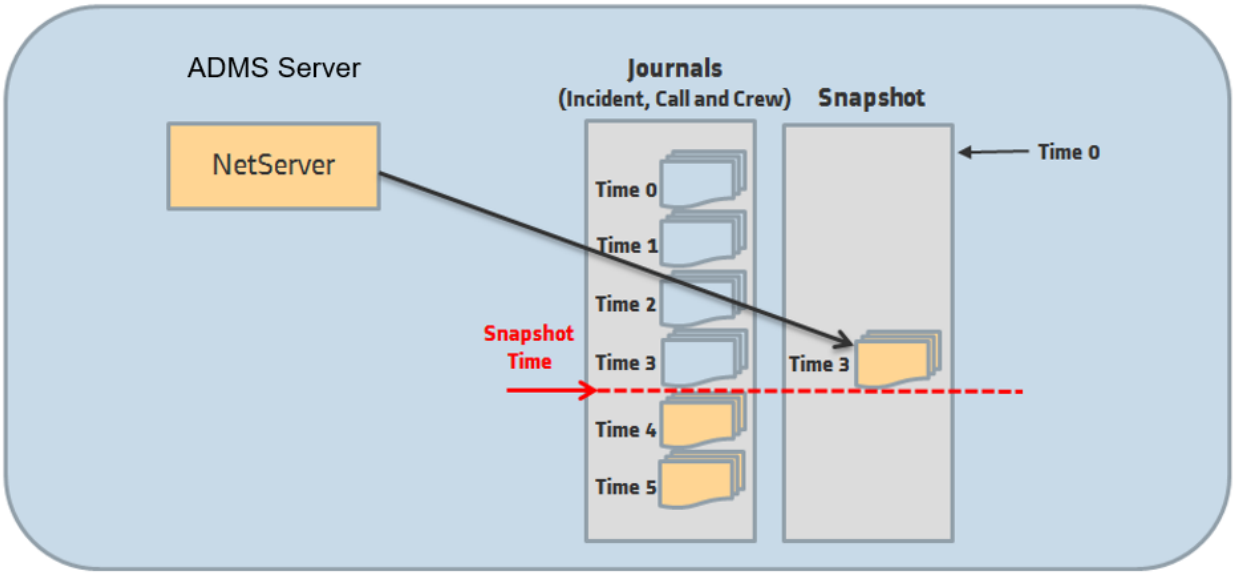


Figure 36. Creating Snapshot Files

7.3. Daily Snapshots

Net Data server is responsible for serving OMS data for the OMS history correction.

Net Data server is always a reader of journal files. It queries for the journals and keeps a "Current Query" time for each of the journals. When the recent journal entries are received, the "Current Query" time is updated to the time of the most recent message. As more activity enters the system and the NetServer adds to the journal files, Net Data server repeats its periodic queries.

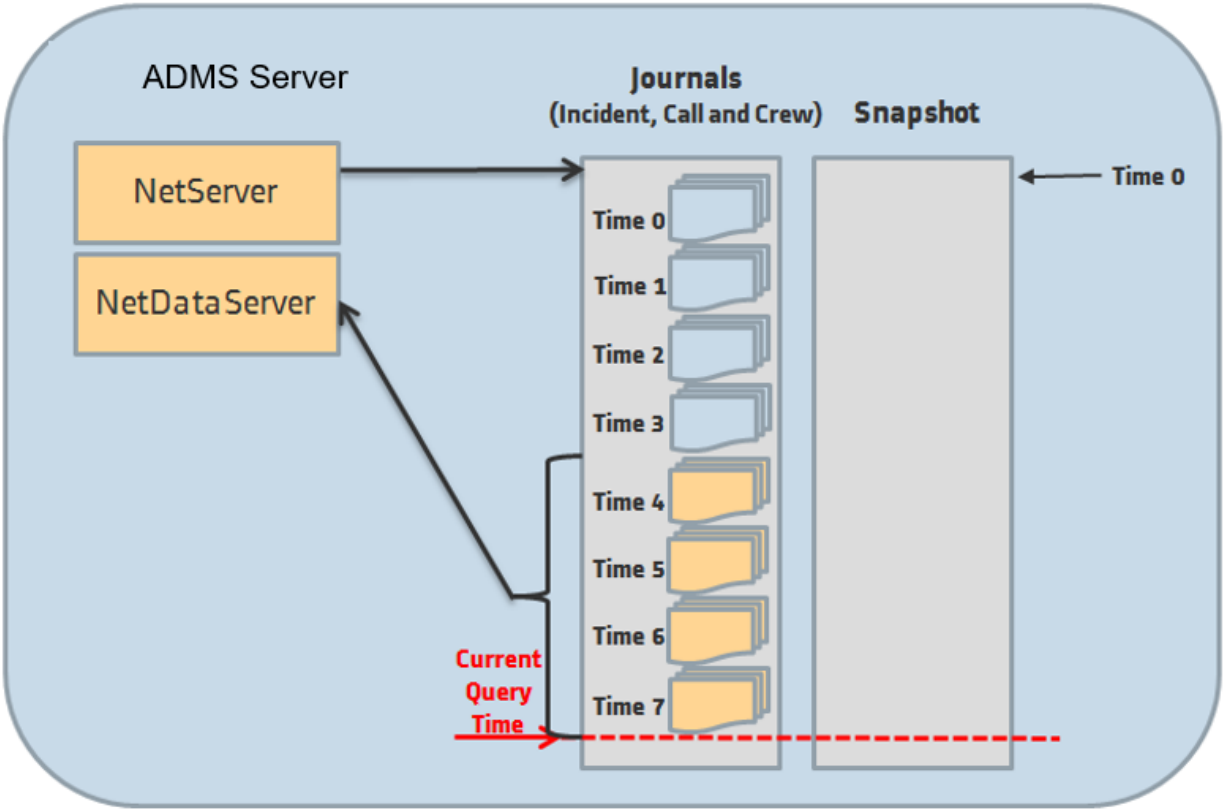


Figure 37. Net Data Server Queries

The main task of Net Data server is to keep history data, namely cleared incidents. Since NetServer keeps cleared incidents only for 5 minutes, and a given customer might want to see cleared incidents for the last 45 day, the enabled Net Data server creates a daily snapshot for the day that just ended shortly after midnight each day. The snapshot contains all incidents that were cleared in the previous day, and the active calls and incidents at the end of the day.

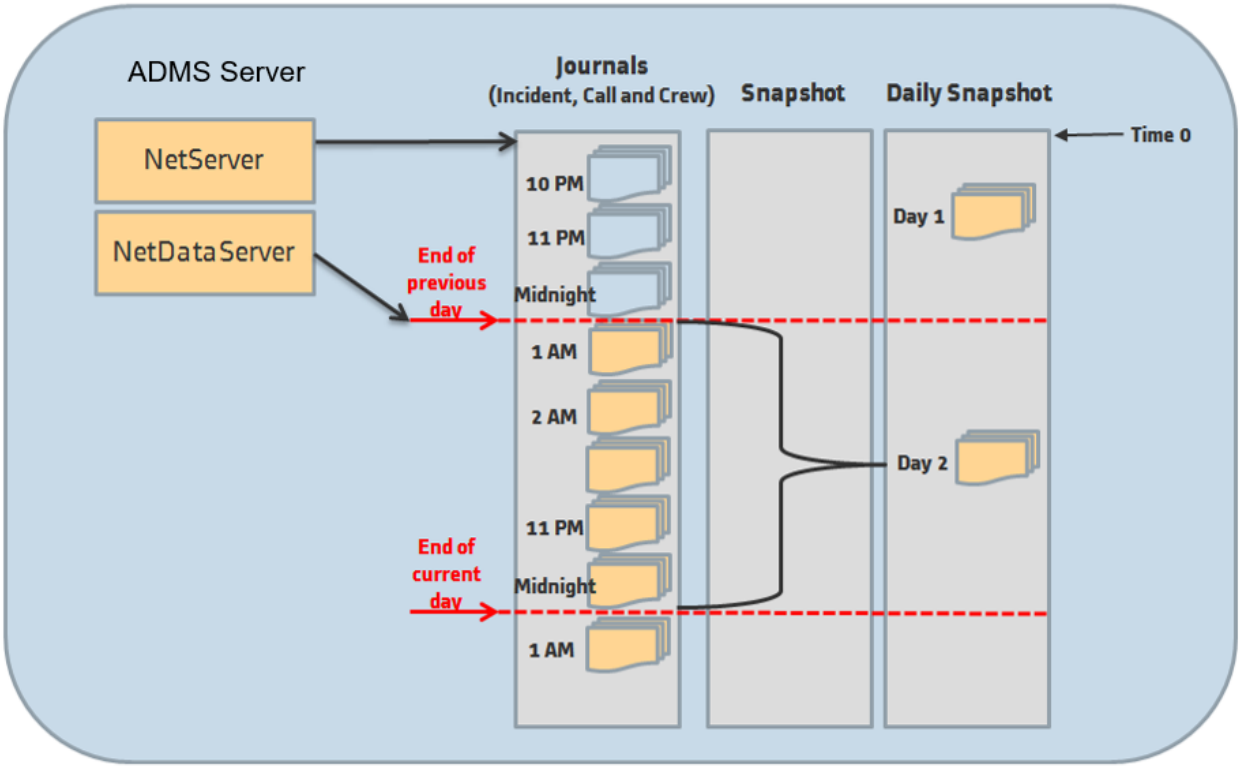


Figure 38. Creating Daily Snapshots

The Net Data server's startup after restart is slightly different than that of NetServer.

For example, Net Data server has not been running for the last 2 days. Upon startup, the Net Data server discovers that the last two daily snapshot files (Day -2 and Day -1) are missing. First it loads data from the most recent daily snapshot (Day -3), and then gets the active content for that day from journal files and creates the Day -2's daily snapshot. The Net Data server repeats this procedure to create all missing daily snapshots. When the snapshots are restored, the Net Data server loads the recent activity from the journal files and resumes its normal work.

The number of days of history data required can be set in the Net Data server configuration file.

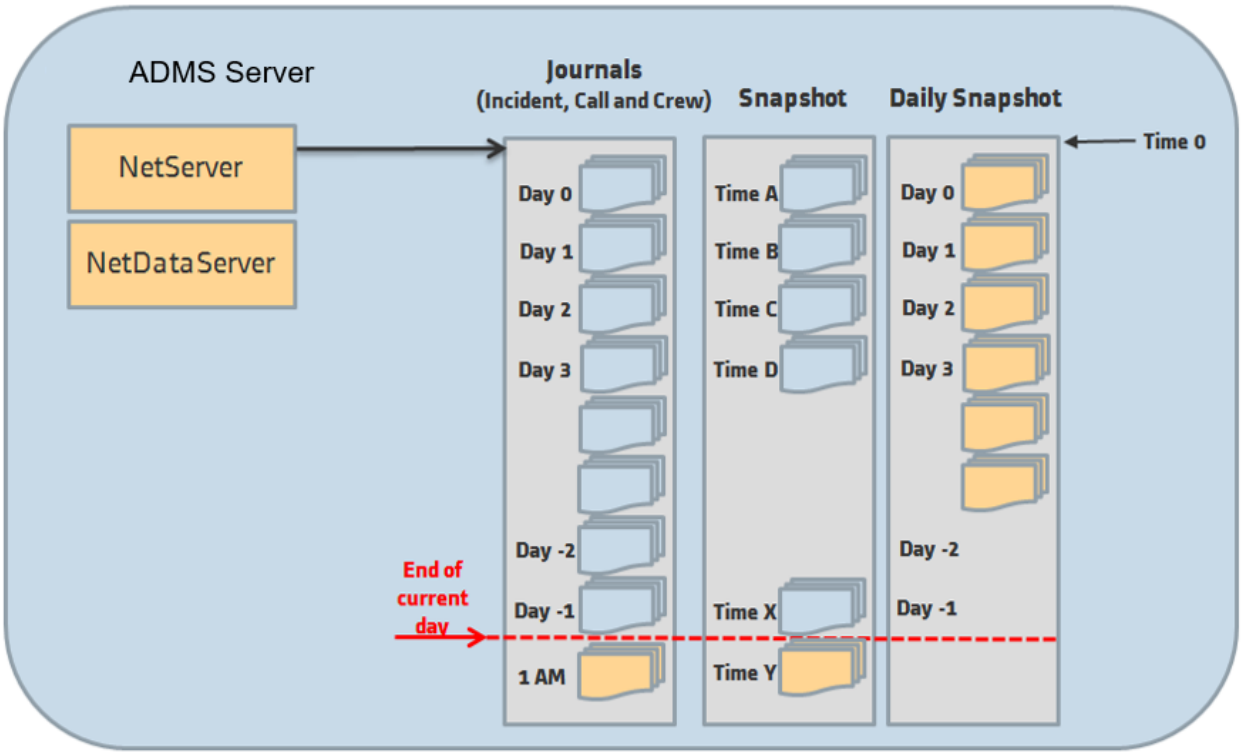


Figure 39. Restoring Daily Snapshots

7.4. Replicating OMS Data

Net Data Replicator is responsible for replicating OMS data and Switching Order files to other machines, such as the Standby server and Read-Only servers.

The enabled Net Data Replicator acts as a reader of journal, daily snapshot and switching order files. It monitors for the journal files exactly as the Net Data server.

The Net Data Replicator running on the enabled server acts as the "listener" and serves data to one or more "client" replicators on standby or read-only servers. Based on the last query time the listener replicator is asked to provide the recent journal entries for a given journal type.

All of the read-only and standby servers query for the previous days daily snapshots as the first step on startup. The next initialization step is to query for journal entries after midnight.

The OMS data replication procedure is provided below:

1. The client replicator requests a configurable number of daily snapshots.
2. The listener replicator sends the requested snapshots.
3. The client replicator saves the snapshots.
4. The client replicator requests the recent journal entries.
5. The listener replicator queries the journals on the enabled server for any new entries since the last time and sends them.

6. The client saves the received journal entries.
7. The standby NetServer reads in the last daily snapshot.

> **Note**
>
> Net Data Replicator does not replicate regular snapshots.

8. The standby NetServer starts querying journals for recent entries and receives them.
9. The standby Net Data server reads in the last daily snapshot.
10. The standby Net Data server starts querying journals for recent entries and receives them.
11. The client replicator periodically queries listener for new journal entries.
12. The listener replicator queries for new journal entries and returns them to the client.
13. The client replicator records new entries.
14. The standby NetServer and Net Data server detect the new journal entries.

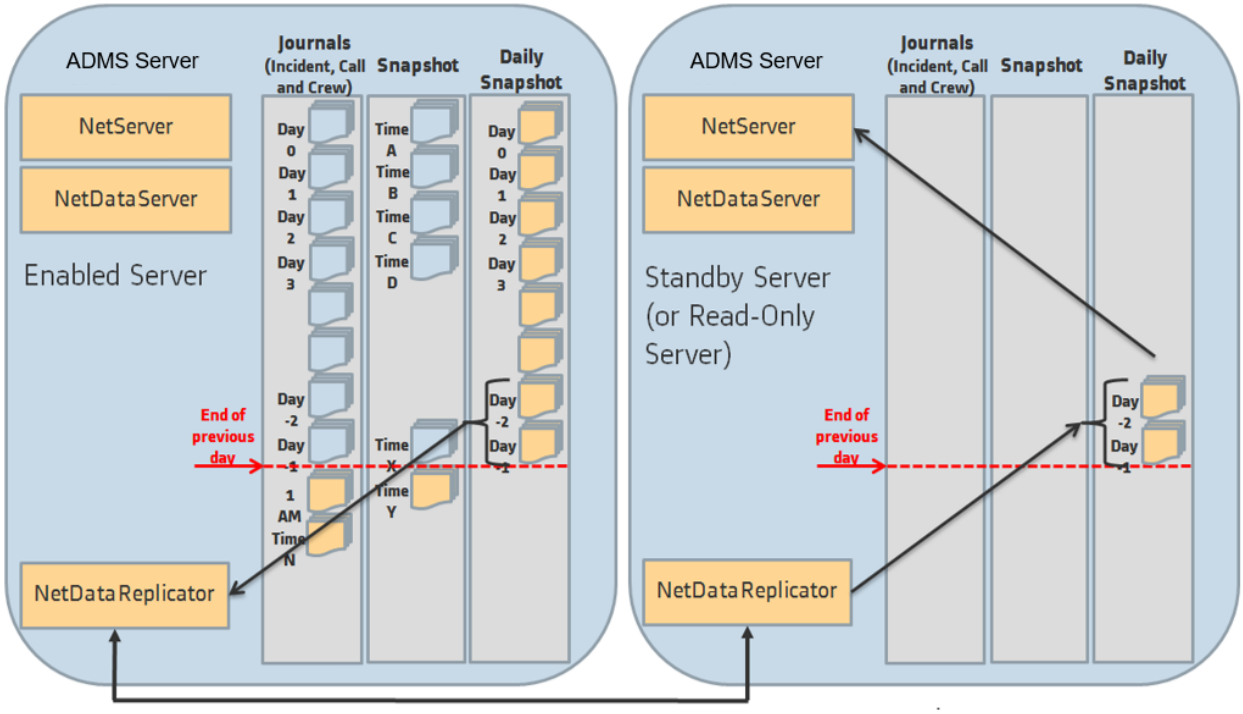


Figure 40. Replicating Daily Snapshots

> **Note**
>
> The standby NetServer does not create regular snapshots. The standby Net Data server does not create daily snapshots at midnight.